

Anthropic

Claude Certified Architect Foundations

Total Question 300

Question 1:

A customer support agent built with the Claude Agent SDK occasionally skips an identity verification step before processing refund requests, leading to unauthorized refunds in approximately 8% of interactions. The team has already added instructions to the system prompt emphasizing the importance of verification. What is the most effective way to ensure the verification step always occurs before refund processing?

- A. Implement a PreToolUse hook that blocks the process_refund tool until get_customer has returned a verified customer ID
- B. Add more emphatic language to the system prompt requiring verification before any financial operation
- C. Create a separate verification subagent that runs independently before the main support agent
- D. Configure tool_choice to force the get_customer tool on every interaction

Answer : A**Explanation:**

A PreToolUse hook provides programmatic enforcement that deterministically blocks tool execution until prerequisites are met. This is the correct approach because prompt-based guidance (Option B) has a non-zero failure rate — the exam guide explicitly states that prompts alone are insufficient when deterministic compliance is required for operations with financial consequences. Option C adds unnecessary complexity without guaranteeing ordering. Option D would force get_customer on every turn, not just before refunds, breaking the natural conversation flow.

Question 2:

A development team is building a multi-agent research system with a coordinator that delegates to three specialized subagents: `web_search`, `document_analysis`, and `synthesis`. During testing, the `synthesis` subagent produces reports that lack details from the `web_search` agent's findings. The coordinator correctly receives results from all subagents. What is the most likely cause of this issue?

- A. The coordinator is not passing the complete `web_search` findings in the `synthesis` subagent's prompt context
- B. The `synthesis` subagent's system prompt needs more explicit instructions about including all sources
- C. The `web_search` subagent is returning results in a format incompatible with the `synthesis` subagent
- D. The `synthesis` subagent needs access to the `web_search` tool to verify findings independently

Answer : A

Explanation:

Subagents have isolated contexts — they do NOT inherit the coordinator's conversation history or other subagents' results automatically. The coordinator must explicitly include complete findings from prior agents directly in each subagent's prompt. Option A might help marginally but doesn't address the root cause of missing information. Option C is unlikely since the coordinator receives results correctly. Option D violates the principle of scoped tool access and adds unnecessary complexity.

Question 3:

An architect is designing an agentic loop using the Claude API. The loop needs to handle tool calls, detect completion, and manage errors. Which combination of `stop_reason` values correctly identifies when to continue the loop versus when to exit?

- A. Continue on `'tool_use'` or `'end_turn'`; exit on `'max_tokens'` or `'refusal'`
- B. Continue on `'tool_use'` or `'max_tokens'`; exit on `'end_turn'` or `'stop_sequence'`
- C. Continue on `'tool_use'`; exit only on `'end_turn'`
- D. Continue on `'tool_use'`; exit on `'end_turn'`, `'max_tokens'`, `'refusal'`, or `'stop_sequence'`

Answer : D

Explanation:

The agentic loop pattern continues only when `stop_reason` is `'tool_use'`, meaning Claude wants to execute a tool. All other `stop_reason` values indicate the loop should exit: `'end_turn'` means Claude is done, `'max_tokens'` means output was truncated, `'refusal'` means the model declined, and `'stop_sequence'` means a custom stop was hit. Option B incorrectly continues on `'max_tokens'`, which indicates truncation, not a tool request. Option C ignores important error conditions. Option D reverses the logic for `'end_turn'`.

Question 4:

A team is building a code review system that generates reviews and then validates them. They notice that when the same Claude session generates code and then reviews it, the reviews are less critical than

when a separate session performs the review. What architectural change would most effectively address this bias?

- A. Add stronger instructions in the review prompt to be critical and thorough
- B. Use a separate Claude instance without the generation context to perform the review
- C. Implement a multi-pass review within the same session with explicit self-critique prompts
- D. Increase the temperature parameter during the review phase to encourage diverse perspectives

Answer : B

Explanation:

The exam guide explicitly states that a session retaining reasoning context from generation is less effective at self-review. An independent review instance without prior reasoning context provides more objective evaluation. Option A attempts to override the bias with instructions but doesn't address the fundamental context contamination. Option C keeps the same contaminated context. Option D doesn't address the root cause — temperature affects randomness, not objectivity.

Question 5:

A coordinator agent manages three research subagents investigating 'AI applications in creative industries.' After receiving reports, the coordinator finds all three subagents covered visual arts topics extensively but missed music composition, creative writing, and film production entirely. The subagents executed their assigned tasks correctly and thoroughly. What is the root cause of this gap?

- A. The subagents need larger context windows to cover more topics
- B. The subagents should have been given web search tools to discover additional creative domains
- C. The synthesis step should have identified and requested additional research on missing topics
- D. The coordinator's task decomposition was too narrow, limiting scope to visual arts subtopics

Answer : D

Explanation:

This matches the exam guide's sample question exactly. When subagents execute correctly within their assigned scope but the overall output has gaps, the problem is at the coordinator's task decomposition level. The coordinator assigned overly narrow subtopics all within visual arts instead of distributing across the full creative industries spectrum. Option A is irrelevant — context window size doesn't affect task scope. Option C addresses symptoms, not the root cause. Option D wouldn't help if the tasks themselves are scoped too narrowly.

Question 6:

An architect needs to implement a multi-agent system where two analysis paths explore different hypotheses from the same shared baseline data. The paths should be independent but start from identical context. Which Claude Code feature best supports this requirement?

- A. Creating two separate sessions and manually copying the baseline data into each
- B. Using `fork_session` to create independent branches from the shared analysis baseline
- C. Using agent teams with a shared task list to coordinate the parallel investigation
- D. Running both analyses sequentially in the same session with clear context separation prompts

Answer : B

Explanation:

`fork_session` is specifically designed for exploring divergent approaches from a shared baseline — it creates independent branches that start with identical context but can diverge. Option B achieves a similar result but requires manual duplication and risks inconsistencies. Option C (agent teams) is designed for collaborative work, not independent divergent exploration. Option D loses the benefits of parallel execution and risks context contamination between analyses.

Question 7:

A support agent system needs to handle multi-concern customer requests (e.g., a billing dispute AND a feature question in the same message). Currently, the agent addresses only the first concern and ignores subsequent ones. What is the recommended approach to handle this?

- A. Instruct the agent to process both concerns in sequence within a single response
- B. Route the message to two different specialized agents simultaneously
- C. Decompose the multi-concern request into distinct items and investigate each in parallel
- D. Ask the customer to submit separate tickets for each concern

Answer : C

Explanation:

The exam guide recommends decomposing multi-concern requests into distinct items that can be investigated in parallel. This ensures comprehensive coverage of all customer concerns. Option A risks the same attention dilution problem. Option B creates coordination overhead without proper decomposition. Option D provides a poor customer experience and doesn't solve the technical challenge.

Question 8:

When designing hooks for a Claude Agent SDK application, a developer wants to enforce a compliance rule that prevents refund processing for amounts exceeding \$500 without manager approval. Which hook configuration correctly implements this?

- A. A `PostToolUse` hook on `process_refund` that reverses transactions over \$500
- B. A `SessionStart` hook that sets a global \$500 limit for all financial operations
- C. A system prompt instruction specifying the \$500 threshold with examples of correct behavior

D. A PreToolUse hook on process_refund with exit code 2 and a JSON output containing a deny decision when the amount exceeds \$500

Answer : D

Explanation:

PreToolUse hooks intercept tool execution before it happens. Exit code 2 blocks the operation, and the JSON output can include a permissionDecision of 'deny' with feedback explaining why. This provides deterministic enforcement. Option A (PostToolUse) would only catch the issue after the refund is already processed. Option C relies on probabilistic prompt compliance. Option D is too broad and doesn't handle per-transaction logic.

Question 9:

A team is resuming a Claude Code session after making significant code changes outside the session. The agent starts providing recommendations based on the code state from the previous session. What is the most reliable approach to ensure accurate responses?

- A. Use --resume with the session ID and immediately ask Claude to re-read all changed files
- B. Fork the previous session and provide a diff of all changes made since the last interaction
- C. Use --continue to automatically load the most recent session with updated file context
- D. Start a fresh session with a structured summary of prior findings and the current code state

Answer : D

Explanation:

The exam guide states that starting fresh with a structured summary is more reliable than resuming with stale tool results. When code has changed significantly outside the session, the previous session's tool results and reasoning are based on outdated code. Option A attempts to patch stale context but the session's reasoning chain is still based on old code. Option C has the same issue. Option D adds complexity without addressing the fundamental staleness problem.

Question 10:

A developer is building a customer support agent and needs to decide between using adaptive decomposition versus a fixed sequential pipeline for handling support tickets. Tickets range from simple password resets (70% of volume) to complex multi-system outage investigations (5% of volume). Which approach is most appropriate?

- A. Fixed sequential pipeline — applies the same thorough process to every ticket for consistency
- B. A single adaptive agent with a mandatory initial classification step before any processing
- C. Two separate agents: one fixed pipeline for simple tickets, one adaptive for complex tickets
- D. Adaptive decomposition — generates subtasks dynamically based on what's discovered at each step

Answer : D

Explanation:

Adaptive decomposition is the better choice when task complexity varies significantly. It dynamically generates subtasks based on discoveries, allowing simple tickets to resolve quickly while complex ones trigger deeper investigation. Option A wastes resources on simple tickets. Option C adds operational complexity. Option D is close but unnecessarily constrains the system — adaptive agents can naturally adjust depth without a mandatory classification gate.

Question 11:

A team has configured an MCP server with 18 tools accessible to their support agent. The agent frequently selects incorrect tools, particularly confusing `get_order_status` with `get_order_details` and `check_shipping` with `track_package`. What is the most effective solution?

- A. Add retry logic that detects incorrect tool selection and re-routes to the correct tool
- B. Use `tool_choice` to force specific tools based on keywords detected in the customer message
- C. Reduce the tool count by consolidating overlapping tools and expanding descriptions with clear differentiators
- D. Train the agent with few-shot examples of correct tool selection for each disambiguation case

Answer : C

Explanation:

The exam guide emphasizes that too many tools (especially 18 vs a recommended 4-5 per agent) degrades tool selection reliability, and tools with minimal or overlapping descriptions cause misrouting. The solution is to consolidate related operations into fewer tools and expand descriptions with input formats, example queries, and boundary explanations. Option A treats symptoms, not causes. Option C bypasses Claude's reasoning. Option D may help but doesn't address the root structural problem.

Question 12 :

An MCP tool for processing customer returns accepts a JSON object with nested fields including `order_id`, `return_reason`, `item_conditions`, and `refund_method`. Agents frequently provide malformed inputs, especially for the nested `item_conditions` array. Which tool definition improvement would most effectively reduce input errors?

- A. Add validation logic inside the tool to reject and return helpful error messages for malformed inputs
- B. Include `input_examples` in the tool definition showing correctly structured nested objects
- C. Simplify the tool to accept only flat string parameters instead of nested objects
- D. Split the tool into separate tools for each parameter to eliminate nesting

Answer : B

Explanation:

The input_examples field in tool definitions is specifically designed for complex tools with nested objects. It provides concrete examples that show Claude exactly how to structure inputs, significantly reducing malformed calls. Option A handles errors after they occur instead of preventing them. Option B sacrifices functionality. Option D creates unnecessary tool proliferation, which itself degrades tool selection quality.

Question 13 :

A developer is configuring tool_choice for a document extraction pipeline. The first step must always call the extract_metadata tool, but subsequent steps should let Claude decide which tools to use based on document content. Which tool_choice configuration sequence is correct?

- A. Use {"type": "tool", "name": "extract_metadata"} for the first request, then switch to "auto" for subsequent requests
- B. Use "any" for the first request to guarantee a tool call, then "auto" for subsequent requests
- C. Use "auto" throughout and add instructions to always start with extract_metadata
- D. Use "none" for the first request to let Claude plan, then "auto" for execution

Answer : A

Explanation:

Forced tool selection ({"type": "tool", "name": "extract_metadata"}) guarantees the specific tool is called first, then switching to "auto" allows Claude to reason about subsequent tool choices. Option B guarantees some tool call but not necessarily extract_metadata. Option C relies on probabilistic prompt compliance. Option D prevents any tool use on the first turn, which is the opposite of the requirement.

Question 14 :

A multi-agent system's web search subagent encounters a timeout when querying an external API. The coordinator receives a generic 'Operation failed' status and terminates the entire research workflow. What changes should be made to the error handling?

- A. The coordinator should catch all errors and proceed without the web search results
- B. The subagent should automatically retry three times before reporting failure to the coordinator
- C. The subagent should return structured error context including failure type, attempted query, partial results, and alternative approaches
- D. The system should fall back to a cached version of search results from previous queries

Answer : C

Explanation:

The exam guide emphasizes returning structured error context with failure type, attempted query, partial results, and alternatives. This enables the coordinator to make informed recovery decisions — such as retrying, delegating to another subagent, or proceeding with partial results. Option B may help for transient errors but doesn't give the coordinator decision-making context. Option C silently suppresses errors (an anti-pattern). Option D may return stale or irrelevant results.

Question 15:

A team needs to share MCP server configurations across all developers while allowing individual developers to also use personal experimental MCP servers. Where should each configuration be placed?

- A. Team config in CLAUDE.md; personal config in ~/.claude/CLAUDE.md
- B. Team config in .mcp.json at project root; personal config in ~/.claude.json
- C. Team config in .claude/settings.json; personal config in env variables
- D. Both configs in .mcp.json with conditional loading based on the developer's username

Answer : B

Explanation:

Project-level MCP configuration goes in .mcp.json (version controlled, shared with the team), while user-level personal configurations go in ~/.claude.json. This separation ensures team tooling is consistent while allowing individual experimentation. Option B uses CLAUDE.md which is for instructions, not MCP server configs. Option C and D use non-standard configuration paths.

Question 16:

A Claude Code user needs to configure different coding conventions for React component files versus API route files versus database migration files, where each file type is scattered across multiple directories in a monorepo. What is the most maintainable approach?

- A. Use .claude/rules/ directory with YAML frontmatter containing glob patterns for each file type
- B. Create separate CLAUDE.md files in each directory containing these file types
- C. Put all conventions in the root CLAUDE.md with clear section headers for each file type
- D. Create custom slash commands for switching between convention modes

Answer : A

Explanation:

The .claude/rules/ directory with YAML frontmatter path-scoping using glob patterns (e.g., `**/*.tsx` for components, `**/api/**/*.ts` for routes) loads rules only when editing matching files. This reduces irrelevant context and works across directories. Option A requires duplicating CLAUDE.md files and doesn't work for files scattered across many directories. Option C loads all conventions regardless of which file is being edited. Option D requires manual intervention.

Question 17:

A development team wants to create a reusable skill for Claude Code that performs database schema migrations. The skill should run in an isolated context to prevent verbose migration logs from polluting

the main conversation, and it should only have access to the Bash and Read tools. Which SKILL.md frontmatter configuration achieves this?

- A. context: fork, allowed-tools: [Bash, Read], disable-model-invocation: true
- B. context: fork, allowed-tools: [Bash, Read]
- C. isolation: true, tools: [Bash, Read], disable-model-invocation: true
- D. context: isolated, allowed-tools: [Bash, Read, Write]

Answer : B

Explanation:

The correct SKILL.md frontmatter uses 'context: fork' for isolated sub-agent context (preventing output pollution) and 'allowed-tools' to restrict tool access. Option A adds disable-model-invocation which prevents the skill from being triggered by the model, but the question doesn't require this restriction. Option C uses non-existent 'isolation' and 'tools' fields. Option D uses 'context: isolated' which is not a valid value (it should be 'fork'), and includes Write which wasn't specified.

Question 18:

An architect is designing a structured data extraction system that processes invoices into a specific JSON schema. Some invoices don't contain all fields (e.g., purchase order numbers are sometimes absent). How should the JSON schema handle potentially missing fields to prevent Claude from fabricating values?

- A. Make all fields required and instruct Claude to use 'N/A' for missing values
- B. Use enum fields with an 'unknown' option for fields that might be missing
- C. Make potentially absent fields nullable in the schema so Claude can return null instead of guessing
- D. Implement post-processing validation to detect and remove likely fabricated values

Answer : C

Explanation:

Making fields nullable allows Claude to explicitly indicate when information is absent from the source document, preventing fabrication. This is a recommended schema design pattern from the exam guide. Option A forces Claude to provide a string value even when information is genuinely missing. Option C misuses enums for presence/absence handling. Option D addresses the problem after fabrication occurs instead of preventing it.

Question 19:

A team is using Claude's Message Batches API to process overnight compliance reports (non-blocking) and also wants to use it for pre-merge code review checks in their CI pipeline. A manager suggests using batch processing for both to save costs. What is the correct recommendation?

- A. Use batch processing for the overnight reports only; use real-time API calls for pre-merge checks
- B. Use batch processing for both — the 50% cost savings justifies the slight delay in CI feedback

- C. Use real-time API for both — batch processing is only for workloads exceeding 100K requests
- D. Use batch processing for pre-merge checks only since they benefit more from structured output

Answer : A

Explanation:

The Message Batches API has no guaranteed latency SLA and a processing window of up to 24 hours. This makes it appropriate for non-blocking, latency-tolerant workloads like overnight reports, but unsuitable for blocking CI pipeline checks where developers need timely feedback. Option B would cause unacceptable delays in the development workflow. Option C incorrectly states batch processing requires 100K+ requests. Option D reverses the appropriate use cases.

Question 20:

A code review tool produces findings with high false positive rates, particularly flagging correct error handling as 'missing error handling' and marking intentional type assertions as 'unsafe casts.' The team has already added general instructions to 'be more conservative.' What approach would most effectively reduce false positives?

- A. Add a confidence threshold that suppresses findings below 90% confidence
- B. Increase the context window to give Claude more surrounding code for better analysis
- C. Provide 2-4 few-shot examples showing the specific false positive patterns and the correct non-flagging behavior
- D. Use tool_choice to force Claude to use a specific static analysis tool instead of relying on its own judgment

Answer : C

Explanation:

The exam guide states that few-shot prompting is the most effective technique for consistent, actionable output, especially for ambiguous scenarios. Targeted examples showing the specific false positive patterns and correct behavior enable Claude to generalize to novel patterns. General instructions like 'be conservative' are explicitly noted as failing to improve precision. Option B adds tokens but doesn't address the reasoning error. Option C relies on self-reported confidence which is poorly calibrated. Option D removes Claude's reasoning capability entirely.

Question 21:

A team is designing a CI/CD pipeline that uses Claude Code for automated code review. The pipeline currently hangs during execution because Claude is waiting for interactive input. What is the correct fix?

- A. Set the CLAUDE_CI=true environment variable to enable headless mode
- B. Add the -p flag to run Claude Code in non-interactive mode

- C. Redirect stdin from /dev/null to prevent interactive prompts
- D. Use the --daemon flag to run Claude Code as a background process

Answer : B

Explanation:

The -p (or --print) flag is the documented way to run Claude Code in non-interactive mode, specifically designed for CI pipelines and scripted usage. It processes the prompt and exits without waiting for interactive input. Options B, C, and D reference non-existent features or workarounds that don't address the root cause.

Question 22:

During a long debugging session in Claude Code, the architect notices the agent starts giving vague, generic responses like 'this follows typical patterns' instead of referencing specific classes and methods it examined earlier. What is happening and what should the architect do?

- A. Context degradation from an extended session — use /compact and persist key findings in scratchpad files
- B. The model's temperature is increasing over time — restart with a lower temperature setting
- C. The model is rate-limited — wait a few minutes before continuing the debugging session
- D. The agent needs more explicit instructions — add detailed debugging steps to the system prompt

Answer : A

Explanation:

The exam guide specifically describes context degradation in extended sessions where inconsistent answers and 'typical patterns' language replace specific references. The solution is to use /compact to reduce context usage and persist key findings in scratchpad files that survive context boundaries. Option A has no basis — temperature doesn't increase during a session. Option C would not affect response quality. Option D doesn't address the root cause of overloaded context.

Question 23:

A customer support agent resolves 55% of tickets but frequently escalates simple password reset requests to human agents while handling complex billing disputes autonomously. The team wants to improve escalation accuracy. Which approach best addresses this calibration issue?

- A. Add a confidence score threshold where the agent self-reports certainty below 70% and escalates
- B. Implement a machine learning classifier to determine escalation decisions outside of Claude
- C. Add explicit escalation criteria with few-shot examples showing when to escalate versus resolve autonomously
- D. Restrict the agent to only handle password resets and escalate everything else

Answer : C

Explanation:

This matches the exam guide's sample question directly. Explicit escalation criteria combined with few-shot examples showing correct behavior for each scenario type is the most effective solution. Option B adds unnecessary engineering complexity. Option C relies on self-reported confidence scores, which the exam guide explicitly states are poorly calibrated proxies. Option D drastically limits the agent's utility.

Question 24:

A data extraction system processes invoices from multiple vendors. The team discovers that while overall accuracy is 97%, invoices from one specific vendor have a 23% error rate on the 'tax_amount' field. Their current evaluation approach checks aggregate accuracy across all vendors. What is the most effective improvement to their evaluation methodology?

- A. Lower the overall accuracy threshold from 97% to 90% to account for the problematic vendor
- B. Add more training examples from the problematic vendor to the system prompt
- C. Create a separate extraction pipeline specifically for the problematic vendor
- D. Implement stratified sampling that measures accuracy by document type AND by field segment

Answer : D

Explanation:

The exam guide emphasizes that aggregate accuracy can mask poor performance on specific types. Stratified random sampling by document type AND field segment reveals these hidden issues. Option A lowers standards without fixing the problem. Option B may help but doesn't address the evaluation gap. Option D is over-engineering a solution without understanding the full scope of issues across other vendors.

Question 25:

An architect is implementing an agentic loop and needs to handle tool results correctly. After Claude responds with a tool_use block, the architect must send the tool result back. Which message format is correct?

- A. A user message with a tool_result content block containing the tool_use_id and content, placed before any text blocks
- B. A separate tool message role with the tool name, result content, and a reference to the tool call ID
- C. An assistant message with a tool_result content block appended after the tool_use block
- D. A user message with the tool result as a plain text string referencing the tool name

Answer : A

Explanation:

Tool results must be sent as user messages containing `tool_result` content blocks with the `tool_use_id` for correlation. Critically, `tool_result` blocks must come FIRST in the content array, before any text blocks — violating this ordering causes a 400 error. Option B references a non-existent 'tool' message role (Claude's API uses only 'user' and 'assistant' roles). Option C incorrectly places results in an assistant message. Option D lacks the structured format required for tool result correlation.

Question 26:

A team building a document processing pipeline needs Claude to output structured JSON conforming to a specific schema. They've tried using prompt instructions alone but occasionally receive malformed JSON with missing required fields. What is the most reliable approach to guarantee schema-compliant output?

- A. Add JSON formatting examples in the system prompt with explicit field requirements
- B. Use `tool_use` with a JSON schema definition and `tool_choice` set to force the extraction tool
- C. Post-process Claude's output with a JSON parser that adds default values for missing fields
- D. Use `output_config.format` with type `'json_schema'` and strict mode enabled

Answer : D

Explanation:

The structured outputs feature (`output_config.format` with type `'json_schema'`) provides guaranteed schema-compliant output through grammar-based generation. With strict mode enabled, responses are guaranteed to conform to the schema. Option A relies on prompt compliance which is probabilistic. Option B (`tool_use`) also provides schema compliance but at the tool call level, not the response level. Option C fixes problems after they occur. Option D is the most direct approach for response-level structured output.

Question 27:

An architect is configuring a Claude Code project that uses a monorepo with frontend (React), backend (Express), and shared library packages. The `CLAUDE.md` configuration needs to provide context-specific instructions. What is the optimal configuration approach?

- A. Single root `CLAUDE.md` with all instructions organized by package name in separate sections
- B. Root `CLAUDE.md` with universal standards plus subdirectory `CLAUDE.md` files for package-specific conventions
- C. Separate `.claude/rules/` files with glob patterns targeting each package's file types
- D. User-level `~/.claude/CLAUDE.md` with all project-specific instructions for personal use

Answer : B

Explanation:

The `CLAUDE.md` hierarchy is designed for exactly this pattern: root-level `CLAUDE.md` for universal standards (coding style, commit messages, testing requirements) plus subdirectory `CLAUDE.md` files

that add package-specific context when working in those directories. Option A loads all instructions regardless of context. Option C works for file-type conventions but not package-level context. Option D is user-level and not shared with the team.

Question 28:

A synthesis subagent in a research system needs occasional access to fact-checking capabilities. Currently, 85% of fact-checks are simple lookups that the synthesis agent could handle independently, while 15% require complex verification through the coordinator. What is the recommended tool distribution approach?

- A. Remove all fact-checking from the synthesis agent and route everything through the coordinator
- B. Give the synthesis agent full access to all research tools to minimize coordinator bottlenecks
- C. Give the synthesis agent a scoped `verify_fact` tool for simple lookups; complex checks route through the coordinator
- D. Create a dedicated fact-checking subagent that handles all verification requests regardless of complexity

Answer : C

Explanation:

This matches the exam guide's sample question: the principle of least privilege recommends giving each agent only the tools it needs for its primary role plus scoped cross-role tools for high-frequency needs. A scoped `verify_fact` tool handles the common 85% while complex cases still go through the coordinator. Option B violates least privilege. Option C creates unnecessary bottleneckson simple lookups. Option D adds coordination overhead for simple lookups.

Question 29:

A developer is building an agentic system and wants to spawn multiple research subagents simultaneously to investigate different aspects of a complex question. How should the coordinator trigger parallel subagent execution?

- A. Make sequential Task tool calls, one for each subagent, with a 'parallel: true' flag
- B. Configure the system prompt with a parallelism parameter specifying concurrent agent count
- C. Use the batch processing API to submit all subagent requests simultaneously
- D. Emit multiple Task tool calls in a single coordinator response to the API

Answer : D

Explanation:

The correct pattern for parallel subagent spawning is to emit multiple Task tool calls in a single coordinator response. Claude naturally supports parallel tool calling — when the coordinator's response contains multiple `tool_use` blocks, they can be executed simultaneously. Option A uses a non-existent

'parallel' flag. Option C uses batch API which is for independent requests, not coordinated subagents. Option D references a non-existent configuration parameter.

Question 30:

A team is evaluating whether to use subagents or agent teams for a complex codebase migration project that requires investigation of the legacy system, design of the new architecture, implementation of changes, and writing tests. The tasks have significant interdependencies and team members need to discuss findings. Which option is most appropriate?

- A. Use agent teams — teammates can share findings, claim tasks from a shared list, and communicate directly
- B. Use subagents — each handles one phase and reports results back to the coordinator
- C. Use a single agent session with extended context to maintain coherence across all phases
- D. Use prompt chaining with separate sessions for each phase, passing summaries between them

Answer : A

Explanation:

Agent teams are designed for complex work requiring discussion and coordination between agents. The shared task list, mailbox messaging, and ability for teammates to claim work and communicate directly makes agent teams ideal for interdependent tasks. Option A limits communication to only report-back to coordinator. Option C risks context degradation in a very long session. Option D loses inter-phase context and prevents real-time coordination.

Question 31:

When configuring the effort parameter for different components of a multi-agent system, which combination of effort levels is most appropriate?

- A. max for coordinator reasoning, high for primary analysis agents, low for simple data-fetching subagents
- B. high for all agents to ensure consistent quality across the system
- C. low for the coordinator to minimize costs, max for analysis subagents to maximize depth
- D. medium for all agents with manual temperature adjustments for different task types

Answer : A

Explanation:

The effort parameter should be tuned per agent role: max/high for complex reasoning (coordinator decisions, analysis), and low for simple focused tasks (data fetching, formatting) where speed and cost efficiency matter more than deep reasoning. Option B wastes tokens on simple tasks. Option C inverts priorities — the coordinator needs the most reasoning capability. Option D ignores task-specific optimization opportunities, and temperature is a separate concept.

Question 32:

A data pipeline extracts financial records from documents. The system uses a validation-retry loop where Claude retries extraction when validation fails. After three retries, the 'total_amount' field still fails validation because the extracted values don't sum to the stated total. The individual line items are correct. What is the most likely issue?

- A. The retry loop needs more iterations to converge on the correct answer
- B. The schema needs to be simplified to reduce extraction complexity
- C. The retries are ineffective because the discrepancy is in the source document, not Claude's extraction
- D. Claude needs a higher effort level to perform accurate arithmetic during extraction

Answer : C

Explanation:

The exam guide states that retries are ineffective when information is absent from or incorrect in the source document. If individual line items are correct but totals don't sum, the issue is likely a discrepancy in the original document itself. More retries won't fix a source data problem. Option A assumes retries will eventually succeed, which isn't true for source errors. Option C changes the schema unnecessarily. Option D won't help with source data discrepancies.

Question 33:

A research agent compiles a report from multiple sources but loses source attribution during the summarization step. The final report contains statistics and claims without any indication of which source they came from, and some conflicting data points are silently resolved by choosing one value. What design change addresses both issues?

- A. Add post-processing that uses web search to verify and re-attribute all claims in the report
- B. Require subagents to return structured data with claim-source mappings, and annotate conflicting statistics with attribution rather than arbitrarily selecting one
- C. Increase the context window to prevent information loss during summarization
- D. Store all raw sources in a database and link report claims to source IDs

Answer : B

Explanation:

The exam guide emphasizes that source attribution is lost during summarization without claim-source mappings, and conflicting statistics should be annotated with attribution rather than arbitrarily selected. Structured data with claim-source mappings at the subagent level preserves provenance through the entire pipeline. Option B is reactive and unreliable. Option C doesn't address the structural problem. Option D requires significant infrastructure without solving the summarization design issue.

Question 34:

A developer is using Claude's API to build an agentic loop. They notice that after Claude makes a tool call, the response includes both a text content block and a tool_use content block. When sending the tool result back, the developer places a text block before the tool_result block in the user message. The API returns a 400 error. What is wrong?

- A. Text blocks are not allowed in messages that contain tool_result blocks
- B. The tool_result should be sent in a separate message from any text content
- C. Tool_result blocks must come FIRST in the content array of the user message, before any text blocks
- D. The developer needs to include the original tool_use block in the tool_result message

Answer : C

Explanation:

The API requires that tool_result blocks come first in the content array of user messages, before any text blocks. Violating this ordering results in a 400 error. This is a critical formatting rule documented in the handle tool calls guide. Option A is incorrect — text blocks are allowed alongside tool_result blocks, just not before them. Option C is unnecessary — they can coexist in the same message. Option D is not required in the user message.

Question 35:

A team wants their Claude Code skill for database migrations to accept a migration name as a parameter and use it within the skill's instructions. Which SKILL.md feature enables this?

- A. A required-params field in the SKILL.md frontmatter listing expected parameters
- B. A custom input_schema JSON definition in the skill frontmatter
- C. Environment variables set before skill invocation using the shell frontmatter field
- D. The \$ARGUMENTS substitution variable in the skill content, with argument-hint in frontmatter to prompt for the parameter

Answer : D

Explanation:

SKILL.md supports string substitutions including \$ARGUMENTS (the full argument string) and \$N (positional arguments). The argument-hint frontmatter field prompts the user for required parameters when not provided. Option B references a non-existent field — skills don't use input_schema. Option C's shell field is for shell setup, not parameter passing. Option D references a non-existent field.

Question 36:

An MCP tool returns a transient timeout error on the first call but the agent's error handling treats all errors identically, causing it to immediately abandon the operation and report failure to the user. How should the tool's error response be improved to enable smarter recovery?

- A. Return structured error metadata including `errorCategory`, `isRetryable` boolean, and a description distinguishing transient from permanent failures
- B. Return a generic `is_error: true` response with a human-readable error message
- C. Implement automatic retry logic within the tool itself, hiding transient errors from the agent
- D. Log all errors to an external monitoring system and return success with empty results

Answer : A

Explanation:

The exam guide requires structured error metadata distinguishing transient (retryable) from permanent (non-retryable) failures. Including `errorCategory` and `isRetryable` enables the agent to make informed recovery decisions — retrying transient errors while escalating permanent ones. Option A provides no recovery guidance. Option C hides useful information from the agent. Option D misrepresents errors as success, which is an anti-pattern.

Question 37:

A developer builds a Claude Code skill that performs comprehensive security audits. During execution, the skill produces verbose logging output that fills the main conversation context and causes subsequent interactions to lose important earlier context. What skill configuration would prevent this?

- A. Set `max_tokens` in the skill frontmatter to limit output length
- B. Add `logging: false` to suppress verbose output from tool executions
- C. Configure `context: fork` in the `SKILL.md` frontmatter to run in an isolated sub-agent context
- D. Use the Explore subagent instead of a custom skill for verbose operations

Answer : C

Explanation:

The `context: fork` frontmatter field runs the skill in an isolated sub-agent context, preventing verbose output from polluting the main conversation. Only the final results are returned to the main context. Option A is not a valid `SKILL.md` frontmatter field. Option C is a non-existent configuration. Option D replaces the custom skill entirely rather than fixing the configuration.

Question 38:

An architect needs to understand which Claude Code memory files are currently loaded in a session and troubleshoot why certain instructions seem to be ignored. Which command provides this information?

- A. `/status` to show all active configurations and their sources
- B. `/config` to display the complete resolved configuration hierarchy
- C. `/memory` to verify which memory files are loaded in the current session

D. /debug to show instruction precedence and conflict resolution

Answer : C

Explanation:

The /memory command shows which memory files (CLAUDE.md files at various levels) are loaded in the current session, helping debug why instructions may not be taking effect. Options A, C, and D reference non-existent Claude Code commands.

Question 39:

A long-running customer support agent conversation is approaching the context window limit. The system needs to continue operating without losing critical transaction details like order numbers, refund amounts, and customer commitments made earlier in the conversation. What is the recommended approach?

- A. Increase the max_tokens parameter to expand the available context window
- B. Extract transactional facts into a persistent 'case facts' block and use compaction to summarize older conversation context
- C. Start a new conversation and ask the customer to repeat their issue
- D. Enable streaming to reduce the memory footprint of the conversation

Answer : B

Explanation:

The exam guide recommends extracting transactional facts (dates, amounts, commitments) into a persistent structured block that survives summarization, combined with compaction to reduce older conversational context. This preserves critical details while freeing context space. Option A is incorrect — max_tokens controls output length, not context window size. Option C provides terrible customer experience. Option D doesn't affect context window usage.

Question 40:

A team is configuring the Explore subagent in Claude Code for a large codebase investigation. They want to understand the system architecture before making changes. What are the key characteristics of the Explore subagent?

- A. It uses Haiku (lightweight model) with read-only access, isolates verbose output, and returns summaries
- B. It uses a high-capability model with full write access for comprehensive exploration
- C. It runs in the same context as the main agent but with enhanced search capabilities
- D. It spawns multiple parallel sub-agents to explore different parts of the codebase simultaneously

Answer : A

Explanation:

The Explore subagent is a built-in agent that uses a lightweight model (Haiku) with read-only access. It isolates verbose discovery output from the main conversation and returns only summaries, preserving the main context. Option A incorrectly describes the model and access level. Option C doesn't isolate output. Option D describes a different pattern — Explore is a single focused agent.

Question 41:

A developer is building a multi-agent system where the coordinator needs to pass research findings from a web_search subagent to a synthesis subagent. The web_search agent returns a large amount of raw data including full page contents, URLs, timestamps, and metadata. What is the best practice for structuring this data transfer?

- A. Use structured data formats that separate content from metadata, including source URLs and page numbers
- B. Pass the complete raw data to preserve information fidelity
- C. Have the coordinator summarize findings into a brief text paragraph before passing to synthesis
- D. Store raw data in a shared database and pass only record IDs to the synthesis agent

Answer : A

Explanation:

The exam guide recommends structured data formats separating content from metadata (source URLs, document names, page numbers). This preserves information provenance while organizing data for efficient consumption. Option A bloats context with unnecessary raw data. Option C risks losing detail during summarization. Option D adds unnecessary infrastructure complexity and the synthesis agent can't access the database.

Question 42:

When using extended thinking with Claude and tool use, an architect strips all thinking blocks from the conversation before sending the next request. The API returns an error. What rule was violated?

- A. Thinking blocks must always be preserved in the full conversation history
- B. Thinking blocks can only be removed after the entire conversation ends
- C. Extended thinking is not compatible with tool use and should be disabled during tool calls
- D. Thinking blocks must be returned with their corresponding tool results; they can be dropped only after tool results are processed

Answer : D

Explanation:

When tool use and thinking are active together, thinking blocks **MUST** be returned alongside their corresponding tool results — this is the only case where thinking blocks must be preserved. After the

tool results have been processed by the model, thinking blocks from previous turns can be dropped. Option A is overly strict. Option C is incorrect — thinking works with tool use. Option D is also overly strict.

Question 43:

An API-based agent harness is managing a long conversation that's consuming over 150,000 input tokens. The team wants to automatically summarize older context while preserving recent interactions. Which Claude API feature handles this?

- A. Token limit parameter that triggers automatic message truncation at 150K tokens
- B. Automatic prompt caching that compresses repeated content across requests
- C. Client-side sliding window that drops the oldest messages when context exceeds 150K tokens
- D. Server-side compaction with the compact_20260112 beta feature, configured with a trigger threshold

Answer : D

Explanation:

Server-side compaction (beta feature compact_20260112) automatically summarizes conversation context when input tokens exceed a configurable trigger threshold (default 150K, minimum 50K). It creates a compaction block containing the summary and subsequent requests ignore all content before the last compaction block. Option A doesn't exist. Option C is a manual approach that risks losing important context. Option D reduces cost but doesn't manage context size.

Question 44:

A developer needs to configure an MCP server that requires API tokens for authentication. The tokens should be kept out of version control but available to all team members. How should this be configured in .mcp.json?

- A. Hardcode the API tokens directly in .mcp.json since it's a configuration file
- B. Use environment variable expansion syntax like \${API_TOKEN} in .mcp.json, with each developer setting the variable locally
- C. Store tokens in a separate .mcp-secrets.json file that's gitignored
- D. Configure tokens at the user level in ~/.claude.json and reference them from .mcp.json

Answer : B

Explanation:

.mcp.json supports environment variable expansion using \${VARIABLE_NAME} syntax. This allows the configuration file to be version controlled while keeping actual secret values in each developer's local environment. Option A exposes secrets in version control. Option C uses a non-standard pattern. Option D doesn't support cross-referencing between configuration files.

Question 45:

A team is building a review system for a large 14-file pull request. Initial results show inconsistent review depth across files, missed bugs in complex files, and contradictory feedback between files. What architectural approach best addresses these issues?

- A. Assign one file per subagent for parallel review with no cross-file coordination
- B. Increase the context window to include all 14 files simultaneously for comprehensive review
- C. Split into per-file local analysis passes followed by a separate cross-file integration pass
- D. Use a single long review session with explicit instructions to maintain consistent depth

Answer : C

Explanation:

This matches the exam guide's sample question directly: splitting into per-file local analysis plus a separate cross-file integration pass addresses the root cause of attention dilution. Local passes ensure thorough analysis of each file, while the integration pass catches cross-file issues. Option B doesn't solve attention dilution. Option C misses cross-file issues entirely. Option D perpetuates the same attention dilution problem.

Question 46:

A prompt engineer is designing a system prompt for a Claude-powered code review tool. They need Claude to check for security vulnerabilities, code style issues, and documentation completeness. They've found that using natural language paragraphs leads to inconsistent rule application. What structuring technique would most improve consistency?

- A. Use XML tags to structure different concern areas with explicit criteria for each
- B. Number each rule and add a priority ranking from 1 to 10
- C. Write all rules as a bulleted list in a single paragraph for easy scanning
- D. Create separate system prompts for each concern and run three independent reviews

Answer : A

Explanation:

The Claude prompting best practices recommend using XML tags (like `<security_rules>`, `<style_rules>`, `<documentation_rules>`) to structure different concern areas, reducing misinterpretation and improving consistent rule application. Option A provides ordering but not structural separation. Option C is still unstructured. Option D triples costs without improving within-review consistency.

Question 47:

A customer support agent encounters a customer who asks to speak with a human manager after one unsuccessful troubleshooting attempt. The agent has additional troubleshooting steps it could try. What should the agent do?

- A. Attempt the remaining troubleshooting steps since the agent may be able to resolve the issue
- B. Honor the customer's explicit request for a human representative immediately
- C. Ask the customer if they'd like to try one more troubleshooting step before escalating
- D. Escalate but first run diagnostic checks to provide the human agent with context

Answer : B

Explanation:

The exam guide states that explicit customer requests for a human should be honored immediately — this is a hard escalation trigger. While the agent may have additional capabilities, overriding the customer's explicit preference damages trust. Option A ignores the customer's stated preference. Option C delays the requested escalation. Option D partially complies but still introduces delay.

Question 48:

A developer is using the Claude Agent SDK (Python) and wants to load project-level configurations including skills, CLAUDE.md memory, and custom commands. How should they configure the query() call?

- A. Pass config_path pointing to the project's .claude directory
- B. Use the --settings CLI flag to point to the project configuration files
- C. Manually read and inject CLAUDE.md content into the system prompt
- D. Set setting_sources=['project'] in the ClaudeAgentOptions to load project configuration

Answer : D

Explanation:

The Agent SDK uses setting_sources=['project'] in ClaudeAgentOptions to automatically load project-level configurations including skills, slash commands, memory (CLAUDE.md), and plugins from the project's .claude directory. Option A uses a non-existent parameter. Option C is manual and misses skills/commands. Option D is a CLI flag, not an SDK configuration.

Question 49:

A team deploys a Claude-based agent that processes both real-time customer interactions and large batch reports. For the batch reports, they observe that Claude sometimes generates very detailed thinking traces that increase processing time and costs significantly, even for straightforward data extraction tasks. What configuration helps optimize this?

- A. Disable extended thinking entirely for batch processing to maximize throughput

- B. Set effort to 'low' for batch extraction tasks, allowing Claude to dynamically reduce thinking depth on simple items
- C. Reduce the context window size to force shorter thinking traces
- D. Set max_tokens to a low value to truncate lengthy thinking traces

Answer : B

Explanation:

The effort parameter at 'low' is a behavioral signal that makes Claude more efficient — it reduces thinking depth, tool calls, and preamble especially on simple tasks. Claude can still engage deeper thinking when genuinely needed. Option A completely removes the benefit of thinking on complex items. Option C doesn't directly affect thinking trace length. Option D truncates output, not thinking traces specifically.

Question 50:

An agentic system uses prompt caching to reduce costs on repeated API calls. The developer notices that cache hits drop to 0% whenever they change the model's tool definitions, even though the rest of the prompt is identical. What explains this behavior?

- A. Tool definitions are not included in the cache computation, so this must be caused by something else
- B. Tool definitions are at the top of the cache hierarchy; changing them invalidates the tools, system, and messages layers
- C. Prompt caching requires a minimum of 4096 tokens for tool definitions to be cached
- D. The cache TTL expired between the tool definition change and the next API call

Answer : B

Explanation:

The prompt cache hierarchy is: tools → system → messages. Changes at any level invalidate that level AND all subsequent levels. Since tool definitions are at the top, changing them invalidates everything, including cached system prompts and messages. Option A is incorrect — tools are part of the cache. Option B applies minimum token thresholds to system/messages, not as a tool-specific rule. Option D is possible but the pattern of 0% hits correlated with tool changes points to hierarchy invalidation.

Question 51:

A Claude-powered extraction system processes legal contracts. The team uses a JSON schema with required fields for 'effective_date', 'termination_clause', and 'governing_law'. Some contracts are drafts that lack termination clauses entirely. When processing these drafts, Claude fabricates plausible-looking termination clauses. What schema design change prevents this fabrication?

- A. Add a 'draft_status' boolean field to the schema to indicate incomplete contracts
- B. Make 'termination_clause' nullable so Claude can return null instead of fabricating content
- C. Remove 'termination_clause' from required fields and make it optional

D. Add validation in post-processing to detect and flag potentially fabricated clauses

Answer : B

Explanation:

Making the field nullable allows Claude to explicitly return null when the information doesn't exist in the source document. This prevents fabrication by giving the model a legitimate way to express absence. Option A adds metadata but doesn't prevent fabrication of the field itself. Option B makes it optional, which may cause it to be omitted without indication. Option D detects fabrication after the fact rather than preventing it.

Question 52:

A developer is building a CLI tool that calls Claude Code in a Docker container for CI/CD automated code reviews. They need consistent results regardless of the developer's local configuration. Which flag combination is most appropriate?

- A. -p with --output-format json for structured CI output
- B. -p with --bare and --output-format json for deterministic, configuration-free execution
- C. -p with --bare and ANTHROPIC_API_KEY to skip all local configuration discovery
- D. -p with --system-prompt to override any loaded CLAUDE.md files

Answer : C

Explanation:

--bare mode skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md, ensuring identical behavior on every machine. Combined with -p (non-interactive) and ANTHROPIC_API_KEY (skips OAuth/keychain), this provides fully deterministic execution. Option A doesn't prevent loading of project-specific configurations. Option B is close but missing the API key requirement. Option D only overrides one aspect of configuration.

Question 53:

A multi-agent system has a coordinator that routes customer requests to specialized subagents. When a customer asks about both billing AND shipping in the same message, the coordinator routes to the billing agent only. What system design change most effectively addresses this?

- A. Train the coordinator with few-shot examples of multi-concern messages and correct routing decisions
- B. Decompose multi-concern requests into separate items and route each to the appropriate specialist agent
- C. Implement keyword detection to identify all topics before routing
- D. Create a generalist agent that handles all types of requests without routing

Answer : B

Explanation:

The exam guide recommends decomposing multi-concern requests into distinct items that are investigated independently. The coordinator should identify all concerns, create separate task items, and route each to the appropriate specialist. Option A may improve but doesn't guarantee complete coverage. Option B is brittle and misses nuanced mentions. Option D removes the benefits of specialization.

Question 54:

A prompt engineer notices that Claude's responses for a code review task vary significantly in format — sometimes returning structured JSON, sometimes returning markdown lists, and sometimes returning prose paragraphs. They want consistent structured output for every review. Which approach provides the strongest format guarantee?

- A. Add explicit format instructions and examples to the system prompt
- B. Use few-shot examples showing the exact desired JSON format for code review findings
- C. Define a tool with a JSON schema for code review output and use `tool_choice` to force it
- D. Post-process Claude's output to convert any format into the desired structure

Answer : C

Explanation:

Using a tool with a JSON schema and forced `tool_choice` provides the strongest format guarantee because `tool_use` with schemas ensures schema-compliant output. This eliminates structural variation entirely. Option A improves consistency but doesn't guarantee it. Option B is the second-best approach. Option D fixes issues after they occur rather than preventing them.

Question 55:

An agent architect is configuring prompt caching for a long-running agent that makes repeated API calls. The system prompt and first few conversation turns are stable, but the most recent messages change frequently. Where should explicit cache breakpoints be placed for optimal savings?

- A. On the last stable content block before the frequently changing messages, with up to 4 explicit breakpoints on stable blocks
- B. On every message in the conversation to cache as much as possible
- C. Only on the system prompt since tool definitions shouldn't be cached
- D. On the most recent messages to cache the newest content

Answer : A

Explanation:

Explicit cache breakpoints should be placed on the last stable content blocks to maximize cache hits. Up to 4 explicit breakpoints can be placed per request. Content after the last breakpoint changes frequently and won't benefit from caching. Option A wastes cache writes on unstable content. Option C misses caching opportunities on stable conversation turns. Option D caches content most likely to change, wasting writes.

Question 56:

A team is building a document classification system. They define an enum for document categories: 'invoice', 'contract', 'memo', 'report'. During testing, 15% of documents don't fit any category and Claude forces them into the closest match, producing incorrect classifications. How should the schema be redesigned?

- A. Add more enum values covering all possible document types
- B. Remove the enum constraint and use a free-text string field for category
- C. Add an 'other' enum value paired with a detail string field for the specific category description
- D. Implement a confidence score that flags uncertain classifications for human review

Answer : C

Explanation:

The exam guide recommends the 'other' + detail string pattern for extensible categorization. This maintains the benefits of enum constraints for known categories while providing a legitimate escape valve for unknown types instead of forcing misclassification. Option A can't anticipate all types. Option B loses the structure benefits. Option D addresses symptoms but the model may still misclassify with high confidence.

Question 57:

A support agent finds two customer records matching the name provided: one with a New York address and one with a Chicago address. The customer hasn't provided additional identifying information. What is the correct agent behavior?

- A. Select the most recently active account as the likely match
- B. Process the request using both accounts and let the customer clarify later
- C. Request additional identifying information from the customer to disambiguate between the two records
- D. Use location-based heuristics from the customer's IP address to select the correct account

Answer : C

Explanation:

The exam guide states that when multiple customer matches exist, the agent should request additional identifiers rather than using heuristic selection. This prevents processing the wrong account with potential financial or privacy consequences. Option A uses a heuristic that could select the wrong account. Option B processes an incorrect account. Option D relies on unreliable data (VPNs, shared networks) for a consequential decision.

Question 58:

A developer configures hooks for a Claude Agent SDK application and wants to verify that any file modifications in the /production directory require explicit approval, while all other directories can be auto-approved. Which hook configuration achieves this?

- A. A PermissionRequest hook that returns 'deny' for all write operations system-wide
- B. A PostToolUse hook that reverts changes to /production after they are made
- C. A PreToolUse hook on Write and Edit tools with an 'if' filter matching /production paths, returning exit code 2 with a permissionDecision of 'ask'
- D. Adding a CLAUDE.md instruction that says 'always ask before modifying production files'

Answer : C

Explanation:

PreToolUse hooks with the 'if' field for argument filtering can match specific paths. For /production, the hook returns exit code 2 with permissionDecision 'ask' to require confirmation, while files outside /production pass through without the hook triggering. Option A blocks all writes everywhere. Option B catches changes after the fact. Option D relies on probabilistic prompt compliance for a safety-critical operation.

Question 59:

An architect is designing a system where Claude needs to process a 500-page regulatory document and answer questions about it. The document exceeds the practical attention span even within the context window. What is the recommended approach for handling this?

- A. Split the document into chunks and process each independently with separate API calls
- B. Summarize the document first, then answer questions from the summary
- C. Use embeddings to find relevant chunks and only include those in the prompt
- D. Place the full document at the beginning of the prompt and put questions at the end, as recommended for long context prompting

Answer : D

Explanation:

Claude's official best practices for long context prompting recommend placing longform data at the top of the prompt and queries at the end, which improves quality by approximately 30%. Claude's large context windows (1M tokens for 4.6 models) can handle 500-page documents. Option A loses cross-section context. Option C adds unnecessary complexity when the full document fits in context. Option D loses detail through summarization.

Question 60:

A team has a shared Claude Code project where they want to enforce that all test files follow specific mocking conventions, but these test files are spread across dozens of directories (components/__tests__/, api/__tests__/, lib/__tests__/, etc.). Using directory-level CLAUDE.md files would require dozens of duplicated files. What is the most efficient solution?

- A. Add all testing conventions to the root CLAUDE.md so they're always available
- B. Create a rule in .claude/rules/ with paths: ['**/__tests__/**', '**/*.test.*'] in the YAML frontmatter
- C. Create a single test-conventions skill in .claude/skills/ that developers invoke before writing tests
- D. Use a pre-commit hook to validate test files against conventions

Answer : B

Explanation:

.claude/rules/ files with YAML frontmatter glob patterns in the paths field (like **/__tests__/** and **/*.test.*) automatically load rules only when editing matching files. This works across all directories without duplication. Option A loads testing conventions even when writing application code. Option B requires manual invocation. Option D validates after the fact instead of guiding generation.

Question 61:

A developer wants to structure their MCP tool results to maximize downstream agent effectiveness. They are building a customer lookup tool that returns order history. Which response format best serves the agent?

- A. A formatted markdown table with all order details for human readability
- B. Raw database JSON with all fields including internal IDs, audit timestamps, and system metadata
- C. A summary paragraph describing the customer's order history in natural language
- D. Only high-signal information with semantic, stable identifiers — order status, amounts, dates, and customer-facing order IDs

Answer : D

Explanation:

The best practices for MCP tool design specify returning only high-signal information with semantic, stable identifiers. This gives the agent the information it needs without noise. Option A is optimized for

human display, not agent processing. Option B includes irrelevant internal data that wastes context. Option D loses structured data that the agent could reason about programmatically.

Question 62:

An architect is building a codebase exploration workflow in Claude Code. They need to understand the architecture of a large unfamiliar codebase before making changes. Which workflow sequence best preserves main conversation context?

- A. Use Explore subagent for verbose discovery, then switch to plan mode for architecture decisions, then direct execution for changes
- B. Use Grep extensively to search for patterns, then manually piece together the architecture
- C. Use Read on all key files, then ask Claude to summarize the architecture
- D. Use Bash to run code analysis tools and pipe output into the conversation

Answer : A

Explanation:

The recommended workflow is: Explore subagent for initial discovery (isolates verbose output, returns summaries), plan mode for architecture decisions (considers multiple approaches), then direct execution for implementation. This preserves main conversation context throughout. Option A fills the main context with raw file contents. Option B is manual and inefficient. Option D also bloats the main context.

Question 63:

When implementing server-side compaction, an architect wants to inject additional context (like updated tool results) after compaction occurs but before the conversation continues. Which configuration enables this?

- A. Configure 'pause_after_compaction: true' which returns stop_reason 'compaction', allowing context injection before continuing
- B. Set 'instructions' in the compaction config to include the additional context in the summary
- C. Use a PreCompact hook to inject context before the compaction occurs
- D. Append additional context as a system message after receiving the compaction block

Answer : A

Explanation:

The pause_after_compaction parameter returns stop_reason: 'compaction' after the summary is generated, giving the harness an opportunity to inject additional context (like fresh tool results or updated data) before continuing the conversation. Option A embeds content in the summary, but it's for

summarization instructions, not arbitrary context. Option C injects before compaction, so the content may be summarized away. Option D doesn't use the API's built-in mechanism.

Question 64:

A development team is using Claude for automated code review in CI but notices that the same issue is flagged repeatedly across multiple re-runs of the same PR, generating duplicate review comments. What is the recommended approach to prevent this?

- A. Use a unique session ID per PR and append all previous reviews to the context
- B. Include prior review findings in the context for subsequent review runs to avoid duplicate flagging
- C. Cache review results and skip files that haven't changed between runs
- D. Deduplicate review comments in post-processing before posting to the PR

Answer : B

Explanation:

The exam guide recommends including prior review findings in context for re-runs to avoid generating duplicate comments. This allows Claude to focus on new or unresolved issues. Option A may exceed context limits and doesn't specifically prevent duplication. Option C misses cases where the same issue exists in unchanged code. Option D treats symptoms rather than preventing the cause.

Question 65:

A prompt engineer needs Claude to always follow a specific sequence: analyze the input, identify issues, rank by severity, and then produce recommendations. They've tried instructions but Claude occasionally skips the ranking step. What technique most reliably enforces the full sequence?

- A. Add explicit step numbering in the prompt with 'you MUST complete all 4 steps'
- B. Use XML tags for each step with detailed instructions inside each tag
- C. Implement prompt chaining — separate API calls for each step, where each step's output feeds the next
- D. Add a self-check instruction: 'before responding, verify all 4 steps are completed'

Answer : C

Explanation:

Prompt chaining breaks a complex task into separate API calls for each step, with each step's output feeding the next. This provides deterministic enforcement of the sequence since each step must complete before the next begins. It also allows inspection of intermediate outputs. Options A, C, and D improve compliance but remain probabilistic — the model may still skip steps in a single response.

Question 66:

A customer support agent uses multiple MCP tools including `get_customer`, `lookup_order`, `check_inventory`, and `process_return`. The `get_customer` tool has a minimal description: 'Gets customer information.' The agent frequently calls `get_customer` when it should call `lookup_order` (which also has a minimal description: 'Looks up order information'). What is the root cause?

- A. The tools have overlapping functionality that should be consolidated into one tool
- B. The agent needs more training data showing correct tool selection patterns
- C. The tool descriptions are too minimal — they need detailed descriptions explaining inputs, outputs, edge cases, and when to use each
- D. The `tool_choice` parameter should be configured to force the correct tool based on the conversation state

Answer : C

Explanation:

The exam guide explicitly identifies minimal tool descriptions as the primary cause of tool misrouting. Tool descriptions are the primary mechanism LLMs use for tool selection. Without detailed descriptions explaining input formats, example queries, edge cases, and boundary explanations, the model cannot reliably distinguish between similar tools. Option A may be premature — the tools serve different purposes. Option C isn't applicable to API-based systems. Option D overrides the model's reasoning.

Question 67:

An architect is designing a JSON schema for extracting structured data from scientific research papers. Papers may contain multiple experiments, each with varying measurement types. Some papers have 2 experiments while others have 15. How should the schema handle this variability?

- A. Define a fixed array of 15 experiment slots with most defaulting to empty objects
- B. Use a flexible array with items defined by a reusable experiment sub-schema, with clear required and optional fields
- C. Store all experiments as a single concatenated text field for maximum flexibility
- D. Create separate schemas for papers with different experiment counts

Answer : B

Explanation:

A flexible array with a well-defined item schema handles variable-length data while maintaining structure. Required fields ensure minimum data quality for each experiment, while optional fields accommodate variation. Option A wastes schema space and may confuse the model. Option C loses all structure. Option D creates unmaintainable schema proliferation.

Question 68:

A team building a Claude-powered agent notices that after enabling tools, Claude's extended thinking with `tool_choice` set to 'any' returns an error. The same configuration works fine with `tool_choice` set to 'auto'. What restriction causes this error?

- A. Extended thinking doubles the token usage, exceeding the context window with 'any' `tool_choice`
- B. Extended thinking requires specific tools to be listed in `allowed_tools` before 'any' can be used
- C. The 'any' `tool_choice` is deprecated and should be replaced with 'auto' in all configurations
- D. With extended thinking enabled, only 'auto' and 'none' are supported for `tool_choice`; 'any' and forced tool selection return errors

Answer : D

Explanation:

When extended thinking is active, only `tool_choice` 'auto' and 'none' are supported. Using 'any' or forced tool selection (`{type: 'tool', name: '...'}`) with extended thinking returns an error. This is a documented API restriction. Option A is unrelated to `tool_choice` restrictions. Option C is incorrect — 'any' is still supported without thinking. Option D describes a non-existent configuration.

Question 69:

A team wants to maximize prompt cache hit rates for their agentic system. Their current setup changes `tool_choice` between 'auto' and forced tool selection across consecutive API calls, but keeps the system prompt and tool definitions stable. They observe low cache hit rates. What is causing the cache misses?

- A. Changes to `tool_choice` invalidate the messages-level cache, reducing overall hit rates
- B. The 5-minute cache TTL is expiring between consecutive API calls
- C. The system prompt is below the minimum cacheable length threshold
- D. Forced tool selection disables prompt caching entirely for that request

Answer : A

Explanation:

According to the cache hierarchy rules, `tool_choice` changes invalidate the messages layer of the cache. While tool definitions and system prompt remain cached, the message-level cache is invalidated when `tool_choice` alternates between values. This reduces overall cache hit rates. Option A doesn't explain why rates are low rather than zero. Option B is possible but the pattern correlates with `tool_choice` changes. Option D is incorrect — caching still works with forced selection.

Question 70:

An architect is implementing a handoff protocol for when a support agent escalates to a human representative. The current escalation passes only the customer's name and last message. What structured handoff information should be included?

- A. Customer details, root cause analysis, steps already attempted, and recommended next actions
- B. The complete conversation transcript so the human can read the full history
- C. A confidence score explaining why the agent couldn't resolve the issue
- D. A reference to the customer's previous support tickets for background context

Answer : A

Explanation:

The exam guide specifies that structured handoff protocols should include customer details, root cause analysis, and recommended actions. This gives the human representative everything needed to continue resolution without repeating diagnostic steps. Option A is too verbose for efficient handoff. Option C provides limited actionable information. Option D may be relevant but doesn't summarize the current interaction.

Question 71:

A developer is using Claude's context awareness feature (where the model tracks remaining token budget). They notice the model receives a `system_warning` about high token usage. What does this indicate and what model behavior should they expect?

- A. The API will automatically truncate the next response to conserve tokens
- B. The model will automatically switch to a lower-effort mode to reduce token consumption
- C. An error will occur on the next request due to exceeding the context window
- D. The model should begin proactively managing context by compacting or summarizing, as it's aware of remaining budget

Answer : D

Explanation:

Context awareness allows supported models (Sonnet 4.6, Sonnet 4.5, Haiku 4.5) to track remaining context window through system warnings. When usage is high, the model should proactively manage context — suggesting compaction, summarizing older content, or being more concise. This is a behavioral response, not an automatic API feature. Option A is incorrect — truncation isn't automatic. Option C is premature — the warning comes before the limit. Option D confuses context awareness with effort parameter.

Question 72:

A developer configures Claude Code for a new project using `/init` to generate the initial `CLAUDE.md`. After several sessions, they notice accumulated project-specific memories are affecting behavior in unexpected ways. Where are these automatically accumulated memories stored?

- A. In the root `CLAUDE.md` file, appended after the original content
- B. In `~/.claude/projects/<project>/memory/MEMORY.md`, with the first 200 lines loaded automatically

- C. In `.claude/memory/` directory within the project, shared with all team members
- D. In the session history, persisted between restarts but not in any specific file

Answer : B

Explanation:

Auto-accumulated memories are stored in `~/.claude/projects/<project>/memory/MEMORY.md`. The first 200 lines (or 25KB) are loaded automatically into each session. This is user-level storage, not shared with teammates. Option A confuses auto memory with the project `CLAUDE.md`. Option C describes a non-existent shared directory. Option D doesn't describe a persistent file location.

Question 73:

A team is configuring the MCP Connector at the API level to connect to a custom tool server. They want to restrict which tools the agent can access from the server to only 'search' and 'retrieve' tools, blocking all others. How should they configure this?

- A. List only the allowed tools in the `mcp_servers` configuration
- B. Add tool filtering logic in the MCP server itself to only expose search and retrieve
- C. Configure `tool_choice` to only allow the search and retrieve tools
- D. Use `mcp_toolset` in the tools array with an allowlist pattern matching 'search' and 'retrieve'

Answer : D

Explanation:

The MCP Connector's `mcp_toolset` supports allowlist and denylist patterns for filtering which tools are available from a connected MCP server. Using an allowlist restricts access to only the specified pattern-matching tools. Option A doesn't support per-tool filtering at the server config level. Option C controls which tool Claude calls, not which tools are available. Option D requires server-side changes which may not be possible.

Question 74:

A developer implements adaptive thinking for their Claude 4.6 Opus agent. They want to ensure the agent thinks deeply on complex coding problems but skips extended thinking on simple file reads and straightforward questions. How should they configure this?

- A. Set manual `budget_tokens` for each request type based on estimated complexity
- B. Set `thinking: {type: 'enabled'}` with a very large `budget_tokens` and let Claude self-limit
- C. Implement client-side complexity detection that toggles thinking on/off per request

D. Use thinking: {type: 'adaptive'} with effort='high' — Claude dynamically decides when and how much to think

Answer : D

Explanation:

Adaptive thinking (type: 'adaptive') combined with an appropriate effort level lets Claude dynamically decide when and how deeply to think. At high effort, Claude almost always thinks deeply on complex problems but may skip thinking on simple ones. This is the recommended approach for Opus 4.6, replacing the deprecated budget_tokens. Option A uses the deprecated approach. Option C adds unnecessary client-side complexity. Option D doesn't provide the adaptive behavior.

Question 75:

An agentic system encounters a scenario where the Edit tool fails because the text pattern matches multiple locations in the file. What is the recommended recovery approach using built-in tools?

- A. Use Read to load the full file, modify the content programmatically, then Write the complete file back
- B. Retry Edit with a more specific text pattern that uniquely identifies the target location
- C. Use Bash to run a sed command that replaces only the first occurrence
- D. Split the file into sections and Edit each section separately

Answer : A

Explanation:

The exam guide states that when Edit fails due to non-unique text matches, the fallback is to use Read + Write — read the complete file, make the modification, and write it back. This provides full control over the edit. Option A may work but isn't the documented fallback. Option C introduces shell complexity and may have escaping issues. Option D is unnecessarily complex.

Question 76:

A team is deploying an agent that processes customer orders. The agent must always verify inventory before confirming an order, but occasionally skips this step when the customer seems confident about product availability. Adding emphasis to the system prompt reduced but didn't eliminate the issue. What is the most reliable solution?

- A. Implement a PreToolUse hook that blocks confirm_order until check_inventory has returned available status in the current session
- B. Add even more emphatic system prompt language with all-caps warnings about the verification requirement
- C. Create a dedicated verification subagent that runs before the main order agent
- D. Log all orders that skip inventory checks and review them manually

Answer : A

Explanation:

Programmatic enforcement via hooks provides deterministic guarantees. A PreToolUse hook on `confirm_order` that verifies `check_inventory` has been called prevents the step from being skipped, regardless of the conversation flow. Option B is more of the same probabilistic approach that already failed. Option C adds complexity without guaranteeing ordering. Option D detects issues after they cause problems.

Question 77:

A developer is running a long agentic task in Claude Code and needs to manage token costs. The session is consuming excessive tokens because Claude generates detailed thinking traces and verbose tool call plans even for simple operations. Which approach most effectively reduces token consumption while maintaining quality on complex operations?

- A. Switch from Opus 4.6 to Haiku 4.5 for all operations to reduce per-token costs
- B. Disable extended thinking entirely to eliminate thinking token overhead
- C. Use `effort='medium'` as a baseline and increase to 'high' only for complex operations, reducing thinking and verbosity on simple tasks
- D. Set a strict `max_tokens` limit of 1000 to force concise responses

Answer : C

Explanation:

The `effort` parameter is a behavioral signal affecting all tokens — text responses, tool calls, and extended thinking. Using 'medium' as default significantly reduces unnecessary verbosity on simple tasks while maintaining quality. It can be elevated to 'high' for genuinely complex operations. Option A may reduce quality significantly. Option B sacrifices reasoning quality. Option D truncates output rather than reducing unnecessary verbosity.

Question 78:

An architect is designing an agent that manages support tickets across multiple systems. The agent needs to: (1) verify customer identity, (2) look up the relevant ticket, (3) check account status, and (4) process the resolution. Steps 1-3 can happen in any order but all must complete before step 4. What orchestration pattern best fits this requirement?

- A. Sequential pipeline: execute steps 1, 2, and 3 in fixed order, then 4
- B. Prompt chaining: separate API calls for each step with programmatic transitions
- C. Parallel execution: run all 4 steps simultaneously and reconcile results
- D. Model-driven: let Claude decide the order of steps 1-3 based on available information, with a programmatic gate before step 4

Answer : D

Explanation:

Model-driven orchestration lets Claude reason about the most efficient ordering of steps 1-3 based on what information is available, while a programmatic gate (hook) ensures step 4 only executes after all prerequisites complete. Option A imposes unnecessary ordering constraints. Option C would execute step 4 without waiting for prerequisites. Option D adds unnecessary API call overhead and removes Claude's ability to adaptively order steps.

Question 79:

A team configures compaction for their agent harness. After compaction occurs, they notice that specific numerical values from early in the conversation (account balances, dates, transaction IDs) are lost in the summary. What design pattern prevents this information loss?

- A. Increase the compaction trigger threshold to delay summarization as long as possible
- B. Extract critical transactional facts into a persistent structured block that survives compaction
- C. Use custom compaction instructions that specifically request preservation of numerical values
- D. Disable compaction and rely on larger context windows instead

Answer : B

Explanation:

The exam guide warns about progressive summarization risks where numerical values, dates, and specific identifiers are condensed into vague summaries. Extracting transactional facts into a persistent 'case facts' block that is placed after the compaction summary ensures they survive. Option A delays but doesn't prevent the problem. Option B may help but isn't as reliable as a separate persistent block. Option D avoids the issue but limits conversation length.

Question 80:

A Claude Code project has both team-shared and personal MCP server configurations. A new developer clones the repo and finds that shared team tools work but personal experimental tools from a previous project are not available. Where should the developer look to configure personal MCP servers?

- A. Add personal servers to the project's .mcp.json file in a 'personal' section
- B. Configure personal servers in the CLAUDE.md file under a tools section
- C. Create a .mcp-personal.json file alongside the project's .mcp.json
- D. Add personal servers to ~/.claude.json for user-level configuration

Answer : D

Explanation:

User-level configuration, including personal MCP servers, goes in ~/.claude.json. This file is not version controlled and exists only on the developer's machine. Project-level config (.mcp.json) is for team-shared servers. Option A would share personal configs with everyone. Option B is for instructions, not MCP configuration. Option D uses a non-standard configuration file.

Question 81:

An architect notices that their Claude Code agent frequently over-engineers solutions — creating unnecessary helper functions, adding unused parameters, and building abstractions for one-time operations. The team is using Claude 4.6 Opus. What is the most targeted fix?

- A. Add specific guidance in CLAUDE.md about minimal implementations, as Claude 4.6 tends to overengineer
- B. Switch to a less capable model that generates simpler code
- C. Reduce the effort parameter to 'low' to prevent over-elaborate responses
- D. Use plan mode exclusively to review proposed changes before implementation

Answer : A

Explanation:

The official prompting guide notes that Claude 4.6 specifically tends to overengineer. Adding explicit guidance in CLAUDE.md about keeping solutions minimal and avoiding unnecessary abstractions directly addresses this known tendency. Option A sacrifices quality for simplicity. Option C may reduce quality on complex tasks that need detailed responses. Option D adds overhead to all tasks, not just the problematic ones.

Question 82:

A multi-agent system's coordinator allows subagents to report findings. However, the synthesis subagent occasionally includes lengthy reasoning chains alongside its actual findings, consuming excessive coordinator context. What architectural change best addresses this?

- A. Set a max_tokens limit on subagent responses to force brevity
- B. Add instructions to the synthesis subagent prompting it to be concise
- C. Use the Explore subagent pattern for all research tasks to isolate verbose output
- D. Configure subagents to return structured data separated from reasoning, with reasoning excluded from the coordinator's context

Answer : D

Explanation:

The exam guide recommends having subagents return structured data instead of verbose content with reasoning chains. Separating content from reasoning allows the coordinator to consume only the relevant findings. Option A may cut off important findings. Option B improves but doesn't guarantee structure. Option D replaces specialized subagents with a generic agent, losing specialization benefits.

Question 83:

A developer is implementing an agentic loop and encounters `stop_reason: 'max_tokens'` after requesting a complex analysis. What does this indicate and what should the developer do?

- A. Claude has finished its analysis — treat this the same as `'end_turn'`
- B. The context window is full — start a new conversation with a summary of previous interactions
- C. Claude wants to call a tool but the response was cut off — resume the tool call
- D. Claude's response was truncated due to the token limit — the developer should increase `max_tokens` or handle the partial response

Answer : D

Explanation:

`stop_reason: 'max_tokens'` means the output was truncated because it reached the configured `max_tokens` limit. The response is incomplete. The developer should increase `max_tokens` for subsequent requests or handle the partial response appropriately. This is distinct from `'end_turn'` (Claude is genuinely done) and `'tool_use'` (Claude wants to call a tool). Option A treats incomplete output as complete. Option C assumes a tool call without evidence. Option D confuses output limits with input limits.

Question 84:

A team using Claude for data extraction needs to categorize extracted entities using a predefined enum. However, they discover that 8% of entities don't match any predefined category, and Claude forces them into the nearest match. After adding an `'other'` category, Claude overuses it (25% of entities). What additional schema element is needed?

- A. A detail string field paired with `'other'` that captures the specific description, enabling post-hoc analysis of the actual categories
- B. More detailed descriptions for each existing enum value to improve matching accuracy
- C. A `confidence_score` field that filters out low-confidence `'other'` classifications
- D. A validation step that reclassifies `'other'` entries using a second API call

Answer : A

Explanation:

The exam guide recommends the 'other' + detail string pattern. The detail field captures specific descriptions for entities classified as 'other', enabling systematic analysis of what categories are actually needed. This reduces 'other' overuse because the model must justify the classification. Option A doesn't improve classification. Option B may help but doesn't address the need for a capture mechanism. Option D adds cost without improving the initial classification.

Question 85:

A developer is building an agent that uses both the Anthropic API directly and Claude Code's built-in tools. They want to explore a large repository's architecture using Claude Code tools. Which tool should be used first to find entry points in the codebase?

- A. Read on common entry point files like main.ts, index.ts, or app.ts
- B. Bash to run 'find . -name "*.ts" | head -50' for a file listing
- C. Glob to find files matching common entry point patterns, then Grep to search for specific exports and imports
- D. Write to create an exploration script that maps the codebase structure

Answer : C

Explanation:

The recommended approach is to build understanding incrementally: use Grep to find entry points (function names, exports, imports) and Glob to find files by pattern. The distinction matters: Grep searches content while Glob matches file paths. Together they provide efficient codebase navigation. Option A guesses at specific files. Option B uses a less integrated approach. Option D creates unnecessary artifacts.

Question 86:

An architect is evaluating whether to use Claude's structured outputs (output_config.format) or tool_use with JSON schemas for a data extraction pipeline. The pipeline processes diverse document types and must always return exactly one extraction result per document. Which approach is more appropriate?

- A. tool_use with tool_choice 'any' — guarantees a tool call (structured output) even when the document type is unknown
- B. Neither — use prompt instructions to request JSON output and parse the response
- C. Both approaches concatenated — tool_use for extraction, then output_config for final formatting
- D. output_config.format with json_schema — response-level structured output regardless of content

Answer : B

Explanation:

Both `tool_use` with 'any' and `output_config.format` with `json_schema` can guarantee structured output. However, `output_config` provides response-level guarantees and is the more direct approach for ensuring every response conforms to a schema. It's the recommended choice when you always want structured output. Option A works but is designed for tool-call scenarios. Option C is unnecessarily complex. Tool_choice any with unknown document types is what the exam guide recommends (Option A is also a valid approach), but `output_config.format` is the more appropriate feature for response-level structured output.

Question 87:

A team discovers that their agent system's overall accuracy across document types is 96%, but when they drill down, contracts have 99% accuracy while invoices have only 78% accuracy. Their current evaluation only reports aggregate metrics. What evaluation methodology change is needed?

- A. Increase the total sample size for evaluation to improve statistical significance
- B. Add automated regression tests that compare each new extraction against a golden dataset
- C. Implement stratified sampling that measures accuracy separately by document type AND by individual field
- D. Focus evaluation resources on invoices since they have the lowest accuracy

Answer : C

Explanation:

The exam guide emphasizes that aggregate accuracy masks specific performance issues. Stratified sampling by document type AND field reveals hidden problems — the 78% invoice accuracy might concentrate in specific fields like `tax_amount` or `line_items`. Option A improves statistical power but doesn't reveal type-specific issues. Option B tests against fixed data without revealing systematic patterns. Option C addresses one type but may miss issues in others.

Question 88:

A team building a Claude Code workflow realizes they need to maintain a set of conventions that apply only when editing Python files, regardless of which directory those files are in. They currently use a monolithic root `CLAUDE.md` which loads all conventions for all file types. What is the most targeted approach?

- A. Create a subdirectory `CLAUDE.md` in each directory that contains Python files
- B. Add a Python-specific section to the root `CLAUDE.md` with clear headers
- C. Create a skill in `.claude/skills/` for Python development
- D. Add a rule in `.claude/rules/python.md` with paths: `['**/*.py']` in the YAML frontmatter

Answer : D

Explanation:

.claude/rules/ files with glob patterns in the paths frontmatter load only when editing matching files. A rule with paths: ['**/*.py'] will load Python-specific conventions whenever a Python file is being edited, regardless of directory, without polluting context when editing non-Python files. Option A requires duplication across directories. Option B loads conventions for all file types. Option C requires manual invocation.

Question 89:

When designing an MCP tool that returns customer data, a developer includes internal database IDs, audit trail timestamps, query execution metrics, and the actual customer information in every response. The agent's subsequent reasoning occasionally references internal IDs causing confusion in customer-facing responses. What should the developer change?

- A. Add post-processing to strip internal IDs from Claude's responses before sending to customers
- B. Redesign the tool to return only high-signal information with semantic, stable identifiers relevant to the agent's task
- C. Instruct Claude in the system prompt to never reference internal IDs in customer-facing responses
- D. Create separate tools: one that returns internal data for logging and one that returns only customer-facing data

Answer : B

Explanation:

MCP tool design best practices specify returning only high-signal information with semantic, stable identifiers. Including internal database IDs and query metrics wastes context and introduces irrelevant data that Claude may inadvertently use. Option A treats symptoms. Option B relies on probabilistic compliance. Option D over-complicates the tool distribution.

Question 90:

A developer is using Claude Code and wants to generate a CLAUDE.md file for a new project. They want to capture the project's build commands, testing framework, code style, and directory structure. Which command generates this initial configuration?

- A. /config init to create a new project configuration
- B. /setup to run the project initialization wizard
- C. /memory create to establish the project memory file
- D. /init to generate the initial CLAUDE.md based on project analysis

Answer : D

Explanation:

The `/init` command analyzes the project and generates an initial `CLAUDE.md` with project-specific context including build commands, testing frameworks, code patterns, and directory structure. Options A, B, and D reference non-existent commands.

Question 91:

An architect configures server-side compaction with custom instructions. After compaction, they notice critical technical decisions from early in the conversation are summarized too tersely, losing important rationale. The default summarization prompt isn't preserving architectural decision context. What configuration change addresses this?

- A. Increase the trigger threshold to delay compaction and preserve more of the original conversation
- B. Provide custom compaction instructions that explicitly request preservation of technical decisions, rationale, and architectural choices
- C. Disable compaction and manually manage context by removing non-essential messages
- D. Use `pause_after_compaction` to review and supplement the summary after each compaction

Answer : B

Explanation:

The compaction API supports custom 'instructions' that completely replace the default summarization prompt. By providing instructions that explicitly prioritize preserving technical decisions and their rationale, the resulting summary will maintain this critical context. Option A delays compaction but doesn't fix the summarization quality. Option C removes automatic management. Option D allows supplementation but doesn't fix the base summary quality.

Question 92:

A Claude-powered customer support system encounters a scenario where the customer's reported issue could be caused by three different root causes. The agent currently investigates one cause at a time, often spending all context on the first hypothesis which turns out to be wrong. What orchestration improvement would help?

- A. Spawn parallel subagents to investigate each hypothesis simultaneously, with the coordinator evaluating all results
- B. Create a diagnostic decision tree that the agent follows step by step
- C. Add instructions to check the most common cause first to improve success rate
- D. Increase the context window to allow sequential investigation of all three causes

Answer : A

Explanation:

Spawning parallel subagents for competing hypotheses is a recommended pattern from the agent teams documentation. Each subagent investigates one hypothesis independently, and the coordinator evaluates

all findings to determine the root cause. This is faster and avoids the context exhaustion problem.

Option A is a statistical optimization, not an architectural solution. Option B may be too rigid. Option D addresses a symptom.

Question 93:

A developer is creating a custom subagent definition for Claude Code that should specifically handle SQL-related tasks. The subagent should only have access to Read, Bash, and Grep tools, and should use a plan permission mode. Where is this definition configured?

- A. In the CLAUDE.md file under a ## Subagents section
- B. In .claude/agents/ with frontmatter fields for tools, disallowedTools, and permissionMode
- C. Both B and C are valid approaches for defining custom subagents
- D. In a JSON configuration file passed via --agents CLI flag

Answer : C

Explanation:

Custom subagent definitions can be configured either in .claude/agents/ directory files (with YAML frontmatter defining allowed tools, permission mode, etc.) or passed via the --agents CLI flag as JSON configuration. Both approaches support specifying tools, disallowedTools, permissionMode, and other subagent properties. Option A uses CLAUDE.md which is for instructions, not agent definitions.

Question 94:

An architect is building a system that combines prompt caching with compaction. After compaction creates a summary block, subsequent requests show no cache hits even though the system prompt hasn't changed. What explains this?

- A. Compaction is incompatible with prompt caching
- B. The compaction block itself needs a cache_control breakpoint to establish a new cache entry that subsequent requests can match against
- C. The compaction block is too large for the minimum cacheable threshold
- D. Cache TTL expired during the compaction processing time

Answer : B

Explanation:

After compaction, all content before the compaction block is dropped. To achieve cache hits on subsequent requests, you need to add a cache_control breakpoint on the compaction block itself, establishing it as the new cache base. Without this, each post-compaction request has a different prefix. Option A is incorrect — they work together when properly configured. Option C is unlikely. Option D is possible but not the structural cause.

Question 95:

A team is using the Claude Agent SDK to build an agent that needs to authenticate against a private Bedrock deployment instead of the default Anthropic API. How should they configure the SDK?

- A. Set `CLAUDE_CODE_USE_BEDROCK=1` environment variable and configure AWS credentials
- B. Set `ANTHROPIC_API_KEY` to the Bedrock access key and the SDK will auto-detect the provider
- C. Pass the Bedrock endpoint URL directly to the `query()` function
- D. Use a custom HTTP client that proxies requests to Bedrock

Answer : A

Explanation:

The Agent SDK supports third-party providers through environment variables:

`CLAUDE_CODE_USE_BEDROCK=1` for AWS Bedrock, `CLAUDE_CODE_USE_VERTEX=1` for Google Vertex, and `CLAUDE_CODE_USE_FOUNDRY=1` for Azure. With `CLAUDE_CODE_USE_BEDROCK=1`, the SDK uses standard AWS credentials for authentication. Option A doesn't follow the documented approach. Option B conflates API key types. Option D bypasses the SDK's built-in provider support.

Question 96:

An architect needs a Claude agent to generate a detailed analysis report that must have a specific structure: executive summary, methodology, findings (as an array), and recommendations. The agent should be guaranteed to produce this exact structure. What is the most robust approach for Claude 4.6 models?

- A. Detailed system prompt instructions describing the required report structure
- B. Few-shot examples showing correctly structured reports
- C. A tool definition with the report structure as the JSON schema, using forced `tool_choice`
- D. `output_config` with `json_schema` type and `strict: true` on the corresponding tool

Answer : D

Explanation:

`strict: true` on tool definitions guarantees schema conformance through grammar-based compilation. This eliminates structural errors entirely for 4.6 models. Combined with `output_config` using `json_schema`, this provides the strongest format guarantee. Option A and B improve compliance but don't guarantee it. Option C provides schema compliance at the tool level but strict mode adds grammar-level enforcement.

Question 97:

A team notices their agent sometimes overwrites files without checking existing content, leading to lost work. They want Claude Code to always read a file before modifying it, but only when the file already exists. What is the most appropriate solution?

- A. Use plan mode for all file operations to show proposed changes before execution
- B. Create a PreToolUse hook on Write and Edit that checks if the file exists and blocks without a prior Read
- C. Add 'Always read files before editing them' to the root CLAUDE.md project instructions
- D. Set --permission-mode acceptEdits to require manual approval for every file change

Answer : C

Explanation:

Adding instructions to CLAUDE.md is the appropriate approach for behavioral guidance that doesn't have safety-critical consequences (unlike financial operations that need hooks). Reading before editing is a best practice, not a strict compliance requirement, and CLAUDE.md instructions are reliably followed for this type of convention. Option B over-engineers a non-critical convention. Option C adds overhead to all operations. Option D requires constant manual intervention.

Question 98:

A developer is processing a batch of 5,000 documents through the Message Batches API. Each document requires extraction with a specific JSON schema and then a follow-up validation call that references the extraction results. Can this two-step workflow be achieved in a single batch request per document?

- A. Yes — use multi-turn messages within each batch request to chain extraction and validation
- B. Yes — include both the extraction and validation tool definitions and Claude will call them sequentially
- C. No — the Message Batches API does not support multi-turn tool calling; submit extraction as one batch, then validation as a second batch
- D. No — each batch request is limited to 1,000 tokens, which is insufficient for two-step workflows

Answer : C

Explanation:

The Message Batches API explicitly does not support multi-turn tool calling within a single request. For a two-step workflow, you need two separate batches: first for extraction, then a second batch for validation using the extraction results. Option A and B incorrectly assume multi-turn support. Option D states an incorrect limitation.

Question 99:

A support agent encounters a customer who repeatedly expresses frustration about a product issue. The agent can resolve the technical problem but the customer's tone is increasingly negative. The current system is configured to escalate based on negative sentiment. What is the correct approach per the exam guide's recommendations?

- A. Escalate to a human immediately based on the negative sentiment detection
- B. Acknowledge the frustration while offering resolution when within the agent's capability
- C. Ignore the sentiment, resolve the underlying issue, and confirm resolution with the customer
- D. Ask the customer if they'd like to speak with a manager before attempting a technical resolution

Answer : B

Explanation:

The exam guide states that escalation should NOT be triggered by sentiment alone. Appropriate escalation triggers are explicit customer requests for a human, policy exceptions/gaps, or inability to make progress. The agent should acknowledge frustration (emotional validation) while continuing to work toward resolution within its capability. Option A triggers escalation on sentiment alone. Option B delays resolution for an unnecessary offer. Option C ignores the emotional component entirely.

Question 100:

An architect is building a system that needs to track token usage accurately when using server-side compaction. They notice their billing calculations are lower than actual usage. Where is the complete token usage tracked?

- A. In the usage.iterations array which separates compaction iteration tokens from message iteration tokens
- B. In a separate /v1/usage endpoint that aggregates all token consumption
- C. In the top-level usage.input_tokens and usage.output_tokens fields of the API response
- D. In the X-Token-Usage response header that includes all token types

Answer : A

Explanation:

The top-level input_tokens and output_tokens do NOT include compaction iteration tokens. The complete usage must be calculated by summing across the usage.iterations array, which separately tracks tokens for compaction iterations and message iterations. This is why the architect's billing calculations were too low. Option A misses compaction tokens. Options B and D reference non-existent features.

Question 101:

A team is designing a multi-agent research system where the coordinator spawns three subagents in parallel. Two subagents complete successfully, but the third returns a timeout error while partially

completing its task. The coordinator currently discards the failed subagent's entire output. What is the improved error handling approach?

- A. Proceed with only the two successful results and note the gap in the final report
- B. Retry the failed subagent with the same parameters until it succeeds or exceeds a retry limit
- C. Accept the partial results from the timed-out subagent alongside the structured error context, and evaluate whether re-delegation is needed based on what was and wasn't completed
- D. Terminate the entire workflow since incomplete results may lead to inaccurate synthesis

Answer : C

Explanation:

The exam guide recommends returning structured error context including partial results and alternatives. The coordinator should evaluate what was completed, what's missing, and decide whether to re-delegate the remaining scope. Option B may waste resources on permanently failing operations. Option C loses potentially useful partial results. Option D is the anti-pattern of terminating an entire workflow on a single failure.

Question 102:

A developer wants to use a Claude Code skill that accepts arguments. They invoke /migrate with 'users_table' as an argument. Inside the SKILL.md, which substitution variables can access this argument?

- A. \$PARAMS['migration_name'] contains 'users_table' based on the argument-hint definition
- B. \${input} contains 'users_table' and arguments must be parsed manually
- C. \$ARGUMENTS contains 'users_table' and \$1 contains 'users_table' as the first positional argument
- D. process.argv[2] contains 'users_table' as it's passed through to the shell

Answer : C

Explanation:

SKILL.md supports \$ARGUMENTS (the full argument string) and positional variables like \$1, \$2, etc. So \$ARGUMENTS would contain 'users_table' and \$1 would also contain 'users_table' as the first positional argument. Options B, C, and D reference non-existent substitution mechanisms.

Question 103:

An architect implementing prompt chaining for a complex analysis pipeline wants to inspect intermediate outputs between each step. The pipeline has 4 steps: data extraction, normalization, analysis, and report generation. What is the key advantage of prompt chaining over a single-prompt approach for this pipeline?

- A. Prompt chaining allows each step to use a different model optimized for that task type

- B. Prompt chaining enables parallel execution of all four steps simultaneously
- C. Prompt chaining reduces total token consumption compared to a single long prompt
- D. Prompt chaining provides deterministic step ordering with inspectable intermediate outputs at each stage

Answer : D

Explanation:

The key advantage of prompt chaining is deterministic step ordering with the ability to inspect intermediate outputs at each stage. This enables debugging, quality checks between steps, and guarantees that all steps execute in order. Option A is possible but not the primary benefit. Option C may actually increase total tokens due to repeated context. Option D is incorrect — the steps depend on each other sequentially.

Question 104:

A support agent is configured with `get_customer`, `lookup_order`, `process_refund`, and `escalate_to_human` tools. The system prompt says 'always verify customer identity before processing any financial transaction.' During testing, the agent processes a refund without calling `get_customer` in 5% of cases. The team adds 'THIS IS CRITICAL — NEVER skip customer verification' to the prompt, reducing failures to 2%. What does this pattern indicate about the overall design?

- A. Prompt-based enforcement has a non-zero failure rate for compliance requirements with financial consequences, and programmatic enforcement (hooks) should replace it
- B. The prompt needs even stronger language to achieve 0% failure rate
- C. The 2% failure rate is acceptable for most production systems
- D. The tool execution order should be hardcoded in the system rather than left to the model

Answer : A

Explanation:

This directly illustrates the exam guide's key principle: prompt-based guidance has a non-zero failure rate that cannot be eliminated entirely, even with emphatic language. For operations with financial consequences, programmatic enforcement via hooks is required. The pattern of adding emphasis and reducing but not eliminating failures proves the limitation. Option A will approach but never reach 0%. Option C ignores financial risk. Option D removes model flexibility entirely.

Question 105:

A developer is building an extraction system that needs to handle variable document formats. Some documents have well-structured data in tables while others have unstructured narrative text. The JSON schema includes both structured fields (amounts, dates) and free-text fields (descriptions, notes). What validation approach is most effective?

- A. Schema validation alone is sufficient since tool_use with JSON schemas eliminates all extraction errors
- B. Use strict mode to handle both structural and semantic validation
- C. Skip schema validation and rely entirely on post-processing to fix extraction errors
- D. Schema validation handles structural errors; semantic validation (e.g., values summing correctly, dates in valid ranges) requires separate programmatic checks

Answer : D

Explanation:

The exam guide distinguishes between structural/schema errors (which tool_use eliminates) and semantic errors (which require separate validation). Schema validation ensures correct types and required fields, but semantic validation — like verifying amounts sum correctly or dates fall in valid ranges — requires programmatic checks. Option A ignores semantic errors. Option C discards the benefits of schema validation. Option D overstates strict mode's capabilities.

Question 106:

An architect wants to build a codebase exploration workflow. They need to trace how a specific function is used across multiple files, following through wrappers and re-exports. Which built-in tool sequence is most effective?

- A. Grep to find the function definition → Read to understand its signature → Grep again to find all imports and usages across the codebase
- B. Glob to find all TypeScript files → Read each file to scan for the function name
- C. Bash to run 'grep -r functionName .' across the entire codebase
- D. Read the entry point file → manually trace through each import chain

Answer : A

Explanation:

The exam guide recommends building understanding incrementally: use Grep to find entry points, Read to follow imports and understand signatures, then Grep again to trace usage across the codebase. This is more targeted than scanning all files. Option B reads files unnecessarily. Option C is less integrated with the agent's reasoning. Option D is manual and may miss indirect references.

Question 107:

A team has an MCP server that exposes 12 tools for customer support. Six of these tools are for management analytics (not relevant during customer interactions). Agents occasionally call analytics tools during customer sessions, providing irrelevant information. What is the best solution?

- A. Add instructions to the system prompt explaining which tools to use during customer interactions
- B. Rename the analytics tools with a 'mgmt_' prefix so the agent can differentiate them

- C. Use `tool_choice` to restrict available tools during customer sessions
- D. Create separate MCP server configurations — one for customer support agents and one for analytics agents, each with scoped tools

Answer : D

Explanation:

The exam guide recommends restricting each agent's tool set to role-relevant tools. Agents with tools outside their specialization misuse them. Separate MCP configurations ensure customer support agents only see support tools and analytics agents only see analytics tools. Option A relies on probabilistic compliance. Option C restricts to a single tool, not a subset. Option D is a partial fix that doesn't enforce boundaries.

Question 108:

A developer is creating Claude Code commands for their team. They need a `/deploy` command that's available to all developers via version control and a `/personal-test` command that's only for their personal use. Where should each command be placed?

- A. Both in `.claude/commands/` — personal commands are marked with a `'personal: true'` frontmatter field
- B. `/deploy` in `.claude/commands/` (project-scoped) and `/personal-test` in `~/.claude/commands/` (user-scoped)
- C. `/deploy` in `.claude/skills/` and `/personal-test` in `.claude/commands/`
- D. Both in `~/.claude/commands/` — project commands are deployed via a setup script

Answer : B

Explanation:

Project-scoped commands in `.claude/commands/` are shared via version control and available to all team members. User-scoped commands in `~/.claude/commands/` are personal and only available to the individual developer. Option A doesn't support a personal frontmatter field. Option C puts `deploy` in `skills` which is a different feature. Option D doesn't share commands via version control.

Question 109:

A multi-agent financial compliance system must ensure that all customer interactions follow: (1) identity verification, (2) KYC check, and (3) risk assessment before any transaction. The team initially used prompt instructions but experienced a 3% compliance failure rate. They're now considering hooks but unsure which combination to use. Which hook strategy provides comprehensive enforcement?

- A. `PostToolUse` hooks on all three prerequisite tools that set flags, combined with a `PreToolUse` hook on `process_transaction` that checks all flags

- B. Three separate PreToolUse hooks, one for each prerequisite, each blocking process_transaction independently
- C. A single PreToolUse hook on process_transaction that checks if all three prerequisites have been completed
- D. A SessionStart hook that sets the compliance workflow as mandatory for the entire session

Answer : C

Explanation:

A single PreToolUse hook on process_transaction that verifies all three prerequisites (identity verification, KYC, risk assessment) have been completed in the current session is the cleanest and most maintainable approach. It blocks the transaction tool until all prerequisites are satisfied. Option B creates unnecessary hook proliferation. Option C is more complex without additional benefit. Option D is too broad and doesn't enforce ordering.

Question 110:

An error-prone extraction system processes scientific papers. The team implements retry-with-error-feedback: when validation fails, the specific error is appended to the prompt and Claude retries. This works well for formatting errors but has no effect on a recurring issue where Claude extracts methodology descriptions from the introduction instead of the methods section. Why aren't retries effective for this case?

- A. Retries are limited to 3 attempts by default, which isn't enough for complex extraction tasks
- B. Retries with error feedback are effective for structural/format errors but not when the issue is the model's interpretation of document structure, which requires better prompting
- C. The retry approach is effective but the model needs higher effort to analyze the paper more carefully
- D. The error feedback format doesn't provide enough detail about the methodology location

Answer : B

Explanation:

The exam guide distinguishes between structural/format errors (which retries fix well) and semantic/interpretation errors (which require different approaches). The model consistently misidentifying the methodology section is an interpretation issue that won't be fixed by simply retrying — it needs better prompting or explicit section markers. Option A misidentifies the issue. Option B may help marginally. Option C doesn't address the interpretation root cause.

Question 111:

An architect deploys a Claude-powered agent that handles both English and Spanish customer interactions. The agent's prompt caching is configured with a cache breakpoint on the system prompt. When the system switches between English and Spanish conversations, cache hit rates drop significantly. What is the most likely cause?

- A. Prompt caching doesn't support multilingual content
- B. Spanish tokens are larger than English tokens, exceeding the cache size limit
- C. The system prompt changes between English and Spanish conversations, invalidating the cache at that level and all subsequent levels
- D. The 5-minute cache TTL expires between language switches

Answer : C

Explanation:

If the system prompt includes language-specific instructions that change between English and Spanish conversations, this invalidates the system-level cache and all subsequent levels (messages). Changes at any level in the cache hierarchy invalidate that level and everything below it. Option A is incorrect. Option C misunderstands tokenization. Option D is possible but the pattern correlating with language switches points to content changes.

Question 112:

A developer is building a tool that must always output a specific JSON structure: {"status": "success"|"error", "data": object, "metadata": object}. They want to guarantee this structure on every response with no exceptions. Which approach provides the absolute strongest guarantee on Claude 4.6?

- A. output_config with json_schema type, strict: true, and the specified schema
- B. Tool definition with the schema and tool_choice forced to that tool
- C. System prompt with explicit JSON template and instructions to follow it exactly
- D. Few-shot examples showing 5 different correct JSON responses

Answer : A

Explanation:

output_config with json_schema and strict: true provides grammar-level enforcement — the output is literally constrained to conform to the schema through grammar-based generation. This is the strongest possible guarantee. Tool definitions with forced tool_choice (Option B) also provide strong guarantees but at the tool-call level rather than the response level. Options A and D are probabilistic approaches.

Question 113:

A development team is optimizing an agentic system that uses prompt caching. The system makes 100 API calls per minute with stable tool definitions and system prompt. Currently, automatic caching is enabled with a single top-level cache_control. The architect wants to maximize cache hits. What improvement should they make?

- A. Increase the cache TTL to 1 hour for maximum duration

- B. Switch from automatic to explicit caching with up to 4 breakpoints placed on the last stable content blocks
- C. Remove all caching to reduce per-request overhead
- D. Add `cache_control` to every content block for maximum granularity

Answer : B

Explanation:

Automatic caching with a single breakpoint auto-moves forward, which may miss optimal placement. Explicit caching with up to 4 breakpoints placed strategically on the last stable blocks (tool defs, system prompt, early stable messages) maximizes prefix matching. Option B increases cost (2x write price) and may not be needed at 100 calls/minute if the 5-min TTL is sufficient. Option C removes cost savings. Option D isn't how breakpoints work — only up to 4 are valid.

Question 114:

A development team wants to test a new prompt strategy for their agent without affecting the production `CLAUDE.md` configuration. They want temporary instructions that only apply during testing. What is the most appropriate approach?

- A. Create a temporary branch and modify the project `CLAUDE.md` for testing
- B. Add a `.claude/rules/testing.md` file with a feature flag that enables the new strategy
- C. Modify `~/claude/CLAUDE.md` with testing instructions since user-level overrides project-level
- D. Use `--append-system-prompt` with the testing instructions when invoking Claude Code

Answer : D

Explanation:

The `--append-system-prompt` flag adds instructions to the default system prompt without modifying any configuration files. This is ideal for temporary testing scenarios. Option A creates version control overhead for temporary changes. Option C is incorrect about precedence — user-level doesn't override project-level, and it would affect all projects. Option D requires post-test cleanup.

Question 115:

A team needs to configure different models for different agent roles in their Claude Code setup. The main agent should use Opus for complex reasoning, while the Explore subagent already uses Haiku. They also want a custom 'reviewer' subagent to use Sonnet for cost-effective review. How can they specify the model for the reviewer subagent?

- A. In the reviewer's `SKILL.md` frontmatter using the 'model' field
- B. Using environment variables that specify the model per subagent
- C. In the reviewer's agent definition frontmatter using the 'model' field
- D. Models cannot be configured per subagent — all agents use the same model

Answer : C

Explanation:

Custom subagent definitions support a 'model' field in their frontmatter configuration, allowing different agents to use different models. The Explore subagent uses this mechanism to specify Haiku. A reviewer subagent definition can similarly specify Sonnet. Option A uses skills which are a different construct. Option C doesn't provide the documented configuration mechanism. Option D is incorrect — per-agent model selection is supported.

Question 116:

A developer is building an MCP tool for document search. The tool should accept a query string and optional filters (date range, document type, author). The tool's `input_schema` defines all parameters. The development team finds that Claude frequently omits the optional filters even when the user's request clearly mentions them. What improvement would help?

- A. Make all filter parameters required instead of optional
- B. Add `input_examples` showing queries that correctly use optional filters alongside the query string
- C. Add a 'note' in the system prompt listing all available filter parameters
- D. Split the tool into separate tools — one for simple search and one for filtered search

Answer : B

Explanation:

`input_examples` in the tool definition show concrete usage patterns including optional parameters. When Claude sees examples of correctly structured inputs with optional filters, it learns to use them appropriately. Option A forces unnecessary parameters on simple queries. Option C adds instructions but not contextual examples. Option D creates tool proliferation, degrading overall tool selection quality.

Question 117:

Your team is designing a multi-context-window workflow for a complex refactoring task. The task requires deep analysis of the existing code, design of the new architecture, and implementation. Each phase may approach context limits. What is the recommended approach from the official documentation?

- A. Use the first context window for framework and analysis, then start new context windows for iteration, carrying forward only the essential findings
- B. Complete all phases in a single context window, using `/compact` between phases
- C. Divide the work across three entirely independent sessions with no shared context
- D. Use `fork_session` to create branches for each phase from a shared base

Answer : A

Explanation:

The official prompting guide recommends multi-context-window workflows where the first window establishes the framework and analysis, and future windows handle iteration with only essential findings carried forward. This manages context limits while maintaining coherence. Option A risks context degradation. Option C loses cross-phase context entirely. Option D shares a base but doesn't structure the multi-phase workflow.

Question 118:

A developer has configured MCP servers using environment variable expansion (`${GITHUB_TOKEN}`) in their `.mcp.json` file. On a new team member's machine, the MCP server fails to connect. The `.mcp.json` file is identical. What is the most likely cause?

- A. The new team member's machine has a different version of the MCP protocol
- B. The `.mcp.json` file needs to be regenerated on each machine
- C. The `GITHUB_TOKEN` environment variable is not set in the new team member's local environment
- D. Environment variable expansion only works in `~/.claude.json`, not `.mcp.json`

Answer : C

Explanation:

Environment variable expansion in `.mcp.json` uses `${VARIABLE_NAME}` syntax, which resolves to the local environment variable value. If the new team member hasn't set `GITHUB_TOKEN` in their local environment, the expansion will fail causing the connection error. Option A is unlikely since MCP protocol versions are managed by the client. Option C is incorrect — the file is the same. Option D is incorrect — `.mcp.json` supports env var expansion.

Question 119:

A team is implementing test-driven development with Claude Code. They want to write tests first, then use Claude to implement the code to pass those tests, iterating by sharing test failures. Which development workflow pattern does this represent?

- A. The test-driven iteration pattern: write tests first, then iterate by sharing failures
- B. Plan mode followed by direct execution
- C. The interview pattern: have Claude ask questions before implementing
- D. Prompt chaining with separate steps for testing and implementation

Answer : A

Explanation:

The exam guide specifically describes the test-driven iteration pattern: write tests first, then iterate by sharing test failures with Claude. This leverages concrete test expectations as unambiguous specifications for Claude to implement against. Option A is a general workflow mode. Option C is a different interaction pattern. Option D describes a prompt engineering technique, not a development workflow.

Question 120:

An architect notices that during a multi-agent research task, the coordinator's context is being consumed primarily by verbose tool results from subagents — each subagent returns full page contents, reasoning traces, and metadata alongside findings. By the time the coordinator needs to synthesize, 80% of its context is tool results. What design change best addresses this?

- A. Increase the coordinator's context window to accommodate the verbose results
- B. Reduce the number of subagents to limit the total volume of results
- C. Have the coordinator summarize each subagent result before storing it in the conversation
- D. Trim verbose tool outputs to relevant fields before accumulation in the coordinator's context

Answer : D

Explanation:

The exam guide recommends trimming verbose tool outputs to relevant fields before they accumulate in the conversation context. Tool results that include full page contents, raw data, and reasoning traces consume disproportionate context. Trimming preserves essential findings while freeing context for synthesis. Option A doesn't address the root cause. Option C adds additional processing steps. Option D reduces research coverage.

Question 121:

A developer is debugging an issue where Claude Code loads CLAUDE.md from the project root but ignores a CLAUDE.md in the packages/api/ subdirectory when editing files in that directory. What is the most likely explanation?

- A. Only one CLAUDE.md file is loaded per session — the root level takes precedence
- B. Subdirectory CLAUDE.md files are loaded automatically when editing files in that directory, but macOS file system casing may cause path mismatch
- C. Subdirectory CLAUDE.md files must be explicitly imported using @import in the root CLAUDE.md
- D. The packages/api/CLAUDE.md file may have syntax errors or be excluded by claudeMdExcludes setting

Answer : D

Explanation:

The `claudeMdExcludes` setting can exclude specific directories from `CLAUDE.md` loading, which is common in monorepos. If `packages/api/` is excluded (perhaps to manage context from too many `CLAUDE.md` files), the subdirectory file won't be loaded. Syntax errors could also prevent loading. Option A is incorrect — the hierarchy supports multiple files. Option B is unlikely but possible. Option D is incorrect — subdirectory files load automatically.

Question 122:

An architect needs to implement a system where Claude generates a response, then a separate validation step checks the response against business rules, and if validation fails, the error is fed back to Claude for correction. This pattern should repeat until validation passes or a maximum attempt count is reached. What is this pattern called and how should it be implemented?

- A. The validation-retry loop — append specific validation error messages to the prompt context for each retry iteration
- B. The retry pattern — implement with a simple loop that retries the same prompt on failure
- C. The feedback pattern — have a second Claude instance critique the first instance's output
- D. The convergence pattern — gradually increase temperature on each retry to explore different solutions

Answer : A

Explanation:

The exam guide describes the validation-retry loop pattern where specific validation errors are appended to the prompt for each retry. This allows Claude to learn from the specific failure and correct it. The key is appending error feedback, not just retrying blindly. Option A retries without feedback (ineffective). Option C adds unnecessary complexity. Option D increases randomness without targeted correction.

Question 123:

A team is building a Claude Agent SDK application that needs to process work autonomously without user interaction. They want structured JSON output for downstream consumption. Which SDK configuration is most appropriate?

- A. Use `query()` with the prompt and parse Claude's text response as JSON
- B. Use `claude -p` with `--output-format json` and `--json-schema` to get guaranteed structured output
- C. Use `query()` with `allowed_tools` that include a custom JSON output tool
- D. Pipe Claude's output through `jq` for JSON extraction and formatting

Answer : B

Explanation:

The -p flag (non-interactive mode) with --output-format json and --json-schema provides guaranteed structured output that conforms to the specified schema. The structured output appears in the structured_output field of the JSON response. Option A has no format guarantee. Option C requires custom tool infrastructure. Option D is a fragile post-processing approach.

Question 124:

A development team runs Claude Code in CI to generate test code. They notice inconsistent behavior between developers' local runs and CI runs because of different CLAUDE.md files, installed skills, and MCP servers on each developer's machine. What ensures consistent CI execution?

- A. Running CI jobs on standardized Docker containers with identical configurations
- B. Using the --bare flag which skips auto-discovery of all local configuration (hooks, skills, plugins, MCP, CLAUDE.md)
- C. Setting a CLAUDE_CI=true environment variable that enables CI-specific behavior
- D. Storing all developer configurations in version control and loading them in CI

Answer : B

Explanation:

--bare mode explicitly skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md. Only explicitly passed flags take effect, ensuring identical behavior regardless of the machine's local configuration. This is the recommended mode for scripted and SDK calls. Option A adds infrastructure overhead. Option C references a non-existent variable. Option D may not catch all local variations.

Question 125:

A solutions engineer is designing a content generation system that needs to produce marketing copy for products in a specific brand voice. The copy must always include a product name, tagline, description (50-100 words), and exactly 3 bullet points. What combination of techniques best achieves consistent, branded output?

- A. A tool definition with JSON schema enforcing the structure plus few-shot examples demonstrating the brand voice
- B. Detailed system prompt describing the brand voice with output format instructions
- C. Few-shot examples alone showing the desired brand voice and format
- D. output_config with a strict schema and no additional prompting

Answer : A

Explanation:

Combining a tool definition with JSON schema (for structural guarantees: product name, tagline, description, 3 bullet points) with few-shot examples (for brand voice consistency) addresses both

dimensions. Schema alone guarantees structure but not style. Few-shot alone demonstrates style but doesn't guarantee structure. Option A is probabilistic. Option D enforces structure but doesn't guide brand voice.

Question 126:

A Claude Code user runs `/compact` during a debugging session. After compaction, they notice the agent no longer references specific file paths and function names from earlier investigation. What should the user do to preserve investigation findings across compaction?

- A. Avoid using `/compact` and instead manually remove irrelevant messages from the conversation
- B. Re-read all previously investigated files after compaction to rebuild context
- C. Use scratchpad files to persist key findings (file paths, function names, hypotheses) before compacting
- D. Configure compaction to preserve a higher percentage of the original conversation

Answer : C

Explanation:

The exam guide recommends using scratchpad files to persist key findings across context boundaries, including compaction. Writing investigation findings (file paths, function names, hypotheses, conclusions) to a file ensures they survive compaction. Option A doesn't address context limits. Option C wastes context re-reading files. Option D doesn't exist as a user-configurable option.

Question 127:

An architect is implementing a customer support agent that receives a customer inquiry mentioning both a shipping delay and a product defect. The current system routes the entire inquiry to the shipping department. What design change properly handles both concerns?

- A. Train the routing model with more examples of multi-concern inquiries
- B. Decompose the inquiry into two separate tickets: one for shipping and one for product quality
- C. Route to a senior agent who handles all types of inquiries
- D. Process the shipping concern first, then follow up on the product defect

Answer : B

Explanation:

The exam guide recommends decomposing multi-concern requests into distinct items that can be investigated independently. Creating separate tickets ensures each concern is routed to the appropriate

specialist rather than having one department handle both. Option A may improve but doesn't solve the structural problem. Option C doesn't leverage specialization. Option D may lose track of the second concern.

Question 128:

A developer is configuring skill content lifecycle management. They have a verbose security audit skill that generates over 8000 tokens of content. They're concerned this will survive compaction and permanently consume context. What is the skill content limit for compaction survival?

- A. No limit — all skill content survives compaction indefinitely
- B. 5000 tokens per skill, with a total budget of 25000 tokens across all skills
- C. 2000 tokens per skill, with automatic truncation of excess content
- D. Skill content is completely dropped during compaction and must be reloaded

Answer : B

Explanation:

Skill content survives compaction up to 5000 tokens per skill, with a total budget of 25000 tokens across all active skills. Content exceeding these limits may be truncated during compaction. At 8000 tokens, this skill exceeds the per-skill limit, so the developer should either reduce the skill's output or use context: fork to isolate it. Options A, C, and D state incorrect limits.

Question 129:

An architect implements an MCP connector at the API level. They configure `mcp_servers` and notice that tool calls from the MCP server use different content block types than regular `tool_use` blocks. What are the MCP-specific content block types?

- A. `mcp_request` and `mcp_response` blocks that wrap standard `tool_use` format
- B. Standard `tool_use` and `tool_result` blocks with an `'mcp_server'` field indicating the source
- C. `mcp_tool_use` and `mcp_tool_result` content blocks that extend the standard tool calling format
- D. External `tool_use` blocks that reference the MCP server by name

Answer : C

Explanation:

The MCP Connector uses `mcp_tool_use` and `mcp_tool_result` content blocks that extend the standard tool calling format. These blocks include additional fields like the MCP server name and follow the same general patterns as regular tool calls but are specifically typed for MCP. Option A uses non-existent block types. Option C incorrectly describes them as standard blocks. Option D uses a non-existent block type.

Question 130:

A team wants to configure their Claude agent to use adaptive thinking that dynamically adjusts reasoning depth. They're migrating from a system that used manual `budget_tokens`. What is the current recommended configuration for Claude 4.6 Opus?

- A. Continue using `budget_tokens` as it provides more precise control over thinking depth
- B. Use thinking: `{type: 'adaptive'}` with the effort parameter to control thinking depth dynamically
- C. Set thinking: `{type: 'enabled'}` and let the model self-regulate thinking depth
- D. Disable thinking entirely and use Chain-of-Thought prompting instead

Answer : B

Explanation:

For Claude 4.6 Opus, the recommended approach is adaptive thinking (type: 'adaptive') combined with the effort parameter. This replaces the deprecated `budget_tokens` approach. Adaptive thinking lets Claude dynamically decide when and how deeply to think based on task complexity, while effort provides directional control. Option A uses the deprecated approach. Option C uses basic enabled mode without adaptive capability. Option D removes a key feature.

Question 131:

A developer building an API client for an agentic loop notices that the assistant's response sometimes contains multiple `tool_use` blocks in a single response. How should the developer handle this?

- A. Execute only the first `tool_use` block and ignore the rest
- B. Execute all `tool_use` blocks (potentially in parallel) and return all `tool_result` blocks in the next user message
- C. Return an error because only one `tool_use` block should appear per response
- D. Queue the `tool_use` blocks and execute them one at a time across multiple API calls

Answer : B

Explanation:

When Claude's response contains multiple `tool_use` blocks, they should all be executed (potentially in parallel) and all corresponding `tool_result` blocks should be included in the next user message. Claude naturally supports parallel tool calling. Option A throws away intended tool calls. Option C is incorrect — multiple `tool_use` blocks are valid. Option D unnecessarily serializes what could be parallel work.

Question 132:

A team is processing 50,000 compliance documents through the Message Batches API. They need to correlate each response with its input document. How should they track this correspondence?

- A. Use the custom_id field in each batch request to include a unique document identifier
- B. Process documents sequentially and rely on response order matching input order
- C. Include the document ID in the prompt text and extract it from the response
- D. Use the API's built-in request_id field that auto-generates tracking IDs

Answer : A

Explanation:

The Message Batches API provides a custom_id field specifically for correlating requests with responses. Each request can include a unique custom_id that appears in the corresponding response, enabling reliable mapping between input documents and output results. Option A is unreliable since batch responses may arrive in any order. Option C is fragile. Option D describes a non-existent feature.

Question 133:

A technical lead is building a document processing system and wants to use Claude effectively for long documents (200+ pages). The official prompting guide recommends a specific layout for long context prompts. What is the recommended structure?

- A. Query first, then the document, so Claude knows what to look for while reading
- B. Document split into chunks with queries interspersed between relevant chunks
- C. Document as a system message with queries as user messages for role separation
- D. Longform data at the top of the prompt with queries/instructions at the end, for approximately 30% quality improvement

Answer : D

Explanation:

The official Claude prompting best practices recommend placing longform data at the top of the prompt and queries/instructions at the end. This layout provides approximately 30% quality improvement for long context tasks. This counters the intuitive approach of putting the question first. Option A puts queries first which is less effective. Option B fragments the document unnecessarily. Option D doesn't follow the recommended pattern.

Question 134:

A multi-agent system coordinator needs to pass context from Agent A to Agent B. Agent A discovers that a legacy database uses a non-standard date format (YYYYDDMM instead of YYYYMMDD). If the coordinator only passes Agent A's final results without this discovery, Agent B will misinterpret all dates from the database. What metadata practice prevents this issue?

- A. Have Agent A reformat all dates before returning results so Agent B doesn't need to know about the non-standard format

- B. Include metadata about data format peculiarities, sources, and methodologies alongside Agent A's findings when passing to Agent B
- C. Create a shared configuration file that all agents reference for data format information
- D. Have the coordinator request Agent A to re-run its analysis with explicit format conversion

Answer : B

Explanation:

The exam guide emphasizes requiring subagents to include metadata — sources, methodologies, format details, dates — alongside findings. Including the data format discovery ensures Agent B interprets dates correctly and can make informed decisions. Option A may not catch all date instances or edge cases. Option C adds infrastructure without guaranteeing agents reference it. Option D wastes computation.

Question 135:

A developer notices that their Claude Code agent sometimes uses subagents for simple, focused tasks that could be handled directly, increasing costs and latency. The official documentation notes this tendency with Claude 4.6. What is the recommended mitigation?

- A. Add guidance in CLAUDE.md or system prompt specifying when subagent use is warranted versus handling tasks directly
- B. Disable subagent capabilities entirely to prevent overuse
- C. Set a minimum character count for prompts that can trigger subagent spawning
- D. Reduce the model's temperature to make it less likely to choose creative delegation patterns

Answer : A

Explanation:

The official prompting guide notes that Claude 4.6 may overuse subagents and recommends adding explicit guidance about when subagent delegation is warranted. CLAUDE.md instructions can specify criteria like task complexity, expected context isolation needs, or output volume that justify subagent use. Option A removes a valuable capability. Option C uses an arbitrary metric. Option D conflates temperature with delegation behavior.

Question 136:

An AI engineer is configuring MCP resources alongside MCP tools for a customer support agent. The agent currently makes multiple exploratory tool calls to discover available knowledge base articles. What MCP feature reduces these exploratory calls?

- A. Pre-fetching all tool results at session start and caching them

- B. Tool definitions with comprehensive descriptions listing all available articles
- C. MCP resources that expose content catalogs, allowing the agent to browse available content without tool calls
- D. A master query tool that returns all available articles in a single call

Answer : C

Explanation:

MCP resources expose content catalogs that the agent can browse without making tool calls. This reduces exploratory tool calls by providing a discoverable view of available content. Option A wastes resources fetching everything upfront. Option C is static and quickly outdated. Option D returns excessive data in one call.

Question 137:

A Claude Code project has the following CLAUDE.md hierarchy: root CLAUDE.md says 'use tabs for indentation', and packages/frontend/CLAUDE.md says 'use 2 spaces for indentation.' When editing a file in packages/frontend/, which indentation rule applies?

- A. Tabs (root level always takes precedence)
- B. An error occurs due to conflicting instructions
- C. 2 spaces (subdirectory CLAUDE.md overrides root for files in that directory)
- D. Both are loaded and Claude will randomly choose between them

Answer : C

Explanation:

In the CLAUDE.md hierarchy, subdirectory files add context specific to their directory. When editing files in packages/frontend/, the subdirectory CLAUDE.md provides the more specific instruction. The hierarchy is designed so that more specific (closer to the file being edited) instructions take precedence. Option A incorrectly states root always wins. Options C and D don't reflect the documented behavior.

Question 138:

A developer is designing a tool that needs to handle both successful results and errors. According to MCP best practices, when a tool encounters a business rule violation (e.g., refund amount exceeds policy limit), how should the error be communicated back to the agent?

- A. Return isError: true with a generic 'Operation failed' message
- B. Throw an exception that terminates the agentic loop
- C. Return isError: false with the policy limit details embedded in the success response
- D. Return isError: true with structured metadata including errorCategory='policy_violation', isRetryable: false, and a customer-friendly description

Answer : D

Explanation:

MCP tools should return structured error metadata that helps the agent make informed decisions. For a business rule violation: `errorCategory` identifies the type, `isRetryable: false` prevents useless retries, and a customer-friendly description can be relayed to the user. Option A provides no recovery guidance. Option B is overly aggressive. Option D misrepresents an error as success.

Question 139:

An architect is implementing context awareness for their agent system. They want the agent to proactively manage context before hitting the context window limit. Which Claude models support context awareness (tracking remaining token budget)?

- A. All Claude models automatically track context window usage
- B. Context awareness is a client-side feature that works with any model
- C. Only Opus 4.6 supports context awareness as it requires the highest capability model
- D. Sonnet 4.6, Sonnet 4.5, and Haiku 4.5 support context awareness with budget tracking

Answer : D

Explanation:

Context awareness — where the model receives `budget:token_budget` tags at conversation start and `system_warning` updates about token usage — is supported by Sonnet 4.6, Sonnet 4.5, and Haiku 4.5. This allows these models to proactively manage context. Notably, Opus 4.6 is not listed. Option A is incorrect. Option C incorrectly identifies Opus. Option D confuses it with a client-side feature.

Question 140:

A team building a multi-agent system notices that when the coordinator sends a task description to a subagent, the subagent performs better when given goals and quality criteria rather than step-by-step procedures. Why is this the case?

- A. Goals and quality criteria allow the subagent to reason about the best approach, leveraging its own capabilities rather than following rigid instructions
- B. Step-by-step procedures exceed the subagent's context window limit
- C. Subagents are only capable of executing goals, not following procedures
- D. Step-by-step procedures introduce latency in the subagent's execution

Answer : A

Explanation:

The exam guide recommends that coordinator prompts specify goals and quality criteria rather than step-by-step procedures. This allows the subagent to leverage its reasoning capabilities to determine the best approach for the specific situation, which often leads to better outcomes than following rigid

procedures. Option A is unlikely for reasonable instructions. Option C understates subagent capabilities. Option D is incorrect — procedure length doesn't significantly affect latency.

Question 141:

A developer wants to run a complex Claude Code skill that requires file system access but should not be triggered automatically by the model during normal conversations. The skill should only run when explicitly invoked by the user via a slash command. Which frontmatter field controls this?

- A. user-invocable: true combined with disable-model-invocation: true
- B. auto-invoke: false to prevent automatic triggering
- C. trigger: manual to restrict to user-initiated invocations
- D. interactive-only: true to require explicit user action

Answer : A

Explanation:

The disable-model-invocation frontmatter field prevents the model from triggering the skill during normal conversations, while user-invocable: true ensures it can still be invoked via slash commands. This combination restricts the skill to explicit user invocation only. Options A, C, and D use non-existent frontmatter fields.

Question 142:

An architect notices that in their agent system, when Claude encounters an ambiguous tool (both get_product_info and search_products could match a user's request), it consistently picks the wrong one. Both tools have one-sentence descriptions. What is the primary mechanism Claude uses for tool selection?

- A. Claude analyzes the tool's input schema to determine parameter compatibility
- B. Tool descriptions are the primary mechanism — Claude uses them to understand each tool's purpose and appropriate use cases
- C. Claude selects based on the tool name matching keywords in the user's message
- D. Claude uses statistical patterns from training data to predict which tool is more commonly used

Answer : B

Explanation:

The exam guide explicitly states that tool descriptions are the primary mechanism LLMs use for tool selection. With minimal one-sentence descriptions, Claude lacks the detail needed to distinguish between similar tools. Expanding descriptions with input formats, example queries, edge cases, and boundary explanations resolves misrouting. Option A is secondary. Option B oversimplifies. Option D isn't how API-based tool calling works.

Question 143:

When using server-side compaction, an architect wants to track the actual tokens billed for a conversation that has undergone multiple compactions. The top-level usage fields show lower numbers than expected. Where should they look for complete billing information?

- A. The X-Usage-Total response header that includes all compaction tokens
- B. The billing dashboard which automatically sums all token usage including compaction
- C. The usage.iterations array which shows tokens for each compaction and message iteration separately
- D. The usage.compaction_tokens field which tracks compaction-specific usage

Answer : C

Explanation:

The top-level usage fields do NOT include compaction iteration tokens. The complete token usage must be calculated by summing across the usage.iterations array, which separately tracks each compaction iteration and each message iteration. This explains why the top-level numbers are lower than expected. Option A references a non-existent header. Option C may be correct for billing but not for per-request tracking. Option D references a non-existent field.

Question 144:

A team wants to build a Claude agent that creates initial project scaffolding scripts. They want Claude to produce an init.sh file that sets up the project environment. According to the official agentic prompting guide, what is the benefit of this approach?

- A. init.sh scripts provide graceful starts for future context windows, establishing the working environment reliably
- B. Shell scripts are more reliable than Claude's direct file operations
- C. Shell scripts bypass Claude's permission system for faster setup
- D. init.sh is a reserved filename that Claude Code treats specially

Answer : A

Explanation:

The agentic prompting guide recommends creating setup/init scripts for graceful starts. These scripts establish the working environment reliably and can be re-run in future context windows to quickly restore the project state. Option A isn't the primary rationale. Option C is incorrect about permission bypassing. Option D is incorrect — init.sh is not a reserved name.

Question 145:

A developer is building an agent that needs to read, modify, and write files in a codebase. During a modification, the Edit tool fails because the search string appears in three places in the file. The developer wants to modify only the second occurrence. What is the correct approach?

- A. Use Edit with a 'occurrence: 2' parameter to target the second match
- B. Include more surrounding context in the Edit search string to uniquely identify the second occurrence
- C. Use Bash with sed to target the specific line number containing the second occurrence
- D. Read the file, use Write to replace the entire content with the modification applied to the correct occurrence

Answer : D

Explanation:

When Edit fails due to non-unique matches and adding context still doesn't uniquely identify the target, the documented fallback is Read + Write: read the complete file, make the modification programmatically at the correct location, and write the complete file back. Option A references a non-existent parameter. Option B may work if sufficient context exists. Option C introduces shell complexity. Option D (Read + Write) is the documented fallback pattern.

Question 146:

A support agent handles a customer who asks the agent to check their order status. The agent matches the customer to two accounts with similar names. The agent selects the more recently active one and provides order status for that account. Later, it's discovered this was the wrong account. What should the agent have done differently?

- A. Checked both accounts' order statuses and presented both options to the customer
- B. Escalated to a human agent because the ambiguity was too complex to resolve
- C. Used IP geolocation to determine which account was more likely the customer's
- D. Asked the customer for additional identifying information such as email address or account number to disambiguate

Answer : D

Explanation:

The exam guide explicitly states that when multiple customer matches exist, the agent should request additional identifiers rather than using heuristic selection (such as recency). Selecting based on activity level is a heuristic that can lead to accessing the wrong account. Option A reveals both accounts' information unnecessarily. Option B is premature — the situation is resolvable with more info. Option C uses unreliable data.

Question 147:

A platform engineer is designing an agentic system and needs to decide how Claude determines which tools to call. They're debating between a pre-configured decision tree (if user mentions 'order', call lookup_order) versus letting Claude reason about tool selection based on context. Which approach does the exam guide recommend?

- A. Pre-configured decision trees provide more reliable tool selection
- B. Model-driven tool selection based on context, not pre-configured decision trees or tool sequences
- C. A hybrid approach where decision trees handle common cases and Claude handles edge cases
- D. A routing layer that classifies the request type before passing to Claude

Answer : B

Explanation:

The exam guide emphasizes that Claude should reason about which tool to call based on context, NOT follow pre-configured decision trees or tool sequences. This leverages Claude's reasoning capability to handle nuanced, varied situations. Option A contradicts the guide. Option C partially constrains Claude's reasoning. Option D adds unnecessary complexity for classification already within Claude's capability.

Question 148:

A developer is building a Claude-powered system that processes both time-sensitive user requests and large-scale data analysis. They want to optimize costs without impacting latency for user-facing requests. Which combination of API features best achieves this?

- A. Use batch processing for everything and implement a priority queue for urgent requests
- B. Use streaming for all requests to reduce perceived latency
- C. Use real-time API with prompt caching for user-facing requests; Message Batches API for non-urgent large-scale analysis
- D. Use effort='low' for all requests to minimize costs

Answer : C

Explanation:

Prompt caching reduces costs by 90% on cache reads for real-time requests while maintaining low latency. Message Batches API provides 50% cost savings for non-urgent analysis with no latency guarantee (up to 24 hours). This combination optimizes costs for both use cases. Option A adds latency to urgent requests. Option C doesn't reduce costs. Option D may reduce quality on complex analysis.

Question 149:

An architect implements agent teams for a complex software project. They configure 8 teammates, each assigned 10+ tasks. The team frequently experiences file conflicts and coordination overhead. What best practice should be applied?

- A. Reduce to 3-5 teammates with 5-6 tasks each, and assign tasks to avoid same-file edits across teammates
- B. Increase the team size to 12 teammates to reduce the per-teammate workload
- C. Add a dedicated 'conflict resolution' teammate to handle merge conflicts
- D. Switch from agent teams to subagents to eliminate coordination overhead

Answer : A

Explanation:

The agent teams best practices specify 3-5 teammates for most workflows and 5-6 tasks per teammate. Teams should be designed to avoid same-file edits across teammates, which causes conflicts. The current setup of 8 teammates with 10+ tasks violates both guidelines. Option A makes the problem worse. Option C addresses symptoms. Option D loses the benefits of agent collaboration.

Question 150:

A developer is working with Claude on a large codebase and encounters the interview pattern. When should this pattern be used according to the official documentation?

- A. When the developer wants Claude to generate documentation for the entire codebase
- B. When the developer needs Claude to interview end users for requirements gathering
- C. When multiple developers need to collaborate on the same Claude session
- D. When the task is ambiguous and having Claude ask clarifying questions before implementing leads to better outcomes

Answer : D

Explanation:

The interview pattern involves having Claude ask questions before implementing, which is recommended when the task is ambiguous. This ensures Claude understands the requirements and constraints before writing code, leading to better outcomes. Option A describes a documentation task, not the interview pattern. Option C describes a collaboration scenario. Option D is outside Claude Code's scope.

Question 151:

A senior developer is building an agent that processes customer feedback surveys. The surveys may contain numerical ratings, open-ended text, or both. The extraction schema uses strict mode for guaranteed conformance. During testing, Claude refuses to produce output for a survey that contains only profanity with no constructive feedback. What is happening?

- A. Strict mode doesn't support free-text fields in the schema
- B. The schema requires fields that the input cannot populate, causing a validation failure

- C. The profanity triggers a content filter separate from schema validation
- D. On refusal or max_tokens, structured outputs with strict mode may produce invalid output since the schema can only be guaranteed on successful end_turn completions

Answer : D

Explanation:

The structured outputs documentation notes that invalid outputs can occur on refusal or max_tokens — the schema guarantee only applies when stop_reason is end_turn. When Claude refuses due to content concerns, it may not produce valid schema-compliant output. Developers must handle these edge cases. Option A is incorrect. Option B describes a different issue. Option D is partially correct but doesn't explain the invalid output.

Question 152:

A team is building an agentic system that needs to handle financial transactions. They implement hooks for compliance but also want Claude to provide a good customer experience by explaining why certain operations are blocked. Which hook type allows injecting explanatory feedback to the model when blocking an operation?

- A. PreToolUse with exit code 2 and a JSON output containing the block reason that Claude uses to explain to the customer
- B. PreToolUse with exit code 0 and a description in stdout
- C. PostToolUse with a modified tool result that includes the block reason
- D. Notification hook that sends the block reason directly to the customer

Answer : A

Explanation:

PreToolUse hooks with exit code 2 block the operation. The JSON output from the hook can include a reason/description that Claude receives and uses to craft a customer-friendly explanation. Exit code 0 proceeds without blocking (Option A). PostToolUse (Option C) fires after the tool executes. Notification hooks (Option D) don't interact with the conversation flow.

Question 153:

A developer is configuring the MCP Connector at the API level and needs to pass an authorization token for their MCP server. Where does the token go in the configuration?

- A. In a separate authentication header alongside the Anthropic API key
- B. In a .env file that the MCP server reads independently
- C. In the system prompt as a tool-accessible credential
- D. In the mcp_servers array configuration: {type: 'url', url, name, authorization_token}

Answer : D

Explanation:

The MCP Connector configuration includes an `authorization_token` field directly in the `mcp_servers` array: `{type: 'url', url: '...', name: '...', authorization_token: '...'}`. This token is passed to the MCP server during connection. Option A conflates API authentication with MCP server auth. Option C exposes credentials in the prompt. Option D requires server-side configuration.

Question 154:

An architect needs to understand the structured output schema complexity limits for Claude 4.6 with strict mode. Their schema has 22 tools with strict mode enabled. What issue will they encounter?

- A. Strict mode only supports 10 tools maximum per request
- B. There is no limit on the number of strict tools as long as total schema size is under 100KB
- C. Strict mode has a maximum of 20 strict tools; the schema will need to be simplified
- D. The 22 tools will work but compilation will take longer than the API timeout

Answer : C

Explanation:

The structured outputs documentation specifies schema complexity limits: maximum 20 strict tools, 24 optional parameters per object, and 16 types per union. At 22 strict tools, the schema exceeds the 20-tool limit and will need simplification. Option B incorrectly states no limit. Options C and D state incorrect limits.

Question 155:

A team is building a knowledge management agent that needs to compile information from multiple sources. Some sources provide conflicting statistics (e.g., two reports show different market sizes for the same market). How should the synthesis agent handle these conflicts?

- A. Average the conflicting values to provide a balanced estimate
- B. Use the most recent source's value as more likely to be accurate
- C. Flag the conflict and escalate to a human reviewer for resolution
- D. Annotate the conflicts with full attribution for each source, presenting both values with their provenance rather than arbitrarily selecting one

Answer : D

Explanation:

The exam guide requires annotating conflicts with attribution rather than arbitrarily selecting one value. Both statistics should be presented with their sources, publication dates, and methodologies so the

consumer can make an informed judgment. Option A manufactures a new number. Option B assumes recency equals accuracy. Option D unnecessarily escalates what can be transparently handled.

Question 156:

A developer wants to reference a file's content dynamically within a SKILL.md file. They want the skill to always have the latest version of a configuration file injected into its context. Which SKILL.md feature supports this?

- A. The `!command`` syntax for dynamic context injection that runs a command and injects its output
- B. `@import` syntax for referencing external files
- C. A 'files' frontmatter field listing files to include in skill context
- D. `${FILE_PATH}` variable substitution pointing to the file location

Answer : A

Explanation:

SKILL.md supports the `!command`` syntax (backtick-wrapped commands) for dynamic context injection. A command like `!cat config.json`` would read the latest file content and inject it into the skill's context at invocation time. Option A is for CLAUDE.md, not skills. Options C and D use non-existent features.

Question 157:

An agent system processes insurance claims. When a claim involves multiple coverage types (auto + medical), the agent currently processes all coverage types in a single analysis pass but occasionally misses the secondary coverage. What scheduling approach helps ensure comprehensive coverage?

- A. Decompose the multi-coverage claim into separate items and investigate each coverage type independently before combining results
- B. Create a fixed checklist of all coverage types that the agent must iterate through
- C. Increase the agent's context window to prevent information loss
- D. Route multi-coverage claims to a senior agent with broader training

Answer : A

Explanation:

Decomposing multi-concern requests into distinct items ensures each is addressed independently. For insurance claims with multiple coverage types, separately analyzing auto and medical coverage before combining results prevents the secondary coverage from being overshadowed. Option A doesn't address attention issues. Option B is rigid. Option D defers the problem.

Question 158:

A developer configures an MCP server and includes tool definitions with standardized namespacing. Which naming convention follows the recommended best practice?

- A. Tool names like 'tool1', 'tool2', 'tool3' with descriptions explaining each
- B. Tool names matching internal function names like 'handlePRList', 'sendSlackMsg'
- C. Tool names like 'github_list_prs', 'slack_send_message', 'jira_create_ticket' using service-based namespacing
- D. Single-word tool names like 'list', 'send', 'create' for simplicity

Answer : C

Explanation:

The MCP tool design best practices recommend meaningful namespacing using the pattern `service_action_target`, such as `github_list_prs`, `slack_send_message`. This makes tool purpose clear from the name alone and prevents ambiguity across services. Option A provides no semantic meaning. Option C uses implementation-specific naming. Option D is ambiguous across services.

Question 159:

An architect is building a system that generates compliance reports from regulatory databases. The system needs to run monthly reports on the 15th, processing 10,000 documents each time. The reports are consumed by analysts the next business day. The current real-time API approach costs \$2,000 per run. What API feature could reduce costs?

- A. Prompt caching to reduce repeated document processing costs
- B. Effort='low' to reduce per-document processing costs at the expense of quality
- C. Message Batches API for 50% cost savings, since the reports don't need real-time processing
- D. Smaller model (Haiku) for document processing at lower per-token cost

Answer : C

Explanation:

The Message Batches API provides 50% cost savings and processes batches within 24 hours. Since the reports are consumed the next business day, the latency tolerance makes this ideal — reducing costs from \$2,000 to approximately \$1,000. Option A helps with repeated prompts but not the volume cost. Option B may reduce report quality. Option D may reduce the accuracy needed for compliance.

Question 160:

A team notices that their agent's performance degrades when they add images to the prompt alongside thinking mode. After adding several product images for analysis, the context window fills more quickly. What explains this interaction between images, thinking, and context?

- A. Images are not compatible with extended thinking and should be processed separately
- B. All tokens including thinking tokens count toward the context window limit, so images plus thinking traces consume context faster than either alone
- C. Each image consumes fixed tokens regardless of thinking mode, limiting remaining capacity
- D. Images disable thinking mode automatically, reducing reasoning capability

Answer : B

Explanation:

The documentation states that all tokens — including thinking tokens — count toward the context window limit. When images (which consume significant tokens) are combined with extended thinking (which generates additional tokens), the effective available context shrinks faster. Previous thinking blocks are stripped from subsequent turns but current-turn thinking still counts. Options A and D are incorrect. Option B is partially true but misses the thinking interaction.

Question 161:

A developer is building a Claude Code workflow for database migration. They want to: (1) analyze the current schema, (2) generate migration SQL, (3) dry-run the migration, and (4) apply if successful. For step 1, the analysis is verbose and involves reading many files. Which workflow best preserves context for subsequent steps?

- A. Execute all four steps in sequence within the same conversation
- B. Use Explore subagent for step 1 (verbose analysis), then handle steps 2-4 in the main conversation with the Explore summary
- C. Create four separate conversations, one for each step
- D. Use plan mode for all four steps before executing any

Answer : B

Explanation:

The Explore subagent isolates verbose discovery output (schema analysis across many files) and returns only a summary to the main conversation. This preserves context for the subsequent migration generation, dry-run, and execution steps. Option A fills context with verbose analysis output. Option C loses continuity between steps. Option D plans all steps but doesn't address the context consumption of step 1.

Question 162:

An architect implements a PostToolUse hook that transforms tool results before Claude processes them. The hook strips sensitive PII fields from customer data returned by MCP tools. However, Claude still occasionally references PII in its responses. What is the most likely issue?

- A. PostToolUse hooks can't modify tool result content — only inspect it
- B. Claude retrieves PII from its training data rather than from tool results
- C. The PII was included in earlier messages in the conversation history, not just the most recent tool result
- D. The hook has a race condition allowing some results to pass through unmodified

Answer : C

Explanation:

PostToolUse hooks can transform tool results, but if PII appeared in earlier conversation messages or previous tool results that weren't transformed, Claude has access to that context. The hook only affects the current tool result, not the entire conversation history. Option A is incorrect — PostToolUse can transform results. Option C is unlikely for specific customer data. Option D is possible but less likely than historical context.

Question 163:

A team needs Claude to analyze a pull request with 14 files. They want thorough analysis of each file AND detection of cross-file issues (like incompatible API changes between a service and its consumer). Their initial approach of analyzing all files simultaneously produced inconsistent depth. What multi-pass strategy addresses this?

- A. Randomly sample 5 files for deep analysis and provide summary-level review for the rest
- B. Group files by directory and analyze each directory independently
- C. Single pass with explicit instructions to maintain consistent depth across all files
- D. First pass: per-file local analysis for thorough individual review. Second pass: cross-file integration analysis using summaries from the first pass

Answer : D

Explanation:

The exam guide's sample question describes exactly this multi-pass approach: per-file local analysis (ensuring thorough depth for each file) followed by a separate cross-file integration pass (catching inter-file issues like incompatible API changes). This addresses the root cause of attention dilution. Option A misses files. Option B may miss cross-directory issues. Option D repeats the failed approach.

Question 164:

A developer is using Claude Code and wants to reference content from another file within their CLAUDE.md. They want to include their team's coding standards from a separate document without duplicating the content. Which syntax supports this?

- A. #include './coding-standards.md' syntax similar to C preprocessor directives
- B. An 'includes' field in the CLAUDE.md YAML frontmatter
- C. @import syntax for referencing external files, keeping CLAUDE.md modular
- D. Symbolic links to the external document in the same directory as CLAUDE.md

Answer : C

Explanation:

CLAUDE.md supports @import syntax for referencing external files. This keeps the main CLAUDE.md modular while including content from other documents, such as coding standards maintained in a separate file. Option A uses an unsupported syntax. Option C references a non-existent feature. Option D is a filesystem trick that doesn't integrate with CLAUDE.md's loading mechanism.

Question 165:

Your organization is building a large-scale document processing pipeline and needs to decide on the right Claude model. Documents range from simple forms to complex legal contracts. They want 1M token context windows for large documents. Which models support the 1M token context window?

- A. All Claude 4 models support 1M token context windows
- B. Claude Opus 4.6, Sonnet 4.6, and Mythos Preview support 1M token context windows
- C. Only Claude Opus 4.6 supports 1M tokens; other models are limited to 200K
- D. Claude Sonnet 4.5 and Opus 4.5 support 1M tokens, while 4.6 models support 2M

Answer : B

Explanation:

The documentation specifies that Claude Opus 4.6, Sonnet 4.6, and Mythos Preview have 1M token context windows. Older models like Sonnet 4.5 and Sonnet 4 have 200K token context windows. Option A is incorrect — not all Claude 4 models have 1M. Option B omits Sonnet 4.6 and Mythos. Option D states incorrect values.

Question 166:

A developer is trying to understand when their MCP tool encountered an error versus when the tool executed successfully but found no results. The agent currently treats both cases identically by saying 'I couldn't find that information.' What tool response distinction should be implemented?

- A. Use HTTP status codes (404 for not found, 500 for errors) in the tool response

- B. Distinguish access failures (isError: true, isRetryable: true for timeouts) from valid empty results (isError: false with an empty result set)
- C. Return the same response format but include a log_level field (warning vs error)
- D. Always return isError: true and include a 'severity' field to differentiate

Answer : B

Explanation:

The exam guide specifically highlights the importance of distinguishing access failures (which should trigger retries or alternatives) from valid empty results (which mean the data simply doesn't exist). Timeouts get isError: true with isRetryable: true. 'No results found' gets isError: false with empty results. This enables the agent to retry appropriately or inform the customer accurately. Options A, B, and D don't make this critical distinction.

Question 167:

A team runs compaction on their agent conversation and then immediately sends a new request. The new request needs current market data. They configured pause_after_compaction: true. What stop_reason will they receive after compaction, and what should they do before continuing?

- A. stop_reason: 'end_turn' — the compaction completed and the conversation can continue normally
- B. stop_reason: 'tool_use' — compaction triggers a tool call to fetch updated data
- C. stop_reason: 'compaction' — they should inject updated market data into the conversation, then continue with the next request
- D. stop_reason: 'max_tokens' — the compaction summary consumed the token budget

Answer : C

Explanation:

With pause_after_compaction: true, the API returns stop_reason: 'compaction' after generating the summary. This gives the harness an opportunity to inject additional context (like updated market data) before continuing the conversation. Option A is incorrect about the stop_reason. Options C and D describe different mechanisms.

Question 168:

An architect wants to verify whether their Claude Code project has any conflicting settings between CLAUDE.md files at different levels. They suspect the root CLAUDE.md and a subdirectory CLAUDE.md have contradictory testing instructions. Which diagnostic approach is most efficient?

- A. Use the /memory command to see all loaded memory files and check for conflicts
- B. Read each CLAUDE.md file manually and compare instructions
- C. Run /init to regenerate the CLAUDE.md, which will detect and resolve conflicts
- D. Add a debug: true flag to the root CLAUDE.md to enable conflict logging

Answer : A

Explanation:

The /memory command shows which memory files (CLAUDE.md files at various levels) are loaded in the current session. This helps verify what Claude actually sees and identify conflicting instructions across levels. Option A works but is manual. Option C would overwrite existing configuration. Option D uses a non-existent feature.

Question 169:

A developer is building a complex agent with the Claude API and needs Claude to generate interleaved thinking blocks during tool use. On which model family is interleaved thinking automatically available with adaptive thinking mode?

- A. Claude 4 models (Opus 4.6, Sonnet 4.6) with adaptive thinking mode enabled
- B. Only on Sonnet 4.6 with explicit interleaved_thinking: true configuration
- C. All Claude models when thinking is set to 'enabled'
- D. Only on Mythos Preview as an experimental feature

Answer : A

Explanation:

Interleaved thinking is supported on Claude 4 models (Opus 4.6, Sonnet 4.6) and is automatically enabled with adaptive thinking mode. This allows Claude to think between tool calls, improving reasoning in multi-step agentic workflows. Option A is too broad. Option B requires no special flag. Option D incorrectly limits it to Mythos.

Question 170:

A team building a Claude-based code reviewer wants to reduce false positives where the tool flags intentional design patterns as issues. General instructions to 'be less aggressive' have not helped. According to the exam guide, what is the most effective technique?

- A. Add a whitelist of approved patterns that should never be flagged
- B. Lower Claude's confidence threshold for flagging issues
- C. Use a simpler model that generates fewer false positives overall
- D. Provide 2-4 targeted few-shot examples showing the specific false positive patterns and the correct behavior (not flagging them)

Answer : D

Explanation:

The exam guide explicitly states that general instructions like 'be conservative' fail to improve precision, while few-shot prompting with 2-4 targeted examples showing the specific problematic patterns and correct behavior is the most effective technique. Examples enable generalization to novel patterns beyond the specific cases shown. Option A is rigid and incomplete. Option B isn't a configurable parameter. Option D may miss true positives.

Question 171:

An architect implements a Claude agent with extended thinking. They strip thinking blocks from previous turns when sending subsequent requests. However, they notice that after a tool call, removing the thinking block causes an API error. What rule about thinking blocks and tool use are they violating?

- A. Thinking blocks must never be removed from the conversation history
- B. Thinking blocks associated with tool_use responses must be returned alongside the tool results — they can only be dropped after those tool results have been processed
- C. Only the model itself can decide when to remove thinking blocks
- D. Thinking blocks must be moved to the system prompt after tool use

Answer : B

Explanation:

When tool use and thinking co-occur, thinking blocks **MUST** be returned with their corresponding tool results. This is the one exception where thinking blocks must be preserved. After the model processes the tool results, the thinking blocks from that turn can be dropped for subsequent turns. Option A is too strict (they can be dropped after processing). Option C is incorrect. Option D is not how thinking blocks work.

Question 172:

A startup is evaluating the connection between agent teams and subagents in Claude Code. A team lead wants to use subagent definitions as templates for teammates. Is this possible?

- A. No — agent teams and subagents are completely separate systems with no interoperability
- B. Yes — subagent definitions can be referenced when spawning teammates, allowing reuse of agent configurations
- C. No — teammates must be fully defined at spawn time with no reference to existing definitions
- D. Yes — but only for the Explore subagent which has built-in team capabilities

Answer : B

Explanation:

The agent teams documentation states that subagent definitions can be referenced when spawning teammates. This allows teams to reuse well-defined agent configurations, maintaining consistency

between standalone subagent use and team-based collaboration. Option A and B incorrectly deny interoperability. Option D incorrectly limits this to the Explore subagent.

Question 173:

A developer is building a document classification pipeline. They need to process 80,000 documents, each requiring a single API call for classification. The results are needed for a weekly report generated every Friday. Each document needs: classification category, confidence level, and key entities. What is the most cost-effective approach?

- A. Real-time API calls processed sequentially throughout the week
- B. Real-time API calls with prompt caching to reduce per-document costs
- C. Multiple small batch jobs submitted daily throughout the week
- D. Message Batches API submitted Thursday with results available within 24 hours for Friday's report

Answer : D

Explanation:

The Message Batches API supports up to 100K requests per batch with 50% cost savings and a 24-hour processing window. Submitting Thursday for Friday delivery fits perfectly within these constraints. Option A is the most expensive approach. Option B saves on cache hits but still at full per-request pricing. Option D adds operational complexity without additional benefit.

Question 174:

A team configures prompt caching with explicit breakpoints. They notice cache writes (cache_creation_input_tokens) are being charged but cache reads (cache_read_input_tokens) are 0 on subsequent requests with identical content. The TTL hasn't expired. What is the most likely configuration issue?

- A. The content after the breakpoint changes between requests, preventing prefix matching
- B. More than 4 breakpoints are configured, causing all to be ignored
- C. The cacheable content is below the minimum token threshold for the model being used
- D. The API key doesn't have caching permissions enabled

Answer : C

Explanation:

Each model has a minimum cacheable token threshold: 4096 tokens for Opus 4.6 and Opus 4.5, 2048 for Sonnet 4.6, 1024 for Sonnet 4.5. If the content up to the breakpoint is below this minimum, the cache write occurs but subsequent requests can't create a matching cache entry. Option A is possible but the question states identical content. Option B would be noted in the API response. Option D is not a separate permission.

Question 175:

An engineering manager is designing a state management strategy for a complex, multi-session agentic workflow. The workflow involves data collection, analysis, and report generation across multiple context windows. According to the official agentic prompting guide, what is the recommended state management approach?

- A. Use structured JSON for data state, unstructured text for progress notes, and git for version tracking
- B. Store all state in environment variables that persist across sessions
- C. Store everything in a single JSON file that's read at the start of each session
- D. Use a database for all state management

Answer : A

Explanation:

The agentic prompting guide recommends a multi-format state management approach: structured JSON for data (parseable, queryable), unstructured text for progress notes (flexible, human-readable), and git for tracking changes across iterations. This balances structure with flexibility. Option A is limited and fragile. Option C lacks granularity. Option D adds unnecessary infrastructure.

Question 176:

A team's agent handles multi-step workflows involving file creation, testing, and deployment. They want Claude Code to automatically run their test suite after any file edit, without adding test commands to every prompt. What mechanism achieves this automatic post-edit testing?

- A. Configure a PostToolUse hook on the Edit and Write tools that executes the test suite
- B. Add 'always run tests after editing' to the CLAUDE.md instructions
- C. Create a custom slash command that combines edit + test operations
- D. Use the --auto-test flag when starting Claude Code

Answer : A

Explanation:

PostToolUse hooks can be configured to trigger on specific tools (Edit, Write). By hooking into these file-modification tools, the test suite runs automatically after every edit without requiring manual invocation or prompt instructions. Option B is a suggestion, not enforcement. Option C requires manual invocation. Option D is not a real flag.

Question 177:

An architect builds a customer support agent that handles billing inquiries. The agent must NEVER issue refunds over \$500 without human approval. They initially put this rule in the system prompt. During load testing, they discover that with extended conversations, compaction sometimes loses this instruction. What is the correct architectural approach?

- A. Repeat the refund limit instruction every 10 messages to ensure it survives compaction
- B. Use a lower max_tokens to prevent conversations from getting long enough to trigger compaction
- C. Add the rule to custom compaction instructions to ensure it persists through summarization
- D. Implement the \$500 limit as a PreToolUse hook that programmatically blocks the refund tool when the amount exceeds \$500, rather than relying on prompt-based instructions

Answer : D

Explanation:

The exam guide emphasizes using hooks for enforcement and prompts for guidance. Safety-critical rules like refund limits should be implemented as deterministic programmatic checks (PreToolUse hooks) that cannot be lost through compaction or prompt manipulation. Option A is fragile. Option B limits functionality. Option D helps but still relies on the model to follow the summarized rule.

Question 178:

A developer configures an MCP tool that searches a product catalog. The tool accepts optional filters: category, priceRange, brand, and inStock. When a customer asks 'show me laptops under \$1000,' the agent sometimes passes null for brand and inStock. The schema uses nullable types for optional parameters. What improvement would help the model understand when to omit parameters?

- A. Change nullable types to required types with default values
- B. Remove optional parameters and require all fields
- C. Add a 'required' field listing which parameters are mandatory
- D. Add input_examples showing correct usage: e.g., {'category': 'laptop', 'priceRange': {'max': 1000}} without including brand or inStock fields

Answer : D

Explanation:

The tool design best practices recommend using input_examples to show correct parameter usage for tools with optional parameters. Examples demonstrate the expected behavior more effectively than schema definitions alone, showing which fields to include and which to omit. Option A changes semantics. Option C is already standard JSON Schema. Option D removes useful flexibility.

Question 179:

An architect configures Claude Code for a monorepo with three services: frontend (React), backend (Python/FastAPI), and infra (Terraform). Each service has different linting, testing, and coding conventions. How should the CLAUDE.md hierarchy be structured?

- A. A single root CLAUDE.md with sections for each service
- B. Root CLAUDE.md plus .claude/rules/ files for each service

- C. Only subdirectory CLAUDE.md files with no root CLAUDE.md
- D. Root CLAUDE.md for shared conventions, plus frontend/CLAUDE.md, backend/CLAUDE.md, and infra/CLAUDE.md for service-specific rules

Answer : B

Explanation:

For a monorepo, the CLAUDE.md hierarchy works best with a root CLAUDE.md for shared conventions plus .claude/rules/ files with glob patterns to match specific service directories. This provides automatic context-aware rule loading — when working on Python files, only Python rules apply. Option B works but subdirectory CLAUDE.md files require working in that directory. Option A becomes unwieldy. Option C loses shared conventions.

Question 180:

A company uses Claude for customer onboarding. The process involves six steps: identity verification, credit check, account setup, product selection, compliance review, and welcome message. Currently all steps run sequentially in one conversation. The credit check occasionally times out, requiring the entire flow to restart. How should this be redesigned for reliability?

- A. Add retry logic for the credit check step within the existing sequential flow
- B. Run all six steps in parallel to reduce end-to-end time
- C. Move the credit check to a separate service outside the agent flow
- D. Implement each step as an independent stage with checkpointing, so failures at any stage resume from that point rather than restarting

Answer : D

Explanation:

The exam guide recommends building checkpointed workflows where each stage's completion is recorded. If the credit check fails, the system resumes from the credit check stage rather than restarting from identity verification. Option A doesn't address the broader reliability pattern. Option B has dependencies between steps (credit check before account setup). Option D still lacks checkpointing for other steps.

Question 181:

A developer is debugging why their tool consistently receives malformed parameters from Claude. The tool expects dates in ISO 8601 format but receives natural language dates like 'next Tuesday' or 'last month'. The tool's schema specifies type: 'string' for the date parameter. What schema improvement would fix this?

- A. Add a regex pattern to the parameter schema to enforce ISO 8601 format
- B. Add format: 'date' and a clear description: 'Date in YYYY-MM-DD format' to the parameter definition

- C. Change type to 'date' which auto-formats to ISO 8601
- D. Implement server-side validation that rejects non-ISO dates with an error message

Answer : B

Explanation:

Tool parameter descriptions are the primary way Claude understands expected formats. Adding format: 'date' combined with an explicit description stating 'Date in YYYY-MM-DD format' guides Claude to generate properly formatted dates. Option A may not be understood by all models. Option B uses a non-standard JSON Schema type. Option D adds unnecessary round-trips.

Question 182:

A developer uses the Anthropic API with prompt caching. They have a 6,000-token system prompt, a 3,000-token set of examples, and variable user messages. They want to cache the system prompt and examples. How should they structure the cache breakpoints?

- A. Single breakpoint at the end of the system prompt only
- B. Breakpoint after the system prompt AND after the examples, creating two cached segments
- C. No explicit breakpoints needed — automatic caching handles it with Sonnet 4.5+
- D. Breakpoint only at the end of the examples, caching everything as one segment

Answer : C

Explanation:

Automatic caching is available for Sonnet 4.5+, Opus 4.5+, and Claude 4 models. For these models, no explicit breakpoints are needed — the system automatically identifies and caches optimal prefix segments. Manual breakpoints (Options A-C) are only needed for older models or when you need fine-grained control. Option D is the correct approach for current models.

Question 183:

An architect needs to handle a scenario where Claude generates a tool_use block, but the tool execution is interrupted (the server crashes mid-execution). When the system recovers, the conversation history has the tool_use block but no corresponding tool_result. What must happen before continuing the conversation?

- A. Provide a tool_result block for the pending tool_use — either by retrying the tool execution or returning an error result indicating the failure
- B. Send a new message without the tool_use block, pretending it never happened
- C. Retry the entire conversation from the beginning
- D. Send an empty assistant turn to reset the conversation state

Answer : A

Explanation:

The API requires that every tool_use block has a corresponding tool_result in the conversation. If the tool execution was interrupted, you must still provide a tool_result — either by retrying the tool or sending an error result like {type: 'tool_result', tool_use_id: '...', is_error: true, content: 'Tool execution was interrupted'}. Option A loses all progress. Option B violates the API contract. Option D is not supported.

Question 184:

A developer wants Claude Code to enforce specific import ordering in their TypeScript project. They want: (1) node builtins first, (2) external packages, (3) internal imports, (4) relative imports. Which approach reliably enforces this at the tool level?

- A. PreToolUse hook that inspects the edit content before applying
- B. A PostToolUse hook on Edit/Write that runs an import-sorting linter and rejects non-compliant edits
- C. A custom slash command that sorts imports after each edit
- D. CLAUDE.md instructions describing the import order convention

Answer : A

Explanation:

For format enforcement, .claude/rules/ files with import ordering rules combined with CLAUDE.md instructions is guidance, but a PreToolUse hook on Edit that inspects the proposed content provides deterministic enforcement before the edit is applied. This catches violations before they enter the codebase. PostToolUse (Option B) catches violations after the edit is already applied. Option A is only guidance. Option C requires manual invocation.

Question 185:

A team builds an agent that generates SQL queries from natural language. They use tool_choice: 'auto' but notice Claude sometimes answers from training knowledge rather than querying the database. After switching to tool_choice: {'type': 'tool', 'name': 'run_query'}, Claude always generates SQL even when the user asks a follow-up clarifying question. What is the appropriate solution?

- A. Use tool_choice: 'auto' with stronger system prompt instructions to prefer database queries
- B. Use tool_choice: {'type': 'tool', 'name': 'run_query'} for the first turn only, then switch to 'auto' for subsequent turns
- C. Use tool_choice: 'any' so Claude always uses some tool but can choose which one
- D. Add a 'clarify' tool alongside 'run_query' and keep forced tool choice

Answer : B

Explanation:

Forcing `tool_choice` for the first turn ensures the initial response queries the database rather than answering from training data. Switching to 'auto' for follow-up turns allows Claude to handle clarifying questions naturally without being forced to generate SQL. Option A is unreliable. Option C forces tool use for all turns including clarifications. Option D adds unnecessary complexity.

Question 186:

An engineer is building an AI-powered IDE extension using Claude. The extension needs to display Claude's reasoning process to the user while also getting a structured code suggestion. Can extended thinking and structured outputs be combined?

- A. Yes — but only with `json_object` mode, not `json_schema` mode
- B. No — extended thinking and structured outputs are mutually exclusive features
- C. Yes — enable thinking and set the response schema; thinking traces show reasoning while the final output follows the schema
- D. Yes — but only on Claude Opus 4.6, not other models

Answer : C

Explanation:

Extended thinking and structured outputs can be combined. Claude uses the thinking traces for complex reasoning (visible as thinking blocks), then produces a final response conforming to the specified JSON schema. This is ideal for applications that need both transparency (reasoning) and reliability (structured output). Option B is incorrect. Options C and D impose unnecessary restrictions.

Question 187:

An architect needs their agent to process a 500-page technical manual and extract specific details. They notice that relevant details from the middle of the document are sometimes missed while information from the beginning and end is reliably extracted. What document processing strategy addresses this?

- A. Increase `max_tokens` to ensure the full document is processed
- B. Use extended thinking to improve attention across the full document
- C. Place important queries at the beginning of the prompt, with the document following
- D. Structure the document with data/context at the top and questions at the bottom, following the recommended long-context layout

Answer : D

Explanation:

The long context prompting documentation recommends placing the long document at the top of the prompt and the questions/queries at the bottom. This layout improves extraction accuracy, especially for information in the middle of documents. The 'data first, queries last' pattern ensures Claude

processes the full document before encountering the extraction requirements. Option C reverses the recommended order. Options A and B don't address the attention pattern.

Question 188:

A security team is auditing an agent system and discovers that one of the MCP tools makes HTTP requests. They want to restrict which domains the tool can contact. Where should this restriction be enforced?

- A. In the system prompt by listing allowed domains
- B. In the MCP server implementation with an allowlist, plus a PreToolUse hook that validates the URL parameter against the allowed domains before execution
- C. In the tool's description by documenting the allowed domains
- D. In the API configuration using an `mcp_domain_allowlist` parameter

Answer : B

Explanation:

Security restrictions must be enforced programmatically, not through prompts. The MCP server should implement a domain allowlist that rejects requests to unauthorized domains. A PreToolUse hook provides a defense-in-depth layer by validating the URL parameter before the tool even executes. Options A and B are prompt-level guidance that can be bypassed. Option D is not a real API parameter.

Question 189:

A developer uses Claude Code to refactor a large codebase. After several hours and hundreds of tool calls, they notice Claude repeating earlier actions and making previously fixed mistakes. Claude's behavior suggests it has lost context from earlier in the session. What is happening?

- A. The session has exceeded the context window limit, triggering automatic compaction that lost some earlier context
- B. Claude's model weights have degraded due to prolonged use
- C. Claude Code has a built-in session timeout that resets context
- D. The terminal history buffer has overflowed, causing lost command output

Answer : A

Explanation:

When the conversation exceeds the context window limit, automatic compaction summarizes earlier turns to free space. This summarization can lose specific details from early in the session, causing Claude to repeat actions or lose track of earlier fixes. The solution is to use scratchpad files (like `PROGRESS.md`) to persist important state outside the context window. Option A is not how LLMs work. Options C and D describe different issues.

Question 190:

An architect wants to implement a 'skills marketplace' where teams share reusable agent behaviors across projects. According to the Claude Code documentation, what is the architectural foundation for this?

- A. Shared CLAUDE.md files stored in a central repository
- B. SKILL.md files that package prompt instructions, tool access, and variable interfaces into shareable units
- C. MCP servers that expose skill endpoints for each reusable behavior
- D. Custom slash commands that encapsulate reusable workflows

Answer : B

Explanation:

SKILL.md files are the architectural unit for shareable agent behaviors. They package prompt instructions, define which tools the skill can access, expose variable interfaces (\$ARGUMENTS), and specify context injection — making them self-contained units that can be shared across projects. Option A lacks the tool and variable interface. Option B is for external service integration. Option D doesn't encapsulate the full behavior definition.

Question 191:

A developer is trying to debug why their hook script fails silently in Claude Code. The hook is supposed to block dangerous file deletions, but it never triggers. The hook configuration uses 'if' filter matching the Delete tool. After investigation, they discover the tool name in their hook config doesn't match Claude Code's actual tool name. What tool name does Claude Code use for file deletion?

- A. The Delete tool — Claude Code has a dedicated tool for removing files from the workspace
- B. The RemoveFile tool — a specialized tool that handles file deletion with confirmation prompts
- C. The FileSystem.delete method — part of Claude Code's filesystem API for file operations
- D. The Bash tool with an rm command — file deletion goes through the Bash tool, not a dedicated delete tool

Answer : D

Explanation:

Claude Code routes file deletion through the Bash tool using system commands like rm. There is no dedicated Delete tool in Claude Code's tool set. To hook into file deletions, you need a PreToolUse hook on the Bash tool that inspects the command for rm/delete patterns. Options A, B, and C reference non-existent tool names in Claude Code.

Question 192:

A DevOps engineer is designing an agent that processes medical records. The system must comply with HIPAA regulations. During testing, they notice that when Claude encounters an error processing a record, the error message sometimes includes patient identifiers from the failed record. What design principle should they apply?

- A. Disable error messages entirely to prevent PII leakage
- B. Route all errors to a human reviewer who can sanitize them manually
- C. Implement structured error responses with `errorCategory`, `isRetryable`, and sanitized messages that never include raw input data
- D. Encrypt error messages with a key only administrators can access

Answer : C

Explanation:

The tool design best practices recommend structured error responses with categorized error types (`errorCategory`), retryability information (`isRetryable`), and sanitized messages. Error messages should never include raw input data that might contain PII. Instead, reference the record by a non-sensitive identifier. Option A loses diagnostic value. Option C doesn't scale. Option D adds complexity without addressing the root cause.

Question 193:

A developer needs to run Claude in a CI/CD pipeline where the output format must be strictly JSON for downstream tooling. They also need to prevent Claude from reading any local configuration files. Which CLI flags accomplish both requirements?

- A. `--json` and `--no-config`
- B. `--output-format json` and `--skip-config`
- C. `--format=json` and `--headless`
- D. `--json-schema '{...}'` and `--bare` (which disables CLAUDE.md loading)

Answer : D

Explanation:

The `--json-schema` flag forces Claude to output well-formed JSON matching the specified schema. The `--bare` flag prevents Claude Code from loading CLAUDE.md files and other local configuration, ensuring reproducible CI/CD behavior. Option A uses incorrect flag names. Option B uses non-existent flags. Option D has incorrect flag syntax.

Question 194:

A product team is building a system where Claude analyzes security vulnerabilities. They want Claude to think more deeply about critical vulnerabilities. The current system uses effort='low' universally for cost control. What is the recommended approach for varying effort by task criticality?

- A. Use effort='high' for all requests and accept the higher costs
- B. Implement a routing layer that sets effort based on severity: effort='low' for informational, effort='medium' for medium severity, effort='high' for critical vulnerabilities
- C. Use extended thinking for critical vulnerabilities instead of changing effort
- D. Train a classifier model to determine the appropriate effort level

Answer : B

Explanation:

The effort parameter documentation recommends adaptive effort allocation based on task complexity or criticality. A routing layer that maps vulnerability severity to effort levels optimizes the cost/quality tradeoff — low-severity issues get fast, cheap processing while critical vulnerabilities get thorough analysis. Option A wastes budget. Option C is complementary but doesn't address cost optimization. Option D adds unnecessary complexity.

Question 195:

A team implements an agent with three subagents: researcher, analyzer, and reporter. The researcher subagent frequently gets 'stuck' in recursive web searches, making dozens of tool calls without converging on useful results. What architectural pattern prevents this?

- A. Set an iteration limit for the researcher subagent and require it to report findings at the limit even if incomplete
- B. Increase the researcher's context window to give it more room for exploration
- C. Remove the researcher subagent and have the main agent do research directly
- D. Add a PostToolUse hook that counts tool calls and terminates after 10

Answer : A

Explanation:

Setting iteration limits with mandatory reporting prevents unbounded exploration. The researcher must converge and return findings within the allocated iterations, even if partial. This prevents resource waste while ensuring the pipeline progresses. The exam guide mentions iteration limits as a key reliability pattern. Option A doesn't address convergence. Option C centralizes the problem. Option D is too rigid (10 may not be the right number for all tasks).

Question 196:

A development team is evaluating prompt caching costs. Their application sends 10,000 messages per hour with a 5,000-token system prompt. The system prompt is identical across all requests. With automatic caching, approximately how much do they save on the system prompt portion compared to uncached requests?

- A. 90% savings — cache reads cost only 10% of base input token price
- B. 50% savings — cached reads are half the price of uncached reads
- C. 10% savings — caching provides a marginal reduction
- D. 100% savings — cached tokens are free

Answer : A

Explanation:

The documentation states that cache read tokens cost 10% of the base input token price, providing a 90% cost saving on cached portions. Write tokens cost 25% more than base price, but with 10K requests/hour, the write cost is amortized across many reads. Option A and B understate savings. Option D overstates — there's still a 10% cost.

Question 197:

A developer is implementing the Agent SDK and needs to determine how many iterations their agent performed for billing purposes. A 'turn' in the Agent SDK makes multiple Claude API calls with tool use. What metric tracks this?

- A. The number of API calls multiplied by input tokens
- B. The number of tool_use blocks generated across all responses
- C. The 'usage.iterations' field in the usage section that counts the number of agent turns within an API call
- D. The total output tokens divided by the average tokens per turn

Answer : C

Explanation:

The Agent SDK documentation specifies that usage.iterations counts the number of agent turns within an API call. This is the metric used for billing purposes — each iteration represents one model invocation within the agentic loop. Option A doesn't directly map to agent turns. Option C counts tool uses, not iterations. Option D is an approximation.

Question 198:

A developer is building a structured output pipeline that extracts information from scientific papers. They need Claude to indicate when it's uncertain about an extraction. Their schema has required fields for title, authors, and abstract. How should they handle fields where Claude is uncertain?

- A. Make all fields optional so Claude can omit uncertain ones

- B. Add a confidence score field alongside each extraction field
- C. Use a nullable type with a structured pattern: {value: string | null, confidence: 'high' | 'medium' | 'low', reasoning: string}
- D. Let Claude add uncertainty notes in the explanation field

Answer : C

Explanation:

The structured outputs documentation recommends using nullable types with explicit confidence indicators for uncertain extractions. This pattern preserves the schema structure while allowing Claude to express uncertainty clearly. Each field includes the extracted value (or null), a confidence level, and reasoning for the assessment. Option A loses required field guarantees. Option B doesn't indicate what the value would be when uncertain. Option D lacks structure.

Question 199:

A team is building a testing framework for their Claude-powered agent. They want to test that the agent correctly handles a specific MCP tool timeout. In production, this timeout happens rarely (once per 10,000 requests). How can they reliably test this scenario?

- A. Send 10,000 test requests and wait for the timeout to occur naturally
- B. Test only the happy path and add monitoring for the timeout scenario in production
- C. Set the MCP server timeout to 1ms to force frequent timeouts during testing
- D. Mock the MCP tool to return a timeout error and verify the agent's retry/fallback behavior

Answer : D

Explanation:

Mocking the MCP tool to return specific error conditions (timeout, rate limit, server error) is the standard testing approach for rare failure scenarios. This allows deterministic testing of the agent's retry logic, fallback behavior, and error messaging. Option A is wasteful and unreliable. Option C may cause cascading failures. Option D doesn't verify the error handling works.

Question 200:

Your team is configuring compaction for an agent that manages a complex state machine (e.g., an order processing workflow with 12 states). Standard compaction loses the current state tracking. What configuration preserves the state machine context through compaction?

- A. Increase the context window size to avoid triggering compaction
- B. Store the state machine's current state in a scratchpad file that persists outside the context window, referenced after compaction
- C. Use custom compaction instructions that explicitly tell the summarizer to preserve the state machine's current state, transitions, and pending actions
- D. Disable compaction and manually manage context by truncating old messages

Answer : C

Explanation:

Custom compaction instructions allow developers to specify what information the summarizer should preserve. For state machines, instructions like 'always preserve: current state, valid transitions, pending actions, and recent state change history' ensure the stateful context survives compaction. Option A may not be feasible. Option B works but adds file I/O. Option D is fragile and loses old context entirely.

Question 201:

An architect builds an agent for legal document review. The agent needs to identify clauses that conflict with a set of regulatory requirements. The regulatory requirements document is 80,000 tokens. Each legal document is approximately 20,000 tokens. The total context window is 200,000 tokens. They want to process batches of documents efficiently. How many documents can they process per batch with room for Claude's output and system prompt?

- A. One document at a time to maximize quality per document
- B. Three documents per batch: $80K + 3 \times 20K + \text{system overhead} + \text{output buffer} = \sim 160K$, leaving room for analysis
- C. Five documents per batch: $80K \text{ (regulations)} + 5 \times 20K \text{ (documents)} + \text{system overhead} = \sim 190K$ tokens
- D. Ten documents: load only relevant regulation sections per document to fit within context

Answer : C

Explanation:

With 200K context, 80K for regulations, and 20K per document, five documents use $\sim 180K$ tokens. However, the architect needs to reserve tokens for the system prompt and Claude's output. With typical overhead, five documents push to $\sim 190K$ which is too tight. Option B is actually safer. However, Answer A is calculated as feasible at $\sim 190K$ if output is kept concise. The key insight is planning context budget allocation for batch processing.

Question 202:

A team builds an MCP server for their internal CRM system. The server exposes tools for searching contacts, creating opportunities, and updating deal stages. During testing, they find Claude often calls `search_contacts` before every update, even when the contact ID is already known from a previous tool result. What design change reduces these unnecessary calls?

- A. Add a cache layer in the MCP server that returns the last search result automatically
- B. Return the contact ID prominently in all tool results, and update the tool descriptions to indicate when `contact_id` is already available and `search_contacts` is unnecessary
- C. Remove the `search_contacts` tool and require all callers to provide contact IDs

D. Rate-limit the search_contacts tool to once per conversation

Answer : B

Explanation:

When tool results prominently surface the contact_id and descriptions clarify that search is unnecessary when the ID is already known, Claude learns to reuse IDs from previous results. This is a key MCP design pattern: structured responses that carry forward identifiers eliminate redundant lookups. Option A adds complexity. Option C removes a needed function. Option D is too restrictive.

Question 203:

A developer wants to create a custom slash command in Claude Code that generates comprehensive test suites for a given file. The command should accept a file path and a coverage target. What is the correct way to define this?

- A. Add a shell script to .claude/commands/ with the command name as the filename
- B. Register the command in CLAUDE.md under a commands: section
- C. Create a .claude/commands/generate-tests.md file with a prompt template using \$ARGUMENTS for the file path and coverage target
- D. Create a TypeScript function in .claude/plugins/ that defines the command

Answer : C

Explanation:

Custom slash commands in Claude Code are defined as .md files in .claude/commands/ (project-level) or ~/.claude/commands/ (user-level). The filename becomes the command name, and the file content is a prompt template that uses \$ARGUMENTS to receive parameters. Option A uses the wrong file type. Option C is not how commands are registered. Option D references a non-existent plugin system.

Question 204:

A solutions engineer is designing a multi-model agent system. The orchestrator needs to decide whether to route tasks to Claude Opus 4.6 (expensive, thorough) or Claude Haiku 4.5 (cheap, fast). For a customer complaint handling system, which routing strategy is recommended?

- A. Always use Opus for customer-facing interactions to ensure quality
- B. Randomly distribute requests between models to balance load
- C. Use the cheaper model for everything and only escalate to Opus when Haiku returns low confidence
- D. Use Haiku for classification and routing, Opus for generating the final customer response, and Haiku for logging and follow-up actions

Answer : D

Explanation:

The recommended multi-model pattern routes tasks based on their quality requirements: cheap models handle classification, routing, and administrative tasks, while expensive models handle the high-stakes generation (customer-facing responses). This optimizes cost while maintaining quality where it matters. Option A wastes budget on classification. Option C may miss quality issues. Option D is arbitrary.

Question 205:

A developer builds a structured output pipeline where Claude extracts product specifications from manufacturer datasheets. The extraction schema uses anyOf to allow a measurement to be either {value: number, unit: string} or {range: {min: number, max: number}, unit: string}. After deployment, they notice the schema validation sometimes fails. What is the likely issue?

- A. The union types in anyOf are exceeding the maximum of 16 types per union allowed in strict mode
- B. anyOf is not supported in Claude's structured output schema
- C. Each member of the anyOf union must be a distinct type with no overlapping required fields, and both types share the 'unit' field causing ambiguity
- D. anyOf requires a discriminator field to help the model distinguish between union members

Answer : D

Explanation:

The structured outputs documentation specifies that strict mode has a limit of 16 types per union within anyOf. While two types wouldn't exceed this limit, the issue is that anyOf/oneOf schemas need careful structuring. Actually, the most common issue is that anyOf is supported but both schemas include 'unit' as a required field, which can cause validation ambiguity when strict mode validates. Option D describes a best practice that would fix the issue.

Question 206:

A technical lead is building a financial analysis agent that processes earnings reports. The agent has access to tools for fetching market data, company filings, and analyst ratings. A user asks: 'Compare Apple and Microsoft's last quarter performance.' The agent immediately starts fetching Apple data. What orchestration improvement ensures more balanced analysis?

- A. Ask the user which company they want analyzed first
- B. Queue the tasks sequentially: complete Apple analysis, then Microsoft, then compare
- C. Use parallel subagents — one for Apple and one for Microsoft — that fetch and analyze simultaneously, then combine results
- D. Fetch all data for both companies in a single tool call

Answer : C

Explanation:

Parallel subagents are ideal for symmetric analysis tasks. Each subagent independently fetches and analyzes one company's data, preventing order bias (where the second company's analysis is influenced by the first). The orchestrator then combines the results for comparison. Option B introduces order bias. Option C is unnecessary delay. Option D may not be supported by the tools.

Question 207:

A developer is building a compliance monitoring agent using the Claude API. The agent checks whether employee communications comply with company policy. When a violation is detected, the agent must record the violation type, severity, the exact policy section violated, and a remediation. They use `json_schema` strict mode. Their schema allows 'severity' to be any string. What schema design improves reliability?

- A. Use a regex pattern on the severity string field
- B. Use an enum for severity: ['critical', 'high', 'medium', 'low'] to constrain the output to valid values
- C. Add validation in the application layer to normalize severity values
- D. Use a description on the field explaining the expected values

Answer : B

Explanation:

Using enum in the JSON schema constrains Claude's output to exactly the valid severity values. In strict mode, this guarantees the output will be one of the specified values, eliminating variations like 'High', 'HIGH', 'critical-level', etc. Option B may not be enforced by all implementations. Option C requires additional code. Option D is guidance, not enforcement.

Question 208:

A team's agentic system uses the Agent SDK with multiple tools. One tool, `database_query`, occasionally returns very large result sets (100K+ tokens). When this happens, the agent's subsequent reasoning quality drops because the large result dominates the context. What pattern addresses this?

- A. Implement pagination in the `database_query` tool, returning only the first page with a total count and a `next_page` token
- B. Increase `max_tokens` to give Claude more room for reasoning after the large result
- C. Compress the result using `gzip` before returning it in the `tool_result`
- D. Filter the result set in the application layer before the agent sees it

Answer : A

Explanation:

Pagination is the recommended pattern for tools that can return large result sets. The tool returns a manageable first page with metadata (total count, `next_page` token), allowing the agent to decide whether to fetch more or work with the current results. This preserves context for reasoning. Option A

doesn't reduce the context consumed. Option C isn't supported in tool results. Option D may remove data the agent needs.

Question 209:

An AI engineer is setting up Claude Code for a team of 15 developers. Each developer has different preferences for code style feedback. Some want verbose explanations; others want minimal output. How should this be configured without creating per-developer CLAUDE.md files in the repo?

- A. Each developer passes --style flags when starting Claude Code
- B. Create a CLAUDE.md branch per developer in the git repository
- C. Use environment variables to control Claude's verbosity per developer
- D. Each developer configures ~/.claude/CLAUDE.md with their personal preferences — these take precedence for user-specific customizations

Answer : D

Explanation:

The CLAUDE.md hierarchy includes user-level configuration at ~/.claude/CLAUDE.md. Personal preferences like verbosity, explanation style, and formatting go here, while the project-level CLAUDE.md in the repository contains shared team conventions. This separates individual preferences from team standards. Option B pollutes the repository. Options C and D use non-existent mechanisms.

Question 210:

A developer is testing their Claude integration and notices that responses with extended thinking show a 'redacted_thinking' block instead of actual thinking content. They are using the API directly with a valid API key. What causes this?

- A. The model determined the thinking content contained potentially harmful reasoning and automatically redacted it
- B. Extended thinking is in beta and requires a special feature flag to enable
- C. The API key doesn't have permission to view thinking content
- D. The developer is using a model version that doesn't support transparent thinking

Answer : A

Explanation:

Redacted thinking blocks appear when Claude's internal safety classifiers determine the thinking content may contain sensitive or potentially harmful reasoning. This is an automatic safety feature — the thinking still occurs and influences the output, but the raw thinking content is hidden. It's not a permission or configuration issue. Options A, C, and D describe incorrect causes.

Question 211:

A platform engineer is building an agent that needs to handle a task involving both code generation and natural language explanation. The user wants the code in a specific file and the explanation in the chat. Using Claude Code, how should this dual-output workflow be structured?

- A. Generate both outputs in a single prompt and let Claude decide how to format them
- B. Create two separate conversations: one for code, one for explanation
- C. Use Write tool more for the code file and include the explanation in Claude's text response — Claude naturally handles multi-output by combining tool use with text
- D. Use a PostToolUse hook to capture the generated code and redirect it to a file

Answer : C

Explanation:

Claude Code naturally supports multi-output workflows by combining tool use (Write/Edit for code files) with text responses (explanations in the conversation). A single turn can include both tool_use blocks (writing code to files) and text content blocks (explaining the code). Option A lacks file output. Option C loses context between conversations. Option D is unnecessarily complex.

Question 212:

A team's agent processes customer support tickets. They implement a quality scoring system where each response is evaluated on accuracy (0-10), helpfulness (0-10), and tone (0-10). They use structured outputs for scoring. However, they notice Claude consistently scores its own outputs 8-9 across all dimensions. What bias is this, and how should they address it?

- A. This is self-review bias — use a separate evaluation agent (different from the generation agent) with a rubric that includes calibration examples showing the full range of scores
- B. This is anchoring bias — use randomized starting scores to prevent it
- C. This is recency bias — evaluate outputs in random order rather than sequentially
- D. This is confirmation bias — remove the scoring labels to prevent Claude from optimizing for high scores

Answer : A

Explanation:

Self-review bias occurs when the same agent evaluates its own output, consistently producing inflated scores. The exam guide recommends using a separate evaluation agent with calibration examples that demonstrate what 2/10, 5/10, and 10/10 responses look like. This establishes a proper scoring baseline. Option A addresses a different bias. Option C is about ordering. Option D removes useful structure.

Question 213:

A developer uses the Messages API and needs to handle a response that has stop_reason: 'tool_use'. They currently process only text content blocks. What must change in their response handling to properly support tool use?

- A. Parse the text content block for a special tool call syntax that Claude embeds in the text
- B. Make a separate API call to a tool execution endpoint with the response ID
- C. Check content blocks for type: 'tool_use', extract the tool name, id, and input, execute the tool, then send a tool_result back in the next request
- D. Set auto_execute_tools: true in the initial request to handle tool calls automatically

Answer : C

Explanation:

When stop_reason is 'tool_use', the response content includes one or more blocks with type: 'tool_use' containing the tool name, a unique id, and the input parameters. The developer must execute the tool, then send a new request with the assistant's response and a user message containing tool_result blocks matching each tool_use id. Option A describes an older pattern. Options C and D don't exist.

Question 214:

A senior developer is designing prompt templates for a multi-tenant SaaS application. Each tenant has different product catalogs, pricing tiers, and support policies. The shared portion of the system prompt is 3,000 tokens, and tenant-specific configuration adds 1,000-5,000 tokens. How should they structure prompts for cost efficiency?

- A. Use prompt caching with the shared portion as the first cache breakpoint (shared across all tenants), followed by the tenant-specific portion as a second breakpoint (shared across requests for the same tenant)
- B. Concatenate shared and tenant prompts into a single system message per request
- C. Store tenant configurations in an MCP tool that Claude calls at the start of each conversation
- D. Use different API keys per tenant to isolate their configurations

Answer : A

Explanation:

Prompt caching with hierarchical breakpoints maximizes reuse. The shared 3,000-token prefix is cached across ALL tenants (massive savings at scale). The tenant-specific portion after the second breakpoint is cached per-tenant. This creates two layers of caching: cross-tenant and per-tenant. Option A misses caching entirely. Option C adds latency for every conversation. Option D doesn't help with cost.

Question 215:

A developer implements an agentic loop where Claude makes tool calls. The developer's implementation processes tool_use blocks sequentially, executing each tool one at a time. They notice the loop is slow when Claude requests multiple tools in a single response. What optimization should they implement?

- A. Execute independent tool calls in parallel — when a response contains multiple tool_use blocks, run them concurrently and return all tool_results together
- B. Limit Claude to one tool call per response using tool_choice to prevent multiple simultaneous requests
- C. Queue tool calls and batch them for execution on a fixed schedule
- D. Increase the API timeout to accommodate the sequential execution time

Answer : A

Explanation:

When Claude returns multiple tool_use blocks in a single response, they can be executed in parallel if they are independent (which they typically are). All tool_results are then included in the next user message. This significantly reduces the end-to-end latency of the agentic loop. Option A artificially limits Claude's efficiency. Option C adds unnecessary latency. Option D doesn't address the root cause.

Question 216:

A team is building a document generation system where Claude creates reports in a specific format. They want to ensure every report includes exactly 5 sections: Executive Summary, Methodology, Findings, Recommendations, and Appendix. Which approach guarantees this structure?

- A. JSON schema with strict mode: {sections: [{title: enum['Executive Summary', 'Methodology', 'Findings', 'Recommendations', 'Appendix'], content: string}]} with minItems: 5, maxItems: 5
- B. Few-shot examples showing reports with all 5 sections
- C. System prompt instructions listing the 5 required sections
- D. Post-processing validation that checks for the 5 sections and retries if any are missing

Answer : A

Explanation:

Strict mode with a JSON schema using enum for section titles and array length constraints (minItems/maxItems) guarantees the output will have exactly the 5 required sections. This is deterministic enforcement, unlike prompt-based guidance (Options A, B) which may occasionally miss sections. Option D catches errors but wastes API calls on retries.

Question 217:

A developer adds a new MCP server to their Claude Code configuration. The server provides tools for interacting with their company's internal knowledge base. After adding the configuration to .claude/settings.json, the tools are not appearing. What is the most likely issue?

- A. The MCP server needs to be registered in CLAUDE.md to be discoverable
- B. MCP server tools need explicit permission via the allowedTools pattern in the settings — by default, new MCP tools require approval

- C. The MCP server configuration should be in `.claude/settings.local.json` for local development servers
- D. Claude Code needs to be restarted to pick up new MCP configurations

Answer : B

Explanation:

Claude Code uses an allowlist approach for MCP tools. New MCP server tools need explicit permission through the `allowedTools` configuration pattern in settings. By default, new tools require user approval before they can be used. This prevents untrusted tools from being executed automatically. Option A confuses `CLAUDE.md` with settings. Option B is about file sensitivity, not tool visibility. Option D may help but isn't the root cause.

Question 218:

Your organization is implementing a multi-agent system where a supervisor agent coordinates worker agents. The supervisor needs to make decisions based on aggregated results from multiple workers. However, the workers sometimes return conflicting information. According to the exam guide, how should the supervisor handle conflicting worker outputs?

- A. Always trust the worker that responds first, as it likely had the most relevant information
- B. Average the conflicting values from workers to reach a consensus
- C. Discard all conflicting results and re-run the entire workflow
- D. Implement structured conflict handling: compare results, request clarification from specific workers, and apply domain-specific resolution rules before making decisions

Answer : D

Explanation:

The exam guide emphasizes structured conflict resolution in multi-agent systems. The supervisor should compare conflicting results, potentially request additional context from specific workers, and apply domain-specific resolution rules (e.g., prefer the most recent data source, or the most authoritative source). Option A is arbitrary. Option B works only for numerical data. Option D wastes resources.

Question 219:

A developer is using Claude's structured output with a schema that includes an array of items. Sometimes Claude returns 0 items (empty array) when the input genuinely has no matching data. However, the downstream system interprets an empty array as an error. How should the schema be designed to distinguish between 'no data found' and 'extraction error'?

- A. Use `minItems: 1` to force at least one result, with a placeholder 'no data' item
- B. Add a boolean `found: true/false` field and only populate items when found is true
- C. Return null instead of an empty array for 'no data found'

D. Add a status field alongside the array: {status: enum['success', 'no_data', 'error'], items: [...], errorMessage?: string}

Answer : D

Explanation:

A status field with explicit enum values distinguishes between successful extraction with no results, genuine data absence, and extraction errors. This is the recommended pattern for handling empty states in structured outputs — it provides clear semantics for downstream systems. Option A creates fake data. Option C doesn't distinguish error from absence. Option D works but is less informative than the status enum approach.

Question 220:

A team uses Claude Code in headless mode for automated code review in their CI pipeline. They want the review to focus only on security issues, ignoring style and performance. The prompt includes: 'Only report security vulnerabilities.' However, Claude still reports style issues in some runs. What additional headless mode configuration improves consistency?

- A. Filter Claude's output in a post-processing step to remove non-security findings
- B. Use the --security-only flag for headless mode
- C. Set effort='low' to reduce the breadth of analysis
- D. Add --append-system-prompt 'Focus exclusively on security vulnerabilities. Do not report style, performance, or other non-security issues.'

Answer : D

Explanation:

The --append-system-prompt flag adds authoritative instructions that reinforce focus. In headless/CI mode, combining the prompt with --append-system-prompt provides two layers of instruction, improving consistency for narrow-focus tasks. Option B is not a real flag. Option C may miss security issues. Option D is wasteful and doesn't fix the root cause.

Question 221:

A developer is implementing the Message Batches API. Their batch contains 10,000 requests. After submission, they need to check individual request results. How do they retrieve results for a specific request?

- A. Poll the batch status endpoint until complete, then download a results file where each line contains a result indexed by custom_id
- B. Each individual request returns its result immediately while the batch processes
- C. Query a GET endpoint with the batch_id and custom_id to retrieve individual results
- D. Results are sent to a webhook URL configured during batch submission

Answer : A

Explanation:

The Message Batches API processes all requests within 24 hours. Results are retrieved by streaming the results file after the batch status shows 'ended'. Each result is identifiable by the custom_id assigned during submission. Results cannot be retrieved individually before the batch completes. Option B contradicts batch processing. Option C is not the correct endpoint. Option D is not supported.

Question 222:

An architect needs to handle a scenario where Claude's extended thinking budget of 10,000 tokens is insufficient for complex mathematical proofs. Setting budget_tokens higher increases costs. What is the recommended approach for optimizing thinking budget vs. cost?

- A. Set budget_tokens to the maximum (128K) for all requests to ensure quality
- B. Use adaptive thinking mode which automatically allocates thinking budget based on task complexity
- C. Start with a low budget and retry with higher budgets only when the output quality is insufficient
- D. Disable thinking entirely and rely on few-shot examples for complex reasoning

Answer : B

Explanation:

Adaptive thinking mode (thinking: {type: 'enabled', budget_tokens: N} on Claude 4 models) automatically adjusts the thinking depth based on the problem complexity. Simple tasks use minimal thinking tokens, while complex proofs get deeper reasoning — optimizing cost dynamically. Option A wastes budget on simple tasks. Option C adds latency from retries. Option D loses reasoning quality.

Question 223:

A developer is building a Claude-powered chatbot for a travel agency. They want Claude to always recommend their partner airlines and hotels when appropriate, without being obvious about the preference. This instruction is in the system prompt. During an audit, the team realizes this could be considered deceptive. According to Claude's usage policies, what should they do?

- A. Continue with the current approach since system prompts are not visible to users
- B. Disclose to users that the chatbot may prioritize partner services, and frame recommendations transparently
- C. Remove the preference entirely and let Claude recommend based on merit
- D. Move the preference instruction from the system prompt to tool descriptions so it's less visible

Answer : B

Explanation:

Claude's usage policies require transparency about AI behavior that could affect user decisions. Hidden commercial preferences are deceptive. The appropriate approach is to disclose that the chatbot may prioritize partner services, allowing users to make informed decisions. Option A ignores ethical concerns. Option B may be unnecessarily restrictive. Option D makes deception harder to audit.

Question 224:

A startup is troubleshooting an agent that uses `tool_choice: {'type': 'tool', 'name': 'analyze'}` with extended thinking enabled. The API returns an error. What is the constraint they're hitting?

- A. Extended thinking is not compatible with any `tool_choice` setting
- B. The 'analyze' tool name is a reserved keyword that conflicts with thinking mode
- C. Extended thinking with forced `tool_choice` (specific tool) is not allowed — use `tool_choice 'auto'` or `'any'` instead
- D. Extended thinking requires `tool_choice` to be set to 'none'

Answer : C

Explanation:

The documentation states that extended thinking cannot be used with `tool_choice` set to a specific tool (forced tool choice). Only 'auto' and 'any' are compatible with thinking mode. This is because forced tool choice restricts Claude's ability to reason about whether the tool is appropriate. Option A is too broad (auto/any work). Option C is not about reserved names. Option D would disable tools entirely.

Question 225:

A developer is using Claude Code and wants to understand what happened during the last command execution. They suspect the model made an unnecessary file read before editing. Which diagnostic feature helps review the last interaction?

- A. Check the terminal history for command output logs
- B. Review the `.claude/logs/` directory for interaction traces
- C. Enable verbose mode with `--debug` flag to see all internal operations
- D. Use `/cost` to review token usage and tool call sequence from the last interaction

Answer : D

Explanation:

The `/cost` command in Claude Code shows token usage and the sequence of tool calls from the interaction, allowing developers to see if unnecessary operations (like redundant file reads) occurred. This helps optimize workflows and reduce costs. Option A only shows terminal output. Option C is not a standard flag. Option D may not contain granular tool-call information.

Question 226:

An engineering manager is designing an error recovery strategy for an agent that processes financial transactions. The agent makes API calls to a payment gateway. When a payment call fails with a timeout, the agent retries and a duplicate charge occurs. What pattern prevents duplicate transactions?

- A. Increase the timeout to prevent timeouts from occurring
- B. Check the account balance before retrying to see if the first charge went through
- C. Use idempotency keys with each payment request so retries produce the same result as the original call
- D. Add a 30-second delay between retries to ensure the first request completes or fails

Answer : C

Explanation:

Idempotency keys ensure that retrying a failed request produces the same result as the original request, preventing duplicate charges. Each unique operation gets a unique key, and the payment gateway uses this key to deduplicate requests. Option A doesn't prevent timeouts from network issues. Option C has a race condition. Option D adds arbitrary delay without guaranteeing safety.

Question 227:

A developer builds an MCP server that provides database query tools. The tool returns query results as JSON objects. For a query returning 500 rows, the full result is 200K tokens — exceeding what's useful in the context. Following the MCP design best practices, what should the tool return?

- A. A summary with row count, column names, first 10 rows as examples, and a pagination token for fetching more rows on demand
- B. The full 200K token result so Claude has complete information
- C. Only the first row as a sample with total row count
- D. A compressed binary representation of the data

Answer : A

Explanation:

MCP tool design best practices recommend returning 'Goldilocks' responses — not too much, not too little. For large result sets, return metadata (row count, columns), a representative sample (first 10 rows), and a mechanism to fetch more (pagination token). This gives Claude enough to reason about the data without overwhelming context. Option A wastes context. Option C is too sparse. Option D isn't processable.

Question 228:

A team wants their Claude Code workflow to automatically generate commit messages that follow their conventional commits format (type(scope): description). Which configuration approach enforces this?

- A. Add a Git hook (prepare-commit-msg) that invokes Claude Code to generate the message
- B. Create a custom slash command /commit that generates a conventional commit message and executes git commit
- C. Add instructions in CLAUDE.md specifying the conventional commits format and examples of correct commit messages
- D. Use a PostToolUse hook on the Bash tool that intercepts git commit commands

Answer : C

Explanation:

CLAUDE.md is the primary configuration for behavioral guidance in Claude Code. Adding conventional commit format rules with examples directly in CLAUDE.md ensures Claude follows the convention whenever it generates commit messages, whether through direct prompts, slash commands, or automated workflows. Options A, C, and D are supplementary but CLAUDE.md is the foundational approach.

Question 229:

An architect implements extended thinking for a complex reasoning task. They set budget_tokens: 50000 but notice Claude only uses 2,000 thinking tokens for most requests. The responses are high quality. They're concerned about wasted budget allocation. Is there an issue?

- A. Yes — reduce budget_tokens to 5,000 to prevent resource waste
- B. No — budget_tokens is a maximum, not an allocation. Claude uses only what it needs and you're only charged for tokens actually generated
- C. Yes — unused budget_tokens are still charged at the thinking token rate
- D. No — but the high budget prevents optimal batching of requests

Answer : B

Explanation:

budget_tokens is a maximum cap, not a pre-allocation. Claude dynamically determines how much thinking is needed per request. If a task only requires 2,000 tokens of thinking, that's all that's generated and billed. The 50,000 cap only comes into play for genuinely complex tasks. There's no charge for unused budget. Option A unnecessarily limits complex tasks. Option C is incorrect about billing.

Question 230:

A team builds a customer service agent that handles refund requests. When a refund is approved, the agent calls the refund_payment tool. They want to add an approval workflow where refunds over \$100

require manager approval. The current architecture uses direct tool calls. What is the recommended implementation?

- A. Add a system prompt instruction: 'For refunds over \$100, ask the customer to contact a manager'
- B. Implement a PreToolUse hook on the refund_payment tool that checks the amount parameter and blocks execution if over \$100, returning a message that manager approval is required
- C. Create a separate manager_approve tool that must be called before refund_payment
- D. Set a \$100 limit in the refund_payment tool's parameter validation schema

Answer : B

Explanation:

PreToolUse hooks provide deterministic enforcement of business rules. The hook inspects the refund amount before the tool executes and blocks amounts over \$100, returning a message that the agent uses to inform the customer about the approval requirement. Option A relies on prompt compliance. Option C adds complexity to the agent's decision-making. Option D doesn't implement the approval workflow.

Question 231:

A developer needs Claude to generate TypeScript interfaces from API documentation. The output must be syntactically valid TypeScript. They use json_schema mode with a schema: {code: string, interfaces: [{name: string, properties: [{name: string, type: string, optional: boolean}]}]}. The code field contains the full TypeScript output while interfaces contain structured metadata. What issue might arise with this dual-representation approach?

- A. The schema is too complex for strict mode
- B. JSON schema cannot contain both string and array fields
- C. TypeScript code in a JSON string will have escaping issues that make it invalid
- D. The code field and interfaces array may be inconsistent — the string code might not match the structured metadata

Answer : D

Explanation:

When representing the same information in two formats (free-text code string and structured metadata), there's a risk of inconsistency — the code might include properties not in the interfaces array or vice versa. The recommended approach is to use only the structured representation and generate the code from it programmatically. Option B is incorrect. Option C is manageable with proper escaping. Option D depends on complexity limits.

Question 232:

An architect wants to implement a 'circuit breaker' pattern for their agent's external tool calls. When an MCP tool fails 3 times consecutively, the circuit breaker should open and prevent further calls for 5 minutes. Where should this be implemented?

- A. In the MCP server implementation with state tracking for failure counts and timestamps
- B. In the system prompt: 'If a tool fails 3 times, stop trying for 5 minutes'
- C. In a PreToolUse hook that tracks failure history and blocks calls when the circuit is open
- D. In the orchestrator code that manages the agentic loop, with tool call interception logic

Answer : A

Explanation:

The circuit breaker pattern is best implemented in the MCP server itself, which can track failure counts and timestamps for each tool. When the circuit opens, the server returns an immediate error response without attempting the underlying operation. This encapsulates the reliability logic within the tool layer. Option A relies on prompt compliance. Option C works but is less encapsulated. Option D mixes concerns.

Question 233:

A developer is working with the Agent SDK and wants to understand the execution model. Their agent definition includes 3 tools and a system prompt. When they call `agent.run()`, what happens internally?

- A. The SDK makes a single API call with all tools and returns the complete result
- B. The SDK sends the tools to a separate tool execution service and polls for results
- C. The SDK makes parallel API calls — one per tool — and combines the results
- D. The SDK enters an agentic loop: it sends the messages to Claude, receives a response, checks if the response contains `tool_use` blocks, executes any tools, appends tool results to the conversation, and continues until Claude responds with `end_turn`

Answer : D

Explanation:

The Agent SDK implements an agentic loop: send messages to Claude → receive response → if `tool_use`, execute tools and append results → loop until `end_turn`. This is the standard agentic execution model where the SDK manages the conversation turn-taking between Claude and the tools. Each loop iteration is counted as one 'iteration' for billing via `usage.iterations`. Options A, C, and D describe incorrect execution models.

Question 234:

An AI engineer is evaluating whether to use the MCP Connector (API-level MCP) or a local MCP server for their deployment. Their application runs on a serverless function (serverless) and needs to connect to an MCP server for tool access. Which deployment model is appropriate?

- A. Local MCP server running as a sidecar container alongside the Lambda function
- B. Embed the MCP server logic directly in the Lambda function code
- C. MCP server running on a dedicated EC2 instance that the Lambda function connects to
- D. MCP Connector via the API — Anthropic manages the MCP connection, eliminating the need for a persistent local process

Answer : D

Explanation:

The MCP Connector (API-level MCP integration) is ideal for serverless deployments because Anthropic manages the MCP server connection. Lambda functions are ephemeral and can't maintain persistent local processes (ruling out Option A). The Connector handles connection management through the API. Option C adds infrastructure. Option D violates the MCP abstraction.

Question 235:

A team is implementing a multi-step workflow where Claude first plans, then executes. They use the plan mode in Claude Code (Shift+Tab) for the planning phase. During planning, Claude suggests a risky database migration that the developer wants to modify before execution. What is the correct workflow?

- A. Review the plan, provide feedback in plan mode iterations until the plan is satisfactory, then switch to execution mode where Claude implements the refined plan
- B. Edit Claude's plan text directly in the chat and re-submit
- C. Accept the plan and use hooks to prevent the risky migration during execution
- D. Cancel the plan and start a new conversation with more specific instructions

Answer : A

Explanation:

Plan mode (Shift+Tab) enables iterative plan refinement. The developer reviews Claude's suggestions, provides feedback to modify risky elements (like the database migration approach), and iterates until the plan is safe. Only then do they switch to execution mode. This separates planning from execution, reducing risk. Option A doesn't guarantee Claude will follow the edits. Option C applies a workaround to a bad plan. Option D loses context.

Question 236:

A DevOps engineer is building an agent that summarizes long research papers. They notice that for papers over 100K tokens, the summary quality drops significantly compared to shorter papers. The summaries miss key findings from the later sections. What prompting technique specifically addresses this?

- A. Ask Claude to generate section-by-section summaries first, then combine them into a final summary
- B. Use max_tokens to limit the response length, forcing more concise summaries
- C. Send only the abstract and conclusion sections for summarization
- D. Use extended thinking to improve attention across the full document

Answer : A

Explanation:

For very long documents, section-by-section summarization (a form of prompt chaining) ensures each section receives focused attention. The individual summaries are then combined into a coherent final summary. This addresses the attention degradation that occurs with very long single-pass processing. Option B limits output, not input processing. Option C loses middle-section content. Option D helps but may not fully address 100K+ token documents.

Question 237:

A developer configures Claude Code's init.sh file for their project. They want the script to install dependencies, set environment variables, and start background services needed for development. Which consideration is critical for init.sh design?

- A. init.sh must be written in Python for cross-platform compatibility
- B. init.sh should be idempotent — running it multiple times produces the same result, with graceful handling of already-running services and installed dependencies
- C. init.sh must complete in under 10 seconds or it will be killed
- D. init.sh runs with elevated (root) permissions automatically

Answer : B

Explanation:

init.sh should be idempotent because it may run multiple times (on each Claude Code session start). Commands should check if dependencies are already installed, services already running, and environment already configured before taking action. This ensures graceful restarts. Option A — it's a shell script. Option C — no such strict limit. Option D — it runs with user permissions.

Question 238:

A product team is designing the token budget for an agentic system. The system has: 8K token system prompt, up to 5 tool definitions (2K tokens each), user messages averaging 500 tokens, and needs Claude to generate up to 4K token responses. Using a 200K context model, approximately how many conversation turns can they support before hitting the context limit?

- A. About 40 turns — the system prompt and tools are only counted once
- B. About 20 turns — each turn consumes roughly 9K tokens (user message + response + overhead)

- C. About 180 turns — the 200K limit applies to individual messages, not the full conversation
- D. The number of turns is unlimited because old messages are automatically truncated

Answer : B

Explanation:

Total fixed cost: 8K (system) + 10K (5 tools × 2K) = 18K tokens. Remaining: ~182K tokens. Each turn: ~500 (user) + ~4K (response) + tool use overhead ≈ ~8-9K tokens. 182K / 9K ≈ ~20 turns. After that, compaction is needed. The system prompt and tools are sent with every request (Option B is wrong about counting). Option C confuses message vs conversation limits. Option D requires explicit compaction configuration.

Question 239:

A team is building an agent that handles appointment scheduling. The agent needs to check availability across three calendars (work, personal, team), find common free slots, and propose options to the user. What tool architecture minimizes round trips?

- A. A single find_available_slots tool that accepts an array of calendar IDs and date range, returning merged availability across all calendars in one call
- B. Three separate tools: check_work_calendar, check_personal_calendar, check_team_calendar — requiring three sequential calls
- C. A calendar_query tool that Claude calls once per calendar, with results cached for subsequent calls
- D. A get_all_calendars tool that returns the entire calendar data for Claude to analyze

Answer : A

Explanation:

A single tool that handles the calendar aggregation server-side minimizes round trips. Instead of three separate calls (Option A) or full data dumps (Option D), the tool accepts calendar IDs and a date range, performs the availability intersection server-side, and returns merged results. Option C still requires multiple calls. This follows the MCP principle of designing tools that match the agent's workflow, not just wrapping individual APIs.

Question 240:

A development team is configuring .claude/rules/ files for a project. They create a file at .claude/rules/python-testing.md with glob pattern *.py. However, the rules aren't applying when Claude edits test files in the tests/ subdirectory. What is wrong with the glob pattern?

- A. The rules file needs to be in the tests/ directory to apply to test files
- B. Rules files can only use file extension patterns, not directory-specific patterns
- C. The glob pattern *.py only matches .py files in the root directory, not subdirectories — use **/*.py to match all Python files recursively

D. .claude/rules/ files don't support glob patterns — they apply to all files

Answer : C

Explanation:

The glob pattern *.py matches only Python files in the root directory. To match Python files in all subdirectories (including tests/), the pattern must be **/*.py. The ** glob pattern matches any number of directory levels. Option B is incorrect. Option C would work but is redundant. Option D is wrong — rules files use glob patterns in their frontmatter.

Question 241:

A developer implements a system where Claude generates SQL queries for a data analytics platform. They use structured output to get {query: string, explanation: string, estimated_cost: string}. Users report that the estimated_cost field is often wildly inaccurate. What is the fundamental issue?

- A. The estimated_cost field should be a number, not a string
- B. Claude cannot estimate SQL query costs — this should be computed by the database engine's EXPLAIN command
- C. Claude needs access to the database schema to estimate costs accurately
- D. Estimated cost requires real-time metrics that Claude doesn't have access to

Answer : B

Explanation:

Claude cannot accurately estimate SQL query execution costs because this depends on table sizes, index availability, data distribution, and database configuration — all runtime-dependent factors. The correct approach is to have the tool execute EXPLAIN on the generated query and return the database engine's actual cost estimate. Options B-D address symptoms, not the root cause.

Question 242:

An architect wants to implement 'graceful degradation' for their agent when the primary MCP server is unavailable. The agent should fall back to a cached/static version of the most commonly used tools. Where should this failover logic live?

- A. In the system prompt: 'If tools are unavailable, use your training knowledge instead'
- B. In a separate 'failover' MCP server that activates when the primary is down
- C. In the orchestrator code that manages tool execution — it detects MCP server failures and swaps in cached tool implementations
- D. In the MCP server itself, which should implement internal caching

Answer : C

Explanation:

The orchestrator is the appropriate layer for managing failover logic. It can detect MCP server connectivity issues (timeouts, connection refused) and transparently swap in cached/static tool implementations while the primary server recovers. This keeps the agent functional with degraded but usable tool access. Option A loses tool structure. Option C requires infrastructure coordination. Option D only helps with data caching, not server availability.

Question 243:

A developer is implementing streaming for a Claude API response that includes both text and thinking blocks. They need to display thinking content in a separate UI panel from the response text. How do streaming events distinguish between thinking and text content?

- A. Thinking content arrives first, followed by text content — they're time-separated
- B. Thinking content uses a different streaming endpoint from text content
- C. Streaming events include a `content_block_start` event with the block type (thinking or text), allowing the client to route content to the appropriate UI panel
- D. All content arrives in the same stream with inline markdown tags to distinguish thinking

Answer : C

Explanation:

Streaming responses send `content_block_start` events that identify each block's type (thinking, text, tool_use). The client uses these events to route content to the appropriate UI panel — thinking blocks to the reasoning panel, text blocks to the response panel. Option A isn't guaranteed (thinking can be interleaved). Options C and D describe incorrect mechanisms.

Question 244:

A team is building an agent for automated code deployment. The deployment involves 8 steps, and step 5 (database migration) is irreversible. They want Claude Code to pause and get human approval before step 5. What mechanism enables this?

- A. A `PreToolUse` hook that detects migration-related Bash commands and exits with code 1 (ask user for confirmation before proceeding)
- B. `CLAUDE.md` instructions: 'Always ask for confirmation before database migrations'
- C. Plan mode that requires approval for each individual step
- D. A custom slash command that pauses execution with a confirmation prompt

Answer : A

Explanation:

PreToolUse hooks with exit code 1 pause execution and request user confirmation before proceeding. By filtering for migration-related commands (e.g., database migration CLI commands), the hook creates a human-in-the-loop checkpoint at exactly the right moment. Option A is guidance, not enforcement. Option C adds overhead for all steps. Option D requires manual invocation.

Question 245:

A developer notices their agent makes a tool call, receives a result, then immediately makes the same tool call again with identical parameters. This pattern repeats 3-4 times before the agent proceeds. The agent isn't in an error state. What is the most likely cause and fix?

- A. Network latency is causing the agent to miss the first tool result — increase timeout
- B. Claude's temperature is too high, causing non-deterministic behavior
- C. The agent loop implementation has a bug that doesn't properly append tool results to the conversation
- D. The tool result lacks clear success/failure indicators, causing the agent to be uncertain whether the operation completed — add explicit status fields to tool results

Answer : D

Explanation:

When tool results don't include clear success indicators, Claude may be uncertain whether the operation completed and retries to confirm. Adding explicit status fields (success: true, data: {...}) to tool results gives Claude clear confirmation that the operation completed, eliminating unnecessary retries. Option A would show errors. Option C would cause all tools to fail. Option D doesn't explain repeated identical calls.

Question 246:

Your team is designing a prompt template for a customer email generator. They want Claude to maintain the customer's language (English, Spanish, French) automatically without explicit language instructions. What is the most effective approach?

- A. Detect the customer's language first with a classification call, then include a language instruction
- B. Include a few-shot example in each language to prime Claude for multilingual responses
- C. Use a separate translation API to convert all inputs to English and outputs back to the original language
- D. Place the customer's original message prominently in the prompt — Claude naturally mirrors the input language when generating responses

Answer : D

Explanation:

Claude naturally mirrors the language of the input it's responding to. By placing the customer's original message prominently in the prompt, Claude generates the response in the same language without

explicit instructions. This is simpler and more reliable than classification (Option A), few-shot per language (Option B), or external translation (Option D). The exam guide emphasizes leveraging Claude's natural language matching behavior.

Question 247:

A team has their Agent SDK agent configured with `max_turns`: 10. Their complex workflow typically requires 15-20 tool calls but the agent stops after 10 turns. They need the agent to complete the full workflow. What are the two options for handling this?

- A. Increase `max_turns` to 25 and hope it's enough
- B. Split the workflow into exactly 10 steps or fewer
- C. Either increase `max_turns`, or implement multi-turn looping where the orchestrator checks if the task is complete after `max_turns` and re-invokes the agent with the current state
- D. Remove the `max_turns` limit entirely to let the agent run indefinitely

Answer : C

Explanation:

The two approaches are: (1) increase `max_turns` to accommodate the workflow, or (2) implement outer-loop orchestration that checks task completion after each `max_turns` cycle and re-invokes the agent with accumulated state. The second approach provides better control and prevents runaway agents. Option A is fragile. Option C may not be possible. Option D risks unbounded execution.

Question 248:

A solutions engineer is building a RAG (Retrieval Augmented Generation) system that retrieves documents and uses Claude to answer questions. After retrieval, they inject 5 relevant document chunks totaling 15K tokens into the prompt. They want to maximize cache hits across users who ask similar questions. What prompt structure optimizes caching?

- A. System prompt → document chunks → user question (documents change per query, breaking the cache)
- B. System prompt → user question → document chunks
- C. Put all content in a single user message to simplify caching
- D. System prompt (cached) → common document corpus (cached) → query-specific chunks → user question

Answer : D

Explanation:

Cache matching works on prefixes. Structuring as system prompt → common corpus → query-specific content → user question maximizes the cacheable prefix. The system prompt is shared across all requests. A common document corpus (frequently accessed documents) extends the shared prefix.

Query-specific chunks come after. Option A breaks the cache frequently. Option B loses document caching. Option D doesn't leverage prefix caching.

Question 249:

A developer is configuring Claude Code settings and notices there are three levels of settings files: `.claude/settings.json` (project), `.claude/settings.local.json` (local), and `~/.claude/settings.json` (user). They add an MCP server to the project settings. A teammate adds the same server with different configuration to their local settings. Which takes precedence?

- A. Project settings (`.claude/settings.json`) always take precedence
- B. Local settings (`.claude/settings.local.json`) take precedence over project settings for the same configuration
- C. User settings (`~/.claude/settings.json`) have the highest precedence
- D. The settings are merged with conflicts causing an error

Answer : B

Explanation:

Local settings (`.claude/settings.local.json`) override project settings (`.claude/settings.json`) for the same configuration keys. This allows individual developers to customize their local environment without affecting the shared project configuration. User settings provide defaults that both project and local can override. Option A understates local settings. Option C has the precedence reversed. Option D doesn't match the merge behavior.

Question 250:

An architect needs Claude to generate a complex financial model with nested calculations. They want to verify that intermediate calculations are correct before using the final result. Extended thinking would show the reasoning, but they need the intermediate values in a structured format. What approach combines both?

- A. Enable thinking and ask Claude to include intermediate values in the thinking blocks
- B. Use a structured output schema with nested objects that mirror the calculation hierarchy: `{inputs: {...}, intermediate: {step1: {...}, step2: {...}}, final: {...}}` with thinking enabled for reasoning visibility
- C. Make separate API calls for each calculation step
- D. Use tool calls for each intermediate calculation, collecting results along the way

Answer : B

Explanation:

Combining structured output (which captures intermediate values in a verifiable schema) with extended thinking (which provides reasoning transparency) gives both structured data and reasoning visibility. The schema mirrors the calculation hierarchy, making each step verifiable. Option A puts values in

thinking which isn't reliably parseable. Option C loses context between steps. Option D adds unnecessary complexity.

Question 251:

An architect builds an agent that must handle both simple lookups (e.g., 'What is the status of order #1234?') and complex investigations (e.g., 'Why was order #1234 delayed and what is the impact on the customer's account?'). Currently both types use the same processing pipeline with extended thinking, making simple lookups slow. What architecture balances speed and depth?

- A. Disable extended thinking for all requests and rely on tool results for complex analysis
- B. Let users select 'quick' or 'detailed' mode for their queries
- C. Add a timeout that kills thinking after 3 seconds for simple queries
- D. Use a classifier to route simple lookups (effort='low', no thinking) to a fast path and complex investigations (effort='high', thinking enabled) to a deep-analysis path

Answer : D

Explanation:

A routing classifier separates queries by complexity, sending simple lookups through an optimized fast path (low effort, no thinking) and complex investigations through a thorough deep-analysis path (high effort, thinking enabled). This optimizes both latency for simple queries and quality for complex ones. Option A loses depth. Option C may produce inconsistent results. Option D adds friction.

Question 252:

A developer implements an MCP tool for image processing. The tool accepts an image URL and returns analysis results. During testing, they find that Claude sometimes passes a data URL (base64-encoded image) instead of an HTTP URL. The tool only supports HTTP URLs. How should the tool handle this gracefully?

- A. Return an error: 'Invalid URL format' without further guidance
- B. Accept data URLs by decoding the base64 content within the tool implementation
- C. Return a structured error with the constraint: {isError: true, message: 'Only HTTP/HTTPS URLs are supported. Data URLs (base64) cannot be processed. Please provide the image's HTTP URL.', isRetryable: true}
- D. Silently ignore the data URL and return empty results

Answer : C

Explanation:

Structured error responses with clear constraints help Claude self-correct. By returning the specific requirement (HTTP/HTTPS only), the reason (data URLs can't be processed), and marking it as retryable, Claude can adjust and provide a valid URL. Option A is too vague. Option B adds unnecessary complexity if there's a reason only HTTP is supported. Option D hides the failure.

Question 253:

A team uses Claude Code's `/init` command to generate the initial `CLAUDE.md` for their project. After generation, they want to customize it. The `/init` command has already analyzed the repository. What is the recommended approach for customization?

- A. Delete the generated `CLAUDE.md` and write one from scratch
- B. Keep the generated `CLAUDE.md` as a base and iteratively refine it — add project-specific conventions, remove irrelevant sections, and commit the customized version
- C. Generate multiple `CLAUDE.md` files with `/init` and merge the best parts
- D. Never modify the `/init` output — it's optimized for the repository

Answer : B

Explanation:

The `/init` command creates a good starting point based on repository analysis. The recommended workflow is to keep this base and iteratively customize it: add project-specific coding conventions, team preferences, and workflow rules. Remove auto-detected rules that don't apply. Commit the customized version. Option A loses the auto-detected insights. Option C is redundant. Option D misses the opportunity for customization.

Question 254:

A technical lead is implementing a content moderation system using Claude. They need to classify user-generated content as safe, questionable, or unsafe. For legal compliance, they need an audit trail showing why each decision was made. What API features support this requirement?

- A. Enable extended thinking to capture the reasoning, and use structured output for the classification — the thinking blocks provide the audit trail while the structured output provides the consistent decision format
- B. Use the Messages API with logging enabled to capture all request/response pairs
- C. Use a separate API call for classification and another for explanation generation
- D. Store the raw API response JSON as the audit trail

Answer : A

Explanation:

Extended thinking provides transparent reasoning (the 'why' behind the decision) while structured output ensures consistent classification format. Together, they create a complete audit trail: the thinking blocks show deliberation, and the structured output shows the deterministic output. Option A lacks decision reasoning. Option C doubles API costs. Option D contains the decision but not the reasoning.

Question 255:

A developer's agent system has a memory of 50 previous customer interactions stored in a database. For each new interaction, they inject all 50 prior interactions into the context for personalization. This consumes 100K tokens, leaving limited room for the current conversation. What is the better approach?

- A. Summarize all 50 interactions into a 5K token customer profile that's injected instead
- B. Retrieve only the 3-5 most relevant prior interactions (using embedding similarity or recency) and inject those, keeping context lean while maintaining personalization
- C. Increase to a 1M token context model to fit all history
- D. Store all interactions in a tool that Claude can query on demand

Answer : A

Explanation:

Summarizing interaction history into a concise customer profile preserves the key insights (preferences, recurring issues, communication style) while consuming far fewer tokens. This is more efficient than raw injection (full history) and more complete than selecting only a few interactions (which may miss important patterns). Option A is expensive. Option B may miss important context. Option D adds round trips.

Question 256:

An AI engineer is building a multi-agent system for software testing. Agent A generates test cases, Agent B executes them, and Agent C analyzes results. Agent B discovers a test environment misconfiguration that affects all remaining tests. How should Agent B communicate this blocking issue to the orchestrator?

- A. Continue executing tests and note failures in the results for Agent C to analyze
- B. Stop execution and return an error message to the orchestrator
- C. Return a structured status with escalation metadata: {status: 'blocked', reason: 'environment_misconfiguration', details: '...', blockedTests: [...], suggestedAction: 'fix_environment_and_retry'}
- D. Call Agent A to regenerate test cases that work with the misconfigured environment

Answer : C

Explanation:

Structured escalation metadata enables the orchestrator to make informed decisions. The status includes the blocking reason, affected tests, and suggested remediation. This is superior to a generic error (Option B) because it provides actionable information. Option A wastes resources on known-bad tests. Option D works around the problem instead of fixing it.

Question 257:

A developer needs to pass sensitive credentials to an MCP server configured in Claude Code. They want the credentials stored securely and not committed to the repository. Where should the credentials be configured?

- A. In `.claude/settings.json` with the credentials directly in the MCP server configuration
- B. In environment variables referenced by the MCP server configuration using `${VAR_NAME}` expansion syntax
- C. In the `CLAUDE.md` file with instructions for where to find credentials
- D. Hardcoded in the MCP server source code

Answer : B

Explanation:

MCP server configurations support environment variable expansion using `${VAR_NAME}` syntax. Credentials are stored in environment variables (via `.env` files, shell profiles, or CI secrets) and referenced in the configuration. This keeps secrets out of committed files. Option A commits secrets to the repository. Option C exposes secrets in a documentation file. Option D is a security anti-pattern.

Question 258:

An architect wants to implement a 'dry run' mode for their agent where all tool calls are simulated without executing. This is useful for testing agent behavior and estimating costs. Which implementation approach is most effective?

- A. Add 'this is a dry run, don't actually execute anything' to the system prompt
- B. Replace all tool implementations with mock versions that return realistic simulated responses
- C. Use `PreToolUse` hooks that intercept all tool calls, log them, and return simulated results without executing the actual tools
- D. Set `effort='low'` to minimize the impact of tool calls during testing

Answer : C

Explanation:

`PreToolUse` hooks provide a clean interception layer for dry-run mode. They can log each tool call (for analysis and cost estimation), return simulated results (keeping the agent's behavior realistic), and avoid

any side effects from actual execution. This works without modifying tool implementations (Option B) or relying on prompt compliance (Option A). Option D doesn't prevent execution.

Question 259:

A developer is implementing token counting for their application to monitor costs. They need to count tokens before sending requests to avoid exceeding context limits. Which approach does the Anthropic documentation recommend?

- A. Use the token counting API endpoint to get exact counts before sending the full request
- B. Use a third-party tokenizer library to estimate token counts locally
- C. Estimate based on character count: ~4 characters per token for English text
- D. Send the request and check the usage field in the response for actual token counts

Answer : A

Explanation:

Anthropic provides a token counting API endpoint that returns exact token counts without processing the full request. This enables pre-flight checks to ensure requests fit within context limits and budget constraints. Option A may be inaccurate for Claude's specific tokenizer. Option C is a rough estimate. Option D provides post-hoc information, not pre-flight validation.

Question 260:

A team builds a legal document analysis agent. The agent needs to cross-reference clauses across a 200-page contract. Their first approach loads the full document and asks Claude to find all cross-references. Results are inconsistent — some cross-references are found, others are missed. What document processing strategy improves reliability?

- A. Use a smarter model (Opus instead of Sonnet) for better attention across the full document
- B. Split the document into overlapping sections, have Claude identify cross-references in each section, then merge and deduplicate results across sections
- C. Index the document by clause number first, then query specific clause pairs for cross-references
- D. Use extended thinking with a large budget to improve attention throughout the document

Answer : D

Explanation:

Extended thinking with a large budget allows Claude to methodically work through the entire document, tracking cross-references as it goes. For 200-page documents within the context window, this is more effective than splitting (which may miss cross-section references) or simple full-document scanning. Option A helps but isn't sufficient alone. Option B may miss references spanning sections. Option D requires knowing which clauses to check in advance.

Question 261:

A platform engineer is designing an agent that interacts with a third-party API. The API has rate limits of 100 requests per minute. The agent makes rapid tool calls that can exceed this limit. Where should rate limiting be implemented?

- A. In the system prompt: 'Wait at least 600ms between API calls'
- B. In a PreToolUse hook that adds a delay between tool invocations
- C. In the MCP server implementation, using a token bucket algorithm that queues requests exceeding the rate limit
- D. In Claude's API parameters by setting a request_delay field

Answer : C

Explanation:

Rate limiting is best implemented in the MCP server where it's transparent to the agent. A token bucket algorithm queues excess requests until capacity is available, preventing rate limit errors while allowing burst handling. Option A relies on prompt compliance and isn't precise. Option C adds delays to all tool calls, not just rate-limited ones. Option D doesn't exist.

Question 262:

A developer needs to generate unit tests for a complex function. They have the function implementation and existing tests for similar functions. What prompting strategy produces the highest quality test suite?

- A. Send only the function implementation and ask Claude to generate tests
- B. Send the function, its type definitions, and 2-3 examples of test files for similar functions as reference — the examples calibrate Claude's testing style, coverage patterns, and assertion conventions
- C. Ask Claude to list all edge cases first, then generate one test per edge case
- D. Send the function and a testing framework documentation page

Answer : B

Explanation:

Providing the function, types, and example test files (few-shot reference) calibrates Claude to the team's testing conventions: assertion style, mocking patterns, edge case coverage, and naming conventions. This produces tests that match the existing codebase style. Option A lacks calibration. Option C may produce tests inconsistent with team conventions. Option D provides framework docs but not team patterns.

Question 263:

An architect implements context awareness (formerly citations) to track which sources Claude uses in its responses. The marketing team wants to display source links alongside generated content. What does the context awareness feature provide?

- A. URLs to original source documents that Claude used during training
- B. A list of document IDs that were most relevant to the response
- C. Character-level pointers back to specific locations in the input context, showing exactly which parts of the provided documents Claude referenced for each claim
- D. A confidence score indicating how well-supported each claim is by the input documents

Answer : C

Explanation:

Context awareness provides character-level pointers (start and end indices) back to the input context, showing exactly which parts of the provided documents Claude referenced when generating each claim. This enables source attribution UI features like highlighting or linking to specific passage locations. Option A references training data, not input context. Option C lacks granularity. Option D provides confidence, not attribution.

Question 264:

A developer wants Claude Code to output results in a specific format when run in their CI pipeline. They use headless mode with `--json-schema`. The schema specifies `{issues: [{file: string, line: number, severity: string, message: string}]}`. However, the output sometimes includes extra explanation text alongside the JSON. What flag ensures only the JSON schema output is returned?

- A. The `--json-schema` flag already guarantees only schema-compliant JSON output in headless mode
- B. `--quiet` to suppress all logging and only show the result
- C. `--json-only` to suppress non-JSON output
- D. Pipe the output through `jq` to extract only the JSON portion

Answer : A

Explanation:

When using `--json-schema` in headless mode, Claude Code is designed to output only schema-compliant JSON. The combination of headless mode and `--json-schema` provides clean machine-readable output suitable for CI pipelines. Option A is not a real flag. Option B may suppress useful information. Option D is a workaround, not a solution.

Question 265:

A senior developer is building a chatbot for a hospital that must NEVER provide specific medical diagnoses while still being helpful about general health information. A previous attempt using only

system prompt instructions failed when a patient described symptoms in detail and Claude provided a tentative diagnosis. What is the correct implementation?

- A. Add more emphatic language to the system prompt: 'ABSOLUTELY NEVER provide diagnoses'
- B. Use a two-layer approach: system prompt for general guidance plus a separate classifier model that screens every response before it reaches the patient, blocking anything containing diagnostic statements
- C. Implement a PostToolUse classification step that checks every response for diagnostic language and regenerates if detected
- D. Limit the conversation length to prevent detailed symptom discussions

Answer : B

Explanation:

Safety-critical content filtering requires defense-in-depth. A classifier model screening every response provides a deterministic safety layer that the system prompt alone cannot guarantee. The system prompt provides guidance (often effective), while the classifier provides enforcement (always effective). This is the same 'prompts for guidance, hooks for enforcement' philosophy. Option A is still only prompt-level. Option B is less reliable than a dedicated classifier. Option D degrades user experience.

Question 266:

A developer creates an MCP tool that wraps a SOAP-based legacy API. The SOAP API returns XML with deeply nested namespaced elements. What transformation should the MCP tool apply before returning results to Claude?

- A. Return the raw XML — Claude can parse XML natively
- B. Base64-encode the XML and return it as a string
- C. Convert to a flat key-value pair format
- D. Convert to a simplified JSON structure that strips namespaces and flattens unnecessary nesting, **keeping only the business-relevant fields**

Answer : D

Explanation:

MCP tools should return high-signal, Claude-friendly responses. Complex XML with namespaces and deep nesting wastes context tokens and confuses tool result processing. Converting to simplified JSON with only business-relevant fields improves both context efficiency and Claude's ability to use the data. Option A wastes tokens on XML overhead. Option C may lose hierarchical relationships. Option D makes the data unusable.

Question 267:

Your organization is designing a prompt caching strategy. Their application has: (1) a 3K token system prompt, (2) a 10K token knowledge base, (3) a 2K token conversation history, and (4) user messages averaging 200 tokens. They're using Claude Sonnet 4.5 which has a minimum 1024 token cache threshold. Which portions should be cached?

- A. System prompt (3K) + knowledge base (10K) as a single cached prefix, since both are stable and together form a 13K prefix well above the minimum threshold
- B. Only the system prompt (3K) since it's stable across all requests
- C. Cache everything including conversation history
- D. Only the knowledge base since it's the largest component

Answer : A

Explanation:

The system prompt and knowledge base together form a stable 13K token prefix that's shared across all requests and well above the 1024 token minimum for Sonnet 4.5. Conversation history changes per turn (Option C) making it uncacheable across requests. The system prompt alone (Option A) misses the larger savings from the knowledge base. Option D wouldn't include the system prompt prefix.

Question 268:

A team implements a structured output pipeline where Claude generates quiz questions. Each question must have exactly 4 options and one correct answer. They use json_schema strict mode. Occasionally, two options are semantically identical (e.g., 'Use a load balancer' and 'Implement load balancing'). The schema can't prevent this. What additional quality measure is needed?

- A. Add a schema constraint for unique options (uniqueItems in JSON Schema)
- B. Implement a semantic similarity check in the application layer that compares all option pairs and flags near-duplicates for regeneration
- C. Use extended thinking to encourage Claude to verify distinctness before generating
- D. Add a system prompt instruction: 'All options must be semantically distinct'

Answer : B

Explanation:

Schema validation can enforce syntactic uniqueness (Option A) but not semantic uniqueness. A semantic similarity check using embedding comparison catches near-duplicate options like 'Use a load balancer' vs 'Implement load balancing'. This application-layer validation complements the schema enforcement. Option B may help but isn't reliable. Option D is guidance without enforcement.

Question 269:

A startup is designing a customer service agent. They want the agent to detect when a customer is frustrated based on conversation patterns (multiple repeated questions, escalating language, long wait

times). The detection should trigger a different handling workflow. Where should frustration detection logic be implemented?

- A. In the system prompt: 'Detect frustration and adjust your approach'
- B. In a PostToolUse hook that monitors the conversation for frustration indicators
- C. As a separate sentiment analysis tool that the agent calls periodically
- D. In the orchestrator as a classifier that analyzes each customer message and adjusts the agent's system prompt or routing based on sentiment signals

Answer : D

Explanation:

The orchestrator is the appropriate layer for cross-cutting concerns like sentiment analysis. Analyzing each customer message through a lightweight classifier enables dynamic adjustments: updating the system prompt tone, routing to a specialized empathy-focused agent, or triggering human handoff. Option A relies on the agent self-detecting. Option C adds tool call overhead. Option D only monitors tool interactions.

Question 270:

A developer implements compaction for a long-running code review session. After compaction, Claude loses track of files it already reviewed and re-reviews them. What technique preserves the review progress through compaction?

- A. Use custom compaction instructions: 'Always preserve the list of reviewed files and their review status'
- B. Maintain a scratchpad file (e.g., REVIEW_PROGRESS.md) listing completed reviews, with Claude reading it after compaction to restore state
- C. Increase the compaction summary length to capture all reviewed files
- D. Disable compaction and manage context manually

Answer : A

Explanation:

Custom compaction instructions ensure the summarizer explicitly preserves the review state. By instructing compaction to maintain the list of reviewed files and their status, this information persists through summarization without requiring external files. Option A may not be configurable to target specific content. Option B works but adds file I/O overhead. Option D is fragile for long sessions.

Question 271:

A developer wants to use Claude for generating regex patterns from natural language descriptions. They use the Messages API with a simple prompt. The generated regex often has edge cases that fail. What approach improves regex generation quality?

- A. Use a larger model (Opus instead of Sonnet) for better pattern generation
- B. Ask Claude to generate multiple regex variants and pick the best one
- C. Use extended thinking to let Claude reason through edge cases
- D. Provide test strings (both matching and non-matching examples) in the prompt, then use a tool that tests the generated regex against all examples and returns failures for iteration

Answer : D

Explanation:

Providing test strings with expected outcomes and using a test-execution tool creates a feedback loop: Claude generates a regex, the tool tests it against all examples, failures are returned as `tool_results`, and Claude iterates. This is the 'test-driven iteration' pattern from the exam guide. Option A doesn't address the testing gap. Option C helps reasoning but doesn't verify. Option D doesn't guarantee correctness.

Question 272:

An engineering manager is building an enterprise agent platform that serves multiple departments. Each department has its own MCP servers, tools, and access controls. The IT department should access infrastructure tools, HR should access employee tools, and Finance should access accounting tools. How should tool access be scoped?

- A. Give all departments access to all tools and rely on prompt instructions for scope
- B. Create separate Claude API keys per department with different tool permissions
- C. Use per-agent tool configurations that expose only department-appropriate tools, with MCP server permissions enforcing access control at the tool execution layer
- D. Use a single MCP server that checks the caller's department before returning results

Answer : C

Explanation:

Per-agent tool configurations ensure each department's agent only sees relevant tools, reducing confusion and preventing accidental cross-department access. MCP server-level permissions provide a second enforcement layer. This follows least-privilege principles. Option A violates least privilege. Option C isn't how API key scoping works. Option D centralizes but doesn't restrict tool visibility.

Question 273:

A developer is building a multi-step data pipeline with Claude. Step 1 extracts data from PDFs, Step 2 normalizes the data, Step 3 performs analysis, and Step 4 generates a report. Each step's output feeds into the next. After Step 2, the normalized data is 40K tokens. They worry Step 3 will lose accuracy over long context. What prompting pattern optimizes this pipeline?

- A. Run all 4 steps in a single conversation to maintain context

- B. Combine Steps 1-2 and 3-4 into two larger steps to reduce the number of calls
- C. Run all steps in parallel with separate conversations
- D. Use prompt chaining: run each step as a separate API call, passing only the necessary output from the previous step as input to the next

Answer : D

Explanation:

Prompt chaining — running each step as a separate API call with curated inputs — prevents context bloat and maintains focus. Step 3 receives only the 40K token normalized data (not the full Step 1-2 conversation), leaving maximum context for analysis. Option A accumulates all intermediate context. Option C loses the data flow dependency. Option D partially helps but still accumulates more context than necessary.

Question 274:

A DevOps engineer is building an agent that must generate responses in strict compliance with a company's brand voice guide (a 15-page document). They've included the full brand guide in the system prompt. Responses generally match the brand voice, but occasionally Claude generates off-brand content for edge-case topics not covered in the guide. How can they improve consistency?

- A. Reduce the brand guide to key bullet points to avoid Claude getting confused by verbose instructions
- B. Add few-shot examples showing correct brand voice application for various scenarios, including edge cases similar to the problematic ones
- C. Use a PostToolUse classifier that checks every response against the brand guide and regenerates if off-brand
- D. Fine-tune a custom model on brand-compliant responses

Answer : B

Explanation:

Few-shot examples demonstrating brand voice application across scenarios (especially edge cases) help Claude generalize the brand voice to new situations not explicitly covered in the guide. Examples show HOW to apply the brand principles rather than just listing them. Option A may lose important nuances. Option C adds latency and cost. Option D is expensive and overkill for prompt engineering.

Question 275:

A developer notices that their Claude Code session becomes sluggish after extended use. Responses take longer and quality decreases. They suspect context window saturation. What is the recommended maintenance approach for long coding sessions?

- A. Restart Claude Code to get a fresh context window

- B. Delete all CLAUDE.md files temporarily to reduce context consumption
- C. Use /compact to manually trigger compaction, then verify Claude's understanding of the current task by asking it to summarize what it knows
- D. Switch to a model with a larger context window

Answer : C

Explanation:

The /compact command manually triggers compaction to summarize and reduce context. Following up with a state verification (asking Claude to summarize its understanding) confirms what was preserved and identifies any lost context that needs to be re-provided. Option A loses all context. Option C removes important configuration. Option D may not be available.

Question 276:

A product team is designing a fraud detection agent for a bank. The agent analyzes transactions in real-time and flags suspicious activity. False positives block legitimate transactions, causing customer frustration. False negatives allow fraud through. Currently the agent flags 30% of transactions, of which 90% are false positives. What analysis framework should be applied?

- A. Lower the sensitivity threshold to reduce false positives
- B. Use a tiered confidence system: high-confidence fraud → block immediately, medium-confidence → add friction (2FA), low-confidence → allow with monitoring. Each tier has different precision/recall tradeoffs
- C. Remove automated blocking entirely and rely on post-transaction review
- D. Train a separate classifier to pre-filter the agent's results

Answer : B

Explanation:

A tiered confidence system maps different certainty levels to different actions. High-certainty fraud is blocked (high precision required), medium certainty adds friction without blocking (balanced approach), and low certainty allows transactions with monitoring (prioritizing availability). This framework addresses the precision/recall tradeoff at each tier. Option A reduces all sensitivity without nuance. Options C and D address symptoms.

Question 277:

A developer builds an MCP server that provides a search tool. The search tool returns results with URLs. After deployment, they notice Claude sometimes fabricates URLs that look similar to real results but don't exist. What server-side measures prevent this?

- A. Add URL validation in the system prompt instructing Claude to only use URLs from tool results
- B. Use context awareness (citations) to track which URLs come from tool results vs. model generation

- C. Implement client-side URL validation that checks all URLs in Claude's response
- D. Return URLs as structured data in tool results (not embedded in text) and instruct Claude to reference results by ID rather than regenerating URLs

Answer : B

Explanation:

Context awareness tracks which parts of Claude's response reference specific input context (including tool results). This enables the application to verify that URLs in the response actually originated from tool results rather than being generated by the model. Combined with client-side validation, this provides attribution verification. Option A is unreliable. Option B helps but doesn't fully prevent fabrication. Option C validates but doesn't attribute.

Question 278:

A team is evaluating their Claude Code setup and wants to understand the permission model. They add a new tool via MCP that can execute shell commands. When an agent tries to use this tool, it gets a permission error. Where are MCP tool permissions managed?

- A. In `.claude/settings.json` under the 'allowedTools' and 'blockedTools' arrays, with glob pattern matching for MCP tool names
- B. In the `CLAUDE.md` file under an 'allowed_tools' section
- C. In the MCP server configuration file
- D. Permissions are managed per-conversation and reset each time

Answer : A

Explanation:

Claude Code manages MCP tool permissions through settings files (`.claude/settings.json` for project-level, `settings.local.json` for local overrides) using 'allowedTools' and 'blockedTools' arrays. These support glob patterns for MCP tool names (e.g., `mcp_server_*` to allow all tools from a specific server). Option A conflates documentation with configuration. Option C doesn't control Claude Code permissions. Option D doesn't match the persistent model.

Question 279:

A development team is implementing context awareness (citations) for a RAG application. They want to display inline citations like 'According to [Source A], the market grew by 15%.' The API returns citation objects with character-level indices. How should the client render these?

- A. Use the citation indices to insert reference markers into the text, linking each marker back to the specific passage in the source document
- B. Append all citations as footnotes at the end of the response
- C. All of the above are valid rendering approaches depending on UX requirements

D. Create a sidebar with source highlights that appear when hovering over cited text

Answer : C

Explanation:

Context awareness provides character-level citation data that supports multiple rendering approaches: inline reference markers, footnotes, interactive sidebars, or highlighted source passages. The choice depends on the application's UX requirements. The API provides the raw data (character indices to source locations), and the client decides the presentation. All three approaches are valid uses of the citation data.

Question 280:

A developer's agent needs to process CSV files uploaded by users. The files vary from 100 rows to 1 million rows. For small files, direct context inclusion works well. For large files, the context can't hold all data. What scalable architecture handles both cases?

- A. Always process the full CSV regardless of size, relying on compaction if needed
- B. Always chunk the CSV into 100-row segments and process sequentially
- C. For small files (<5K rows): include directly in context. For large files: load into a database and provide SQL query tools for the agent to explore the data on demand
- D. Convert all CSVs to summary statistics before processing

Answer : C

Explanation:

A size-based routing strategy handles both cases efficiently. Small files work well in context (full data visibility, no tool overhead). Large files are loaded into a queryable database, giving the agent on-demand access without context overflow. Option A can't handle 1M rows. Option C is inefficient for small files. Option D loses individual record access.

Question 281:

Your team is designing an agent evaluation framework. They need to test whether the agent handles tool failures gracefully across 50 different error scenarios. Running each scenario manually takes 10 minutes. What is the most efficient approach?

- A. Create a comprehensive test prompt that covers all 50 scenarios in one conversation
- B. Test 10 representative scenarios and extrapolate for the remaining 40
- C. Hire a QA team to manually test each scenario
- D. Use the Message Batches API to run all 50 scenarios as a batch — each scenario is a separate request with a mocked tool failure, and results are analyzed programmatically

Answer : D

Explanation:

The Message Batches API enables automated, parallel evaluation of all 50 scenarios. Each scenario is a separate request with specific tool failure conditions, and results are analyzed programmatically for expected error handling behavior. This is both cost-effective (50% savings) and comprehensive. Option A can't isolate individual scenarios. Option C is expensive. Option D misses edge cases.

Question 282:

A developer builds an MCP tool that generates images using Stable Diffusion. The tool takes a prompt and returns a base64-encoded image. Claude then describes the image to the user. However, Claude's descriptions don't match the generated images because Claude can't actually see the tool output as an image. How should this be architected?

- A. Have Claude generate the image description from the original prompt without seeing the actual image
- B. Have the tool return a text description of what it generated alongside the image
- C. Store the image and return a URL that Claude mentions to the user
- D. Include the base64 image in a subsequent message with image content type so Claude can actually see and describe it

Answer : D

Explanation:

To have Claude accurately describe a generated image, the image must be sent in a subsequent message using the image content type (source: {type: 'base64', media_type: 'image/png', data: '...'}). This allows Claude's vision capabilities to actually analyze the generated image. Option B is less accurate than direct vision. Option C might work but doesn't leverage Claude's vision. Option D is unreliable.

Question 283:

A team maintains a Claude Code project with a growing CLAUDE.md file that's now 800 lines. Claude Code seems to be ignoring rules defined in the latter half of the file. What is the recommended approach for managing large CLAUDE.md files?

- A. Delete rules that seem to be ignored — they're probably unnecessary
- B. Increase Claude's context window to ensure the full file is processed
- C. Convert the CLAUDE.md to a more compact YAML format
- D. Split into a concise root CLAUDE.md with the most critical rules, use @import for detailed sections, and use .claude/rules/ files for context-specific rules with glob patterns

Answer : D

Explanation:

Large CLAUDE.md files should be decomposed: keep the root CLAUDE.md concise with the most important rules, use @import for detailed sections (like API conventions or testing standards), and use .claude/rules/ for context-specific rules that load only when relevant files are being edited. This ensures important rules get attention. Options B-D don't address the root cause.

Question 284:

A solutions engineer is troubleshooting an agent that uses the Agent SDK. The agent should use the 'search' tool before answering questions, but it frequently answers from training knowledge without searching. The tool is properly defined and the system prompt says to 'use the search tool to find answers.' What is the most effective fix?

- A. Set tool_choice: {'type': 'tool', 'name': 'search'} for the first turn to force the initial search, then switch to 'auto' for subsequent turns
- B. Rewrite the system prompt to be more emphatic: 'ALWAYS use the search tool'
- C. Set tool_choice: 'any' to force tool use on every turn
- D. Add a PreToolUse hook that blocks non-search responses

Answer : A

Explanation:

Forcing the specific search tool on the first turn ensures the agent always searches before answering. Switching to 'auto' for subsequent turns allows natural follow-up behavior (the agent can answer from search results or do additional searches). This is more targeted than forcing any tool on every turn (Option A) and more reliable than prompt emphasis (Option B). Option D operates at the wrong level.

Question 285:

A developer has a prompt engineering challenge: they need Claude to fill in a form that has 20 fields, but typically only 5-8 fields are relevant for any given request. The remaining fields should be null. Using structured output with all 20 fields as required causes Claude to hallucinate values for irrelevant fields. What schema design fixes this?

- A. Make all 20 fields optional (nullable) with clear descriptions indicating when each field applies
- B. Use a dynamic schema generated per-request based on the input context
- C. Keep all fields required but add a 'confidence' field alongside each to flag uncertain values
- D. Create 4 different schemas for different form types, each with only the relevant required fields

Answer : B

Explanation:

A dynamic schema generated per-request ensures only relevant fields are included, preventing hallucination of irrelevant values. The application logic determines which fields apply based on the

request context and generates a schema with only those fields. Option A still risks null vs. genuinely-null confusion. Option B limits flexibility. Option C doesn't prevent hallucination.

Question 286:

A technical lead is migrating their agent from a 200K token context model (Claude Sonnet 4.5) to a 1M token context model (Claude Sonnet 4.6). They expect to handle longer documents. What other consideration should they account for with the larger context?

- A. The 1M context model has the same pricing per token, so costs only increase proportionally to usage
- B. Prompt caching minimum thresholds differ between models — Sonnet 4.5 has 1024 token minimum while Sonnet 4.6 has 2048 token minimum, affecting caching strategy
- C. The architecture doesn't need any changes — just update the model parameter
- D. The response quality is identical regardless of context length

Answer : B

Explanation:

When migrating between model versions, caching minimums may change (Sonnet 4.5: 1024 tokens, Sonnet 4.6: 2048 tokens). This means cache breakpoints that worked with the smaller threshold may need adjustment. Additionally, attention patterns may differ with very long contexts, requiring prompt structure adjustments. Options A, C, and D oversimplify the migration.

Question 287:

A team is building an MCP server that integrates with Salesforce. Their tool catalog includes: search_contacts, get_contact_by_id, update_contact, create_opportunity, get_opportunity_pipeline, update_opportunity_stage, generate_report, and send_email. This is 8 tools. During testing, Claude occasionally picks the wrong tool (e.g., search_contacts when get_contact_by_id is more appropriate). What improves tool selection accuracy?

- A. Add detailed descriptions that clearly differentiate each tool's purpose with usage guidance: e.g., 'search_contacts: Use when you need to FIND contacts by name, email, or company. NOT for retrieving a known contact (use get_contact_by_id instead)'
- B. Reduce to 4 tools by combining related operations
- C. Add a tool_router tool that Claude calls first to determine which tool to use
- D. Rename tools with numbered prefixes: tool1_search, tool2_get, etc.

Answer : A

Explanation:

Tool descriptions are the primary mechanism Claude uses for tool selection. Detailed descriptions that include positive use cases ('use when...') and negative guidance ('NOT for...') significantly improve selection accuracy. This is especially important for semantically similar tools like search_contacts vs

get_contact_by_id. Option A loses granularity. Option C adds unnecessary indirection. Option D doesn't help with semantics.

Question 288:

A developer wants to add a keyboard shortcut in Claude Code that switches between regular mode and plan mode during interactive sessions. What is the default keyboard shortcut for toggling plan mode?

- A. Shift+Tab to toggle between plan mode and regular mode
- B. Tab to cycle between modes
- C. Ctrl+P to toggle plan mode
- D. Escape to enter plan mode

Answer : A

Explanation:

Shift+Tab is the keyboard shortcut to toggle between plan mode and regular mode in Claude Code. Plan mode enables Claude to outline its approach before executing, providing an opportunity for review and modification. Option A is not the correct shortcut. Options B and D reference different keyboard behaviors.

Question 289:

An AI engineer is implementing an evaluation framework for their agent system. They want to measure whether the agent asks for clarification when a customer's request is ambiguous rather than making assumptions. What evaluation metric captures this behavior?

- A. Response accuracy — measuring whether final answers are correct
- B. Clarification rate — measuring the percentage of ambiguous inputs where the agent asks for clarification before acting, compared against a ground truth set of inputs that require clarification
- C. Response time — faster responses indicate the agent made assumptions
- D. Token efficiency — lower tokens mean more decisive responses

Answer : B

Explanation:

A clarification rate metric compares the agent's behavior against a labeled dataset of ambiguous inputs that should trigger clarification questions. This directly measures whether the agent appropriately identifies ambiguity and seeks clarification rather than making assumptions. Options A, C, and D measure different aspects that don't capture clarification behavior.

Question 290:

A developer is using Claude's structured output to generate JSON configuration files. They need the output to include comments explaining each configuration option. JSON doesn't support comments. What is the best structured output approach?

- A. Use json_object mode which allows comments in the output
- B. Use YAML mode instead of JSON for comment support
- C. Generate the JSON and comments in separate API calls
- D. Generate a schema with parallel fields: each config option has a 'value' field and a 'comment' field, e.g., {port: {value: 8080, comment: 'Server listening port'}}

Answer : D

Explanation:

A parallel field schema captures both the configuration value and its documentation in a structured way. The application can then generate the final config file in any format (JSON5 with comments, YAML, or separate documentation). Option A — json_object mode doesn't add comment support. Option C loses the relationship between values and comments. Option D — there's no YAML mode in structured outputs.

Question 291:

A platform engineer is building a system where multiple Claude agents share information through a centralized message bus. Agent A publishes analysis results, Agents B and C consume results relevant to their domains. The messages include metadata, status, and data payloads. What is the key consideration for the message format?

- A. Use the same structured format that each consuming agent can parse without additional tool calls, with clear metadata for routing and versioning
- B. Use plain text messages for maximum compatibility
- C. Use different formats per agent pair for maximum efficiency
- D. Let each agent define its preferred input format

Answer : A

Explanation:

Standardized structured messages with metadata (agent ID, timestamp, message type, version) enable efficient routing and consumption without format translation. Each agent can extract relevant fields without additional processing. Option A lacks structure. Option C increases complexity. Option D creates incompatibilities.

Question 292:

A developer is configuring Claude Code for a project that uses both JavaScript and Rust. The JavaScript code follows Prettier formatting while Rust follows rustfmt conventions. How should CLAUDE.md handle this dual-language setup?

- A. Include both formatting conventions in the root CLAUDE.md with clear section headers
- B. Create separate CLAUDE.md files in the js/ and rust/ directories
- C. Use .claude/rules/ with glob-based rules: one file with `**/*.js,**/*.ts` pattern for Prettier rules, and another with `**/*.rs` pattern for rustfmt rules
- D. Rely on the language-specific formatters and don't include formatting rules in CLAUDE.md

Answer : C

Explanation:

The `.claude/rules/` glob pattern system automatically loads the appropriate formatting rules based on the file being edited. JavaScript/TypeScript files trigger the Prettier rules, Rust files trigger the rustfmt rules. This is cleaner than mixing rules in a single file (Option A) and works even outside dedicated directories (Option C). Option D misses the opportunity to guide Claude's code generation.

Question 293:

A senior developer is optimizing their token costs. They discover that 60% of their API cost comes from input tokens (most content is repeated context). Only 15% comes from output tokens, and 25% from thinking tokens. What optimization provides the biggest cost reduction?

- A. Reduce output length with `max_tokens` to save on the 15% output cost
- B. Reduce thinking budget to save on the 25% thinking cost
- C. Switch to a smaller model that has lower per-token costs across all categories
- D. Implement prompt caching to save on the 60% input cost — cache reads cost only 10% of base input price

Answer : D

Explanation:

Prompt caching provides the highest impact because it targets the largest cost component (60% of total). With cache reads at 10% of base input price, caching can reduce the input portion by up to 90%, translating to a ~54% total cost reduction. Options A and B target smaller cost components. Option D requires quality evaluation.

Question 294:

A team builds an MCP tool that manages Kubernetes clusters. The tool includes operations like `scale_deployment`, `delete_pod`, and `drain_node`. These are high-impact operations that should require confirmation. How should the tool design handle dangerous operations?

- A. Add 'DANGEROUS:' prefix to tool descriptions and rely on Claude to warn the user
- B. Separate dangerous tools into a different MCP server that requires admin credentials
- C. Require the user to type 'CONFIRM' before each dangerous operation
- D. Implement a two-phase execution: the tool first returns a preview of what will happen (dry run), and requires a second call with a confirmation token to execute

Answer : D

Explanation:

Two-phase execution (preview → confirm → execute) provides a programmatic safety layer for dangerous operations. The first call shows the impact (e.g., '3 pods will be terminated'), and execution only proceeds with a confirmation token from the second call. This is deterministic and doesn't rely on the model for safety. Options A and C rely on prompt-level safety. Option D adds infrastructure complexity.

Question 295:

A developer is implementing streaming for their Claude integration. They want to display a 'typing' indicator while Claude is generating thinking blocks, and switch to displaying text when the response text starts. Which streaming events signal this transition?

- A. A 'phase_change' event indicates the switch from thinking to text generation
- B. The content_block_stop event for the thinking block followed by content_block_start for the text block signals the transition
- C. The message_delta event includes a 'current_phase' field
- D. Thinking and text generation happen in separate streaming connections

Answer : B

Explanation:

The streaming protocol uses content_block_start and content_block_stop events to delineate blocks. When the thinking block ends (content_block_stop) and a text block begins (content_block_start with type: 'text'), the client can switch from the 'thinking' UI state to the 'responding' UI state. Options A and C reference non-existent events. Option D is architecturally wrong.

Question 296:

An architect wants their agent to handle a multi-language customer base (English, Spanish, Japanese). The system prompt is in English. When a customer writes in Japanese, the agent should respond in Japanese. System prompt instructions and tool results are in English. How should the translation be handled?

- A. Keep the system prompt and tools in English — Claude naturally responds in the customer's language. Tool results in English are understood by Claude and the response is generated in the detected language
- B. Translate the system prompt into each language and use the matching one per request
- C. Use a translation MCP tool to convert everything to the customer's language before Claude processes it
- D. Create separate agent configurations per language

Answer : A

Explanation:

Claude naturally detects and mirrors the user's language. The system prompt and tool results can remain in English — Claude understands English-language configuration and tool output, then generates the customer-facing response in the detected language. No translation infrastructure is needed. Option A is unnecessary maintenance. Option C adds latency and cost. Option D is operationally complex.

Question 297:

A team implements an agent with the Agent SDK. They want to add observability: logging each tool call with timing, inputs, outputs, and model decisions. The Agent SDK uses `setting_sources` for configuration. Where should they implement the logging?

- A. In the system prompt: 'Log every tool call you make with its parameters and timing'
- B. Enable debug mode in the Agent SDK to get automatic logging
- C. Wrap each tool handler with a logging decorator that captures inputs, outputs, execution time, and errors before returning results to the Agent SDK
- D. Add a 'log' tool that Claude calls after every operation

Answer : C

Explanation:

Wrapping tool handlers with logging decorators provides comprehensive observability without affecting agent behavior. Each tool invocation is logged with timing, inputs, outputs, and errors at the orchestration layer. This is transparent to both Claude and the tools. Option A relies on the model for logging. Option C may not provide the granularity needed. Option D wastes tool call tokens.

Question 298:

A developer needs to understand how Claude handles system prompt instructions vs. user instructions when they conflict. A system prompt says 'Always respond in formal English' but a user says 'talk to me casually bro.' What is the expected behavior according to Claude's instruction hierarchy?

- A. User instructions always override system prompt instructions

- B. Claude will average the two styles and respond in semi-formal language
- C. System prompt instructions take precedence over user instructions — Claude should maintain formal English
- D. The most recent instruction takes precedence regardless of source

Answer : C

Explanation:

Claude's instruction hierarchy places system prompt instructions (from the operator/developer) above user instructions. When they conflict, system prompt instructions take precedence as they represent the application's intended behavior. The developer set formal English for a reason, and user requests shouldn't override this constraint. Option A inverts the hierarchy. Options C and D don't match the actual behavior.

Question 299:

Your organization is building a document review agent that processes hundreds of contracts daily. Each contract review generates a structured report. They want to measure the quality of reviews over time to detect drift or degradation. What evaluation architecture supports continuous quality monitoring?

- A. Randomly sample 5% of reviews for manual human evaluation weekly
- B. Run every review through a second Claude evaluation pass that scores quality on predefined dimensions
- C. Compare each review's structure to a template and flag deviations
- D. Use a combination: automated structural checks on 100% of reviews plus LLM-based evaluation on 10% sample plus human review of flagged outliers

Answer : D

Explanation:

A tiered evaluation architecture provides comprehensive coverage: automated structural checks (fast, 100% coverage), LLM-based semantic evaluation (deeper, sampled), and human review for outliers and calibration. This balances thoroughness with cost and catches different types of quality issues. Option A is too sparse. Option B is expensive at 100%. Option C only catches structural issues.

Question 300:

A developer has built their complete agent system and needs to do a final review before production deployment. According to the exam guide's reliability checklist, which of the following is NOT a recommended pre-deployment check?

- A. Verify that all safety-critical rules are enforced via hooks/code rather than only via prompts
- B. Confirm that rate limiting and circuit breakers are configured for all external tool calls
- C. Ensure 100% test coverage of all possible user input combinations
- D. Validate that compaction preserves critical state information through custom instructions

Answer : D